

Management and Analysis of Physics Datasets

Module B

Data management, distributed processing and streaming

Alvaro Mendez

2026-10-06



Table of contents

Data Management	1
Foreword	1
I. Data Management	3
1. From Data to Datasets	5
1.0.1. Data Pipeline Stages	6
1.0.2. Monte Carlo Simulations as a Data Source	6
1.1. Metadata	7
1.1.1. CMS Metadata Example: Path-as-Metadata	7
1.2. Data Structures	8
1.2.1. Structured Data	8
1.2.2. Semi-Structured Data	9
1.2.3. Unstructured Data	10
1.3. Summary: Data Structure Types	11
1.4. What is Data Management?	11
1.5. Exercises	12
2. Storage	15
2.1. Memory Devices	16
2.1.1. DRAM (Dynamic Random Access Memory)	16
2.1.2. NVM (Non-Volatile Memory) / Persistent Memory	17
2.1.3. SSD (Solid State Drives)	17
2.1.4. HDD (Hard Disk Drives)	18
2.1.5. Magnetic Tapes	19
2.2. Kryder's Law	20
2.3. Scalability	21
2.4. Storage Media at a Glance	22
2.5. Measuring Storage Performance	22
2.5.1. <code>fio</code> — Production-Grade I/O Benchmarking	23
2.6. Exercises	23
3. Reliability and Security	27
3.0.1. Failure Rate Lifecycle: The Bathtub Curve	27
3.1. Reliability Strategies	28
3.1.1. RAID 0	29
3.1.2. RAID 1	29
3.1.3. RAID 1+0	30
3.1.4. RAID 5	30
3.1.5. RAID 6	31
3.2. Cryptography	33
3.3. Authentication and Authorization	34
3.4. Encoding and Decoding of Information	35
3.5. Hash Functions	37

Table of contents

3.6. Encryption vs Hashing: Summary	38
3.7. Exercises	38
4. File System	47
4.1. Characteristics	47
4.2. File Access Modes	48
4.3. Space Allocation	48
4.3.1. Contiguous Allocation	48
4.3.2. Linked Allocation	49
4.3.3. Indexed Allocation	49
4.4. Limits	50
4.5. Common File Systems Compared	51
4.6. Exercises	52
5. Distributed File System	55
5.1. Operation	55
5.2. Example: HDFS	56
5.3. CAP Theorem	58
5.4. Exercises	60
6. Databases	69
6.1. Database Models	69
6.2. Relational Databases	69
6.2.1. Transaction Control in SQL	70
6.2.2. Transaction Isolation Levels	71
6.3. SQL	71
6.3.1. Creating Databases and Tables	72
6.3.2. Keys	72
6.4. Relationships in Relational Databases	74
6.4.1. One-to-One (1:1)	74
6.4.2. One-to-Many (1:N)	74
6.4.3. Many-to-Many (N:M)	75
6.4.4. Constraints	75
6.4.5. Altering Tables	76
6.4.6. Data Insertion	77
6.4.7. Modifying Records	78
6.4.8. Deleting Records	78
6.4.9. Conditional Expressions: CASE	78
6.4.10. Data Selection	79
6.4.11. Other JOIN Types	81
6.5. Subqueries	83
6.5.1. Data Control (DCL)	83
6.6. Exercises	84
6.6.1. Conceptual Questions	84
6.6.2. ACID Questions	86
6.6.3. Schema Design Exercises	88
7. NoSQL Databases	93
7.1. Database Partitioning	93
7.1.1. Consistent Hashing	94
7.2. Characteristics of NoSQL Databases	97
7.2.1. BASE vs ACID	97

7.3. NoSQL Database Categories	98
7.4. Examples of NoSQL Databases	98
7.4.1. Key-Value Databases	98
7.4.2. Document Databases	100
7.4.3. Time Series Databases	100
7.4.4. FullText Search Engine	100
7.5. Wide-Column Databases	100
7.6. Graph Databases	101
7.7. Vector Databases	101
7.8. Exercises	102
 II. Processing	 105
8. Processing and Concurrency	107
8.1. Introduction to Data Processing	107
8.2. CPU Processing	107
8.2.1. Processing Time	107
8.2.2. Historical Context: Moore's Law	108
8.2.3. The Power Wall	108
8.2.4. Pipelining and Alternative Strategies	109
8.3. Multi-Processing Architectures	109
8.4. Parallelism	110
8.5. Flynn's Taxonomy	110
8.6. Speedup: Amdahl's Law	111
8.7. Gustafson's Law	111
8.8. Processes and Threads	111
8.8.1. Definitions	111
8.8.2. Concurrency vs Parallelism	112
8.8.3. Multi-Tasking	112
8.8.4. Multi-Threading	112
8.9. Race Conditions	113
8.10. Thread Synchronisation	113
8.11. The Python GIL	113
8.12. Threading vs Multiprocessing in Python	114
8.13. Hands-on	115
9. Distributed Processing and MapReduce	117
9.1. Scaling Processing	117
9.2. Parallel Computing Architectures	117
9.2.1. Shared Memory Multiprocessor (SMP)	117
9.2.2. Distributed Memory Multiprocessor (Cluster)	118
9.3. Distributed Computing: Clusters	118
9.3.1. Supercomputers as Clusters	119
9.4. Batch Systems	119
9.5. Programming Frameworks for Parallel-Distributed Systems	120
9.6. Big Data and the 3 Vs	120
9.7. Hadoop	121
9.8. HDFS: A Reminder	121
9.9. The MapReduce Paradigm	122
9.9.1. 1. Map	122
9.9.2. 2. Reduce	122

Table of contents

9.9.3. Word Count Example	122
9.10. MapReduce in Hadoop	123
9.10.1. Map Phase	123
9.10.2. Reduce Phase	123
9.11. YARN: Yet Another Resource Negotiator	123
9.12. Hadoop Higher-Level Abstractions	123
9.13. The Hadoop Ecosystem	124
10. Apache Spark	125
10.1. Motivation	125
10.2. Spark Ecosystem	125
10.3. Spark Cluster Architecture	126
10.3.1. Driver and Executors	126
10.3.2. Cluster Resource Managers	126
10.4. RDD: Resilient Distributed Dataset	127
10.5. Transformations and Actions	127
10.5.1. Transformations	127
10.5.2. Actions	127
10.6. Laziness and the DAG Scheduler	128
10.7. Spark DataFrame	128
10.8. Real-Time Analytics and Structured Streaming	128
10.8.1. Spark Structured Streaming	129
10.8.2. Sources and Sinks	129
10.9. Hands-on	130
11. Dask	131
11.1. Introduction	131
11.2. Dask Fundamentals	131
11.2.1. Lazy Evaluation and Task Graphs	131
11.2.2. Partitioning	132
11.3. Cluster Setup	133
11.4. Hands-on	133
12. Apache Kafka	135
12.1. Introduction to Message Streaming	135
12.1.1. Core Concepts	135
12.2. Kafka Architecture	136
12.2.1. Brokers and Clusters	136
12.2.2. Consumer Offset Management	136
12.3. Delivery Semantics	136
12.3.1. Pub/Sub Decoupling	137
12.3.2. Partitions and Keys	137
12.4. Kafka and Spark Structured Streaming	138
12.5. Hands-on	138
III. Labs	139
13. Running the labs	141
13.1. Where the code is	141
13.2. What a lab looks like	141
13.3. The stacks at a glance	142

13.4. Start, use, stop	143
13.5. Three things that confuse everybody	144
13.5.1. 1. Run <code>docker compose</code> from inside the lab folder	144
13.5.2. 2. Two kinds of addresses: service names and <code>localhost</code>	144
13.5.3. 3. The notebook is not on the cluster	144
13.6. From this laptop cluster to the cloud	145
13.7. Troubleshooting	146
14. Lab: MySQL	147
14.1. The idea behind it	147
14.2. Before you start	147
14.3. 01 — Three ways to talk to the server	148
14.4. 02 — Databases and tables	149
14.5. 03 — Keys, constraints and <code>ALTER TABLE</code>	149
14.6. 04 — Inserting, updating, deleting	150
14.7. 04 — Basic queries	151
14.8. 05 — Queries on the <code>world</code> database	152
14.9. 06 — The IMDb database	153
14.10 On a real system	154
14.11 SQL String Functions Reference	155
14.12 SQL Date and Time Functions Reference	155
14.13 SQL Query Exercises	156
15. Lab: Threads and processes in Python	163
15.1. The idea behind it	163
15.2. How to run it	163
15.3. The scripts, in order	164
15.3.1. 00 — Single-Threaded Baseline	164
15.3.2. 01 — Two Concurrent Threads	164
15.3.3. 02 — Thread-Local Data	165
15.3.4. 03extra — Minimal Race Condition	165
15.3.5. 03 — Multi-Threaded Without Lock (Race Condition)	165
15.3.6. 04extra — Minimal Race Condition Fixed with Lock	165
15.3.7. 04 — Multi-Threaded With Lock	165
15.3.8. 05 — Thread Pool with Lock (<code>concurrent.futures</code>)	166
15.3.9. 06 — Thread Pool with Queue	166
15.3.10 07 — Multi-Processing	166
15.3.11 Synchronous vs Asynchronous Execution	166
15.3.12 08 — Multiprocessing Pool (<code>starmap</code> , blocking)	166
15.3.13 09 — Multiprocessing Pool (<code>starmap_async</code> , non-blocking)	167
15.4. Benchmarking: I/O-bound vs CPU-bound Tasks	167
15.4.1. I/O-Bound Tasks	167
15.4.2. CPU-Bound Tasks	167
15.5. On a real system	168
16. Lab: Spark 1, RDDs and map-reduce	169
16.1. The idea behind it	169
16.2. Before you start	169
16.3. Connecting: the <code>SparkSession</code>	170
16.4. Parallelize, partitions and laziness	170
16.5. Map and reduce	172
16.5.1. Exercises 1–2	172

16.5.2. Intermezzo: broadcast variables	173
16.5.3. Exercise 3: estimate	173
16.6. Key–value pairs, <code>reduceByKey</code> , <code>flatMap</code>	174
16.6.1. Exercise 3b: word count	174
16.7. Exercise 4: distributed linear regression	175
16.8. Caching	176
16.9. On a real cluster	176
16.10. Wrap up	176
17. Lab: Spark 2, DataFrames and the dimuon spectrum	179
17.1. The idea behind it	179
17.2. Why DataFrames	179
17.3. Creating DataFrames	180
17.4. The analysis: dimuon invariant mass	180
17.4.1. Step 0: load and inspect	181
17.4.2. Step 1: aggregate on the cluster, plot locally	181
17.4.3. Step 2: filter and flatten	181
17.4.4. Step 3: new features (native functions vs UDFs)	182
17.4.5. Step 4: rediscover the Z boson	183
17.5. On a real cluster	183
18. Lab: Spark 3, Structured Streaming	185
18.1. The idea behind it	185
18.2. The model: a stream is a table that keeps growing	185
18.3. Session	187
18.4. The source: a socket and a producer	187
18.5. First query: raw lines to the console	187
18.6. Develop on static data, then go live	188
18.7. The fraud detector	188
18.8. On a real system	189
19. Lab: Dask 1, delayed and futures	191
19.1. The idea behind it	191
19.2. Before you start	191
19.3. Dask in one paragraph	192
19.4. Lazy: <code>delayed</code>	192
19.5. Eager: <code>submit</code> , <code>map</code> and futures	193
19.6. Exercises	193
19.7. On a real cluster	195
19.8. Wrap up	195
20. Lab: Dask 2, Bags	197
20.1. The idea behind it	197
20.2. What a bag is	197
20.3. Map, filter, fold	198
20.3.1. Broadcast: <code>scatter</code>	198
20.4. Exploring the accounts data	198
20.5. Per-key aggregation: <code>groupby</code> vs <code>foldby</code>	199
20.6. From bag to DataFrame	200
20.7. On a real cluster	200

21. Lab: Dask 3, DataFrames	203
21.1. The idea behind it	203
21.2. What a Dask DataFrame is	203
21.3. Loading	204
21.3.1. Type inference pitfall	204
21.4. Cheap and expensive operations	204
21.5. Share work: one <code>dask.compute</code> for several results	206
21.6. Exercise: the dimuon spectrum in Dask	207
21.7. On a real cluster	208
21.8. Wrap up	208
22. Lab: Kafka, and Kafka + Spark	209
22.1. The idea behind it	209
22.2. Before you start	210
22.3. Kafka in one picture	210
22.4. Part 1: from the shell (on the broker)	210
22.5. Part 2: producing from Python (<code>Producer.ipynb</code>)	211
22.6. Part 3: consuming (<code>Consumer.ipynb</code>)	211
22.7. Part 4: consumer groups (<code>Consumer.ipynb</code> + <code>ConsumerGroup.ipynb</code>)	212
22.8. Part 5: Kafka as Spark source and sink (<code>KafkaSpark_StructuredStreaming.ipynb</code>)	212
22.9. On a real cluster	214
22.10 Wrap up	214

Data Management

Foreword

Disclaimer

I am **NOT** the original author of the content in this document. My contribution is limited solely to the translation and re-organization of the original material. All credit, recognition, and compliments for the ideas, structure, and substance of the work should be directed to the original creator **Original Author**.

Original Author is **NOT** responsible for the management of this version or any errors that it might contain.

No copyright

All the source code of this document is private and therefore is not public for any uses. Any feedback and/or questions related to the document should be directed to the original author of this document **Original Author**.

Contributing

If you want to contribute to this version of the notes open an issue at **my repository of the master**.

Below we will discuss the various steps that lead from data acquisition through sensors to the production of real *information*. The process can be divided into two parts:

1. **Data acquisition system**, divided into the following parts:

- **Sensors**, used to convert physical quantities into electrical signals.
- **Frontend electronics**, with the role of interface between the sensors and the signal processing system; the latter is in fact amplified and filtered in order to remove unwanted noise.
- **Readout electronics**, responsible for the digital conversion of the analog signal; it can also perform the function of temporary storage of the signal.
- **Trigger & DAQ**: the first determines when the system can acquire data, for example after exceeding a certain input threshold or after a certain time interval; the second further filters the signal (now digitized) and stores it.

2. **Calculation model**, divided into the following aspects:

- **Storage**, that is the way and the place where the data obtained from the acquisition system are physically stored.

- **Reliability and security**, that is the techniques adopted to preserve the data and the permissions to access them.
- **Querying and filtering**, that is the process of searching and selecting data; *querying* is that process by which it is possible to extract data from memory according to certain criteria.
- **Processing**, which involves the application of algorithms and calculation tools to derive useful information from the stored data.

It should be borne in mind that the one presented is a system for acquiring *physical* data; more generally, the data source can be of a very different type: it is possible to work with *simulated* data (think of the various Monte Carlo techniques), with text documents, with images and so on.

Part I.

Data Management

1. From Data to Datasets

By **dataset** we mean a set of *information* (data or simulations) collected from a *series of sources* (detectors and devices in general) in a certain *time frame* associated with a single *domain* or a single *purpose*.

The data, that is the basic unit of information contained in a dataset, is often indicated as ***datum*** or also as ***record***.

Below is illustrated the formation of a dataset:

1. A data acquisition system (**DAQ**) constantly writes to memory every piece of information that is collected by the sensors at the highest possible frequency.
2. The stored data are then subjected to a first processing, on average after a certain number of hours or days, which includes cleaning, correction of any errors, normalization and — if necessary — also dimensionality reduction.
3. Over a longer period of time (on average every year) the data are subjected to further processing involving more sophisticated techniques, such as Monte Carlo simulations.
4. The data that are processed in any period of time are long-term archived in a dedicated memory space.

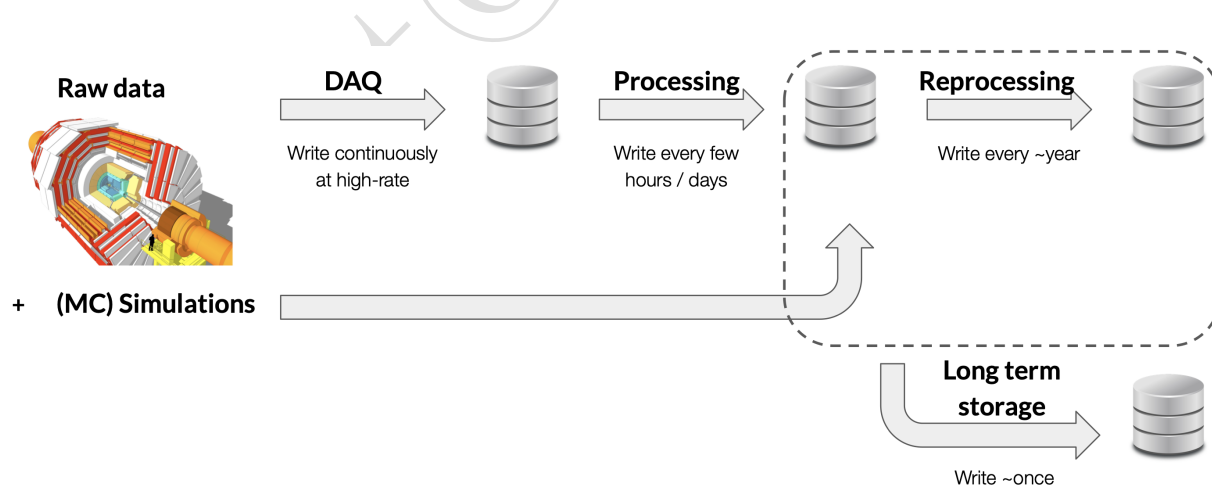


Figure 1.1.: Formation of a dataset

The acquired data sequences are usually divided according to the purpose of the collection: the division can be trivially divided by time frames (for example by year) or by data acquisition campaigns (think of the various *Runs* that are performed at the LHC); another possibility is the division by version of the software used, a choice adopted mainly for simulated data models.

1.0.1. Data Pipeline Stages

The journey from raw detector output to physics results passes through several well-defined pipeline stages. Each stage has a distinctive read/write profile that shapes the storage and computing infrastructure required.

Stage	Write frequency	Read frequency	Description
DAQ	Continuous / high-rate	Rarely read back	Raw detector data written once as sensors fire; write-heavy by design
Processing	Every few hours or days	Moderate	Calibration and reconstruction applied to DAQ output; derived datasets written
Reprocessing	Every ~1 year	High (re-reads raw)	When algorithms improve the original raw data are re-processed; read-heavy on the DAQ tier
Analysis	Infrequent	Very high	Physicists repeatedly read small derived samples; essentially read-only

A key pattern emerges: **write frequency decreases** and **read frequency increases** as data move from DAQ through to analysis. This asymmetry drives many design decisions in physics data management — raw DAQ storage must sustain high ingest rates, while analysis storage must deliver low-latency random reads.

i Write-once, read-many

Raw DAQ data are typically written *once* and subsequently only read. Long-term archives therefore optimise for storage density and read throughput rather than write speed. Tape-based systems, which excel at exactly this profile, remain dominant for DAQ-tier storage at large experiments.

1.0.2. Monte Carlo Simulations as a Data Source

Alongside real detector data, physics experiments produce large volumes of **Monte Carlo (MC) simulated data**. MC simulations generate synthetic datasets by modelling physics processes using random sampling: particles are generated according to theoretical cross-sections, propagated

through a virtual detector model, and digitised to produce output in the same format as real detector readout.

MC datasets are not a shortcut — they pass through *exactly the same processing pipeline* as real data (reconstruction, calibration, tier conversion) so that simulated and measured distributions can be compared on equal footing. This comparison is the basis of most physics analyses: deviations between observed and expected (MC) distributions reveal new phenomena or constrain theoretical parameters.

In practice, MC datasets are often **larger than the corresponding real-data samples**. Because each simulated event is statistically independent, many millions of events are needed to reduce Monte Carlo statistical uncertainty below the level of the real-data uncertainty. A single physics analysis may consume more CPU-hours of simulation than all the data-taking combined.

1.1. Metadata

Datasets are often accompanied by **metadata**, so called because they are data whose function is to give information about the data itself contained in the dataset.

They perform three main functions: identification of the dataset and its contents, description of the acquisition conditions, and indication of access methods and permissions.

To understand the importance of metadata, let's look at two very different files. The first is a generic photo of a cat (*a_nice_cat_photo.jpeg*): in this case the metadata may be superfluous, since any image viewing program will show the image; the metadata could however provide further information, such as who took the photo, in which folder it is located, what is the name of the cat, and so on.

Let's now move on to the other file: *3CBC07C2-4B64-E611-B423-02163E0124E9.root*. Without any metadata it is impossible to trace the meaning of what is contained in the file: it is, in fact, proton-proton collision data at 13 TeV that occurred in the CMS, part of a dataset collected in 2018 belonging to the *Run-D* campaign, 3 GB in size, to which only users of the CMS experiment have access. None of this would have been conceivable without data that explicitly told us what has just been written.

1.1.1. CMS Metadata Example: Path-as-Metadata

A concrete illustration of metadata in practice is the CMS (Compact Muon Solenoid) dataset naming convention used on the WLCG grid. A typical dataset path looks like:

`/CMS/Run2018D/SingleMuon/MINIAOD/UL2018_MiniAODv2-v1/`

Every component of that path encodes structured metadata:

Path component	Meaning
CMS	Experiment
Run2018D	Data-taking period (year + run era)

1. From Data to Datasets

Path component	Meaning
SingleMuon	Physics stream (events passing the single-muon trigger)
MINIAOD	Processing tier (reduced Analysis Object Data format)
UL2018_MiniAODv2-v1	Reprocessing campaign and software version

In addition, each dataset carries per-file and per-dataset metadata records stored separately, for example:

```
Run number:      325170
Luminosity section: 47
Centre-of-mass energy: 13 TeV
Dataset:         SingleMuon
Data tier:        MINIAOD
Number of files:  ~3 000
Total size:       ~12 TB
```

This design is an example of **path-as-metadata**: the file system path *itself* carries structured, human- and machine-readable metadata. It allows data management tools to locate, filter, and catalogue datasets without opening a single file, and it ensures that the provenance of every sample is unambiguous throughout its lifetime.

i Why path-as-metadata matters

When hundreds of petabytes are spread across thousands of storage nodes, scanning file contents to identify datasets is prohibitively expensive. Encoding key metadata in the path allows lightweight string-matching queries to replace full catalogue scans, dramatically reducing the cost of data discovery.

1.2. Data Structures

Datasets can be divided into three different types depending on the way they are sorted:

1.2.1. Structured Data

They are organized in a *rigid* tabular format, so that each element is “boxed” in a unique position determined by a row and a column.

In the following figure we can observe an example of structured data: on the left is shown the *raw* format (the actual bits stored in memory in hexadecimal form), while on the right the same data are in a human-readable text format.

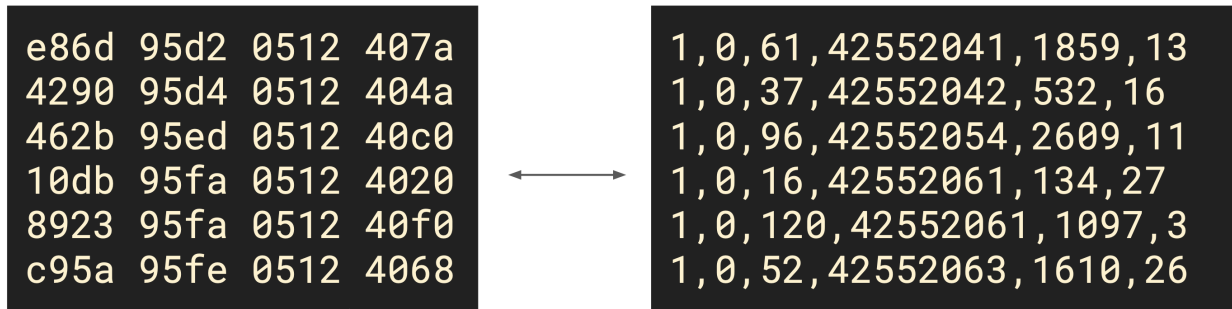


Figure 1.2.: Example of structured data

The tabular form can be displayed using various options, including the standard one offered by the Python *pandas* library.

HEAD	FPGA	TDC_CHANNEL	ORBIT_CNT	BX_COUNTER	TDC_MEAS
1	0	61	42552041	1859	13
1	0	37	42552042	532	16
1	0	96	42552054	2609	11
1	0	16	42552061	134	27
1	0	120	42552061	1097	3
1	0	52	42552063	1610	26

Figure 1.3.: Display via *pandas* of the data shown in the previous figure

Examples of structured data are files in *.csv* (*comma-separated values*) format and *RDBMS* (*Relational DataBase Management System*), also called *relational databases* suitable for the SQL language.

The main advantage of structured data is offered precisely by their structure, being already “packaged” according to a clear and established order. The disadvantage is given instead by the rigidity of this scheme, which limits the possibility of adding new columns and of acting on any errors in positioning.

1.2.2. Semi-Structured Data

They are organized according to structured logics but do not have a fixed scheme a priori, being deducible from the data themselves through *tags* and *markup*.

Examples of semi-structured data are files in *.xml* (*extensible markup language*) format, files in *.json* (*JavaScript Object Notation*) format and files in *.yaml* (*YAML ain't markup language*) format.

Example of semi-structured data in two different formats

To verify the integrity of the data it is always possible to convert semi-structured data into structured data.

1. From Data to Datasets

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>
```

(a) Format .xml

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>
```

(a) Format .json

For example, a JSON document such as:

```
{"name": "Alice", "age": 30, "city": "Bologna"}
```

can be mapped to a table row:

name	age	city
Alice	30	Bologna

Schema validation tools (e.g. JSON Schema) can enforce a structure on semi-structured data before ingestion into a relational system.

First name	Last name	Age	Address	Phone numbers	Sex
John	Smith	25			M

Street address	City	State	Postal code
21 2nd Street	New York	NY	10021

Type	Number
Home	212 555-1234

Figure 1.6.: Semi-structured data converted into structured data

1.2.3. Unstructured Data

They are not schematized through any model nor is there any structure suggested by the data themselves.

These are usually data that define a *continuous flow* of information, such as files of a textual, audio, video and image nature.

The main advantage of this type of format is its maximum flexibility, adapting well to any type of data, while the other side of the coin is the difficult and laborious work to extract single pieces of information from the dataset.

1.3. Summary: Data Structure Types

Data Type	Schema Fixed?	Suitable for RDBMS?	Typical Formats	Main Challenge
Structured	Yes — defined before storing	Yes	CSV, TSV, SQL tables	Rigidity; schema changes are costly
Semi-structured	Optional / flexible	With conversion	JSON, XML, YAML, HDF5	Heterogeneity; variable fields
Unstructured	No	No	Images, audio, text, video	Extracting information requires specialised tools

1.4. What is Data Management?

This chapter introduced the types of data we will work with throughout the course. The subsequent chapters address the full data management lifecycle:

Topic	Chapter
Data storage — media, performance metrics, scalability	Storage and Scalability
Reliability and security — RAID, encryption, authentication	Reliability and Security
File systems — local and distributed	File Systems, Distributed File Systems
Database management systems — relational and NoSQL	Databases, NoSQL Databases

Effective data management means choosing the right storage medium, ensuring data is durable and secure, and providing efficient access through appropriate file systems and database systems.

1.5. Exercises

Exercise 1.1 (*Structured vs Semi-structured Data*)

Explain the difference between structured and semi-structured data. For each, describe the main advantages and disadvantages and provide a concrete example of such data.

Solution

The difference between structured data and semi-structured data is that structured data has well defined structure (like columns in an excel sheet) while semi-structured data does not have a structure but it has a set of rules (like json or yaml files), not all files will have the same disposition of the data but all files can be read in the same way

The main advantage of structured data are that:

- It is easy to read many files as they all have the same structure
- It has clear structure and order

The main disadvantages are:

- It is difficult to add new data (because of the rigid structure)
- It is less human readable

In the case of semi-structured data, the main advantages are:

- They can be used in more complex type of data
- They can be mapped in to a structured format in some cases

The disadvantages are:

- It is harder to keep them consistent as the schema is not fixed
- Each record can carry its own keys, so parsing, querying and validating cost more than for a fixed schema

A typical structured example is a CSV or SQL table; a typical semi-structured example is JSON or YAML.

Exercise 1.2 (*Metadata*)

What data management concept is key to interpreting and contextualising individual records in a dataset? Define the concept in detail and explain why it is essential.

Solution

The key concept is metadata, this can be present in different ways, but the most common is at the beginning of the file, and is there to describe the data. It helps to understand what the data is about, it can give us information about what the data is, how it has been treated, if there is missing information. It helps us understand the origin and the nature of the dataset, otherwise they are just numbers in a selected format.

Exercise 1.3 (*Types of Metadata*)

Name at least two different types of metadata. Describe metadata you have already encountered in your physics experience and explain how you used it to interpret data.

Solution

Metadata can be described in a separate file, in the same file at the beginning, the name of the file, etc. Basically anywhere where we can get information of the data. Two types that I have seen:

- Metadata in the path: When we run an experiment under different conditions I have seen how all the csv files are saved under different folders with different names describing the conditions of the experiments. Imagine an experiment where we vary the Temperature, Pressure and Volume of a gas, then I take measurements of the temperature for different pressure and volume conditions so each file lives in a folder named `P_{ }_bar_V_{ }.ml` where in the brackets is placed the values of pressure and volume of that run.
- Metadata in a separate file: I have seen many times that when someone sends me data they also share an auxiliary file (in markdown or yaml format) describing what each column is, in what units the measurement was taken and how it has been treated. This is important as maybe the name of a column is not self-describing, it can be named `T` and without the file you can not know if it is temperature or period if it's measured in kelvins or celsius.

2. Storage

Each storage unit plays a different role, although the purpose was always to keep the data received as input in memory:

- The DAQ writes to a memory at very high frequencies, performing several write cycles in a short time; however, this data will not remain in that memory unit for long — it will be processed and sent elsewhere, freeing up space for new information from the data acquisition system.
- The processed data are distributed in two different units: in both cases the writing takes place in much wider time frames, and at the same time the amount of memory used grows considerably.
- The same processed data are copied to a further memory, which must therefore be of huge dimensions but from which not many read cycles are expected.

We therefore notice *different needs* regarding the memory units: a decreasing number of read and write operations corresponds to an increasing amount of available space. These different needs translate into *different technologies* suitable to satisfy them.

These technologies are distinguished by several aspects, all related to **I/O** (input/output) requests — read, write, update and delete:

- **Latency:** time taken to perform a single I/O
- **Frequency:** number of I/Os performed per second (**IOPS**, *I/O per second*)
- **Block size:** size in bytes of a single I/O
- **Throughput:** product of *block size* and *frequency*, i.e. the amount of bits transferred in a certain time interval, conventionally measured in MB/s

i Bandwidth vs Throughput

Bandwidth is the *theoretical maximum* capacity of a storage channel — the ideal rate under perfect conditions. **Throughput** is the *actually measured* transfer rate in practice, which is always lower due to overhead, contention, and access patterns. Benchmarking tools measure throughput; vendor datasheets typically quote bandwidth.

Example 2.1 (*and Throughput*)

Given a storage device with the following characteristics: - Block size: 4 kB - Average access latency: 11.1 ms

$$\text{IOPS} = \frac{1}{0.0111 \text{ s}} \approx 90 \text{ ops/s}$$

2. Storage

$$\text{Throughput} = 90 \times 4 \text{ kB} \approx 360 \text{ kB/s}$$

This is typical of a spinning HDD under random-access workloads. For large sequential reads the same HDD may achieve 100–200 MB/s by amortising seek and rotation latency across large blocks.

i $\text{Throughput} = \text{IOPS} \times \text{Block Size}$

The two metrics are linked by a simple identity:

$$\text{Throughput} = \text{IOPS} \times \text{Block size}$$

For example, a device sustaining **1 000 IOPS** with **128 kB blocks** delivers:

$$1,000 \times 128 \text{ kB} = 128 \text{ MB/s throughput}$$

This formula highlights a fundamental trade-off in storage tuning:

- **Maximising throughput** favours large sequential I/Os (big block size, fewer individual operations) — typical of streaming workloads such as video ingest or bulk data analysis.
- **Maximising IOPS** favours many small random I/Os (small block size, high operation count) — typical of database transaction workloads or metadata-heavy file systems.

Optimising for one goal generally comes at the expense of the other: a device configured for peak IOPS (small blocks, high parallelism) will show lower throughput than the same device streaming large sequential blocks, and vice versa.

2.1. Memory Devices

The various devices are listed from the one with the lowest latency to the one with the highest latency.

2.1.1. DRAM (Dynamic Random Access Memory)

It stores data bits in **NAND cells**, composed of a transistor connected to the bus line coupled with a capacitor.

The term *RAM* indicates that it is possible to access data in any position at the same speed; the term *dynamic* specifies that the data do not remain in memory persistently — when the power to the device is interrupted all the data are lost.

Its main flaw lies precisely in this characteristic, which makes it a *temporary storage*; at the same time, its main advantage is the speed in executing I/O operations, with a latency of $\sim 0.1 \mu\text{s}$. DRAMs are used for processes that require a very high writing frequency.

2.1.2. NVM (Non-Volatile Memory) / Persistent Memory

An alternative to DRAM that has a similar latency ($\sim 1 \mu s$) but keeps the data saved even when the device's power is cut off. These qualities require a very high cost: for this reason, devices of this type are not used in standard applications, but in specific projects that require devices with these characteristics.

Unlike DRAM, NVM is **byte-addressable** and **non-volatile** — data survives a power loss just as it does on an SSD, while still being accessible at near-DRAM speeds. A well-known commercial example is **Intel Optane** (based on 3D XPoint technology), which sits between DRAM and NVMe SSD in the storage hierarchy:

Property	DRAM	NVM / Persistent Memory	NVMe SSD
Latency	$\sim 100 \text{ ns}$	$\sim 1 \mu s$	$\sim 100 \mu s$
Persistence	Volatile	Non-volatile	Non-volatile
Byte-addressable	Yes	Yes	No
Cost/GB	Very high	High	Medium–high

Typical use cases include: - **High-speed persistent cache**: absorbing bursty writes before flushing to slower storage, without the risk of data loss on power failure. - **In-memory databases**: replacing DRAM as main memory in latency-sensitive applications (e.g., financial trading, real-time analytics) where durability is also required.

i NVM in the Storage Hierarchy

NVM fills the gap between volatile DRAM and durable SSDs. It is faster than SSD, slower than DRAM, and — unlike DRAM — survives a power loss. This makes it attractive wherever both low latency *and* persistence are required simultaneously.

2.1.3. SSD (Solid State Drives)

Due to their depreciation over time, SSDs have become widespread even for the general public, being included in common personal computers.

The novelty lies in the way the information is stored: no longer in NAND cells but in **multi-level charge traps** — capacitors that distinguish between multiple charge levels. In most devices these levels are eight, thus encoding a total of three bits of information.

2. Storage

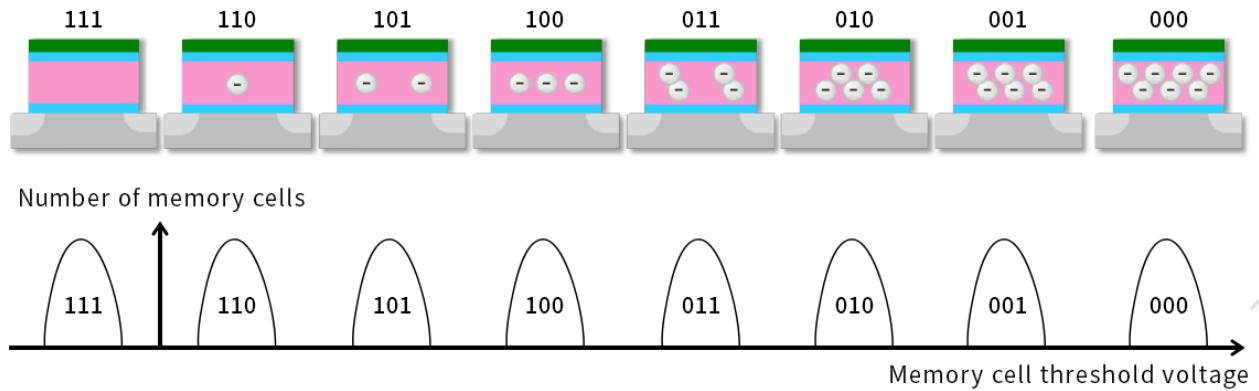


Figure 2.1.: TLC technology: at the top the encoding of the various charge states in three bits; at the bottom the voltage *thresholds* that define the eight states

This technology, called *TLC* (*Triple-level cell*), allows to expand the *storage density*, resulting in much more compact devices with the same memory. On the other hand, it limits the writing capacity: every time it is necessary to write a new piece of information, the entire cell must be erased first. These are therefore devices with a high reading frequency ($\sim 10 \mu s$) but with a considerably lower writing frequency ($\sim 100 \mu s$).

Another limitation lies in the inevitable process of hysteresis that the various cells undergo in the writing cycles: each cycle corresponds to a small residual charge that no longer allows correct operation beyond a certain accumulation. Every SSD has an “expiration date” foreseen from the beginning, the closer the more frequent the use.

2.1.4. HDD (Hard Disk Drives)

They are the most widespread memory devices today, being an excellent compromise in terms of cost relative to their performance.

An HDD is formed by disks magnetized on both sides whose domains are distributed along concentric circles: the individual magnetic domains are called *tracks*, which belong to *sectors* in turn distributed on the side of a disk, called a *platter*. The bits are not associated each with a single track but with two consecutive tracks — bit 0 corresponds to two tracks with the same polarization and bit 1 to two tracks with opposite polarization.

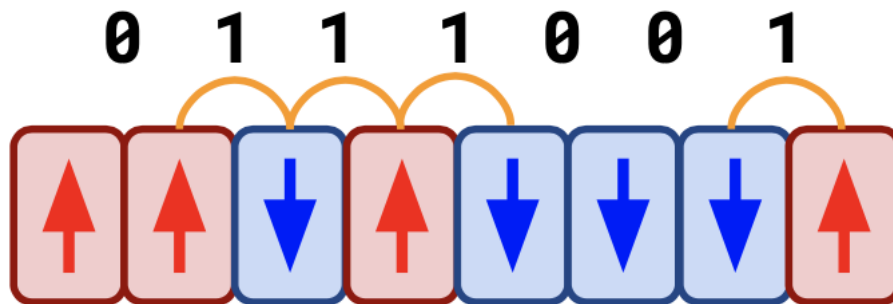


Figure 2.2.: Encoding of bits as a function of the variation of polarization between two contiguous tracks

Data reading does not take place by track but by sector, which usually contain from 0.5 to 4 kB, so even writing to disk is broken down according to this block size.

Reading and writing takes place through mechanical arms, which read the various sectors as a stylus does with a turntable. The limits in reading and writing speed depend on the mechanical nature of the device. The latency ($\sim$ ms) can be divided into two contributions:

- **Seek time:** time taken to align the arm to the track
- **Rotation delay:** time taken by the disk to rotate by aligning itself with the sector

2.1.5. Magnetic Tapes

Magnetic tapes use the same technology as VHS cassettes: magnetic domains arranged one after the other on a tape that can be wound into a compact form. They are designed for data that will not be retrieved except rarely and after a long time. All large scientific projects have a copy of all their data in the form of tapes — CERN currently stores almost 1 EB (10^3 PB) in tape form.

Reading takes place through robotic arms that pick up the requested tape and carry it to the reader. The latency order of magnitude is ~ 100 s, with a wide range due to high randomness.

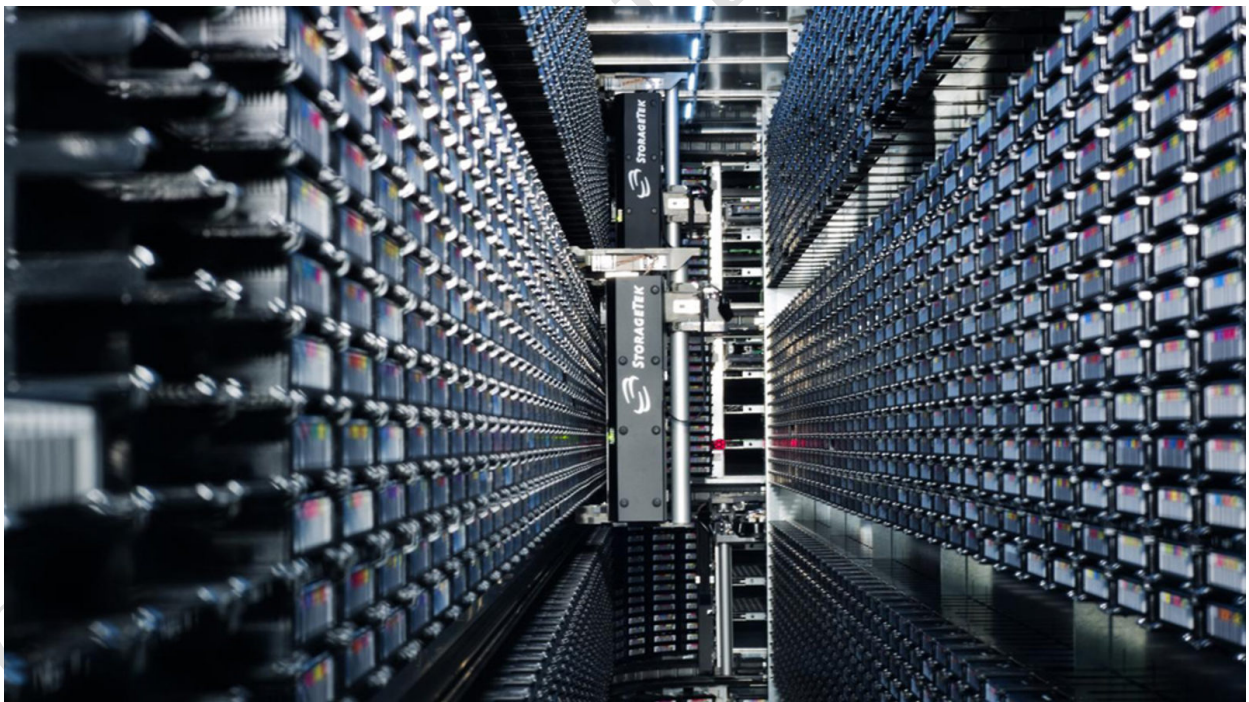


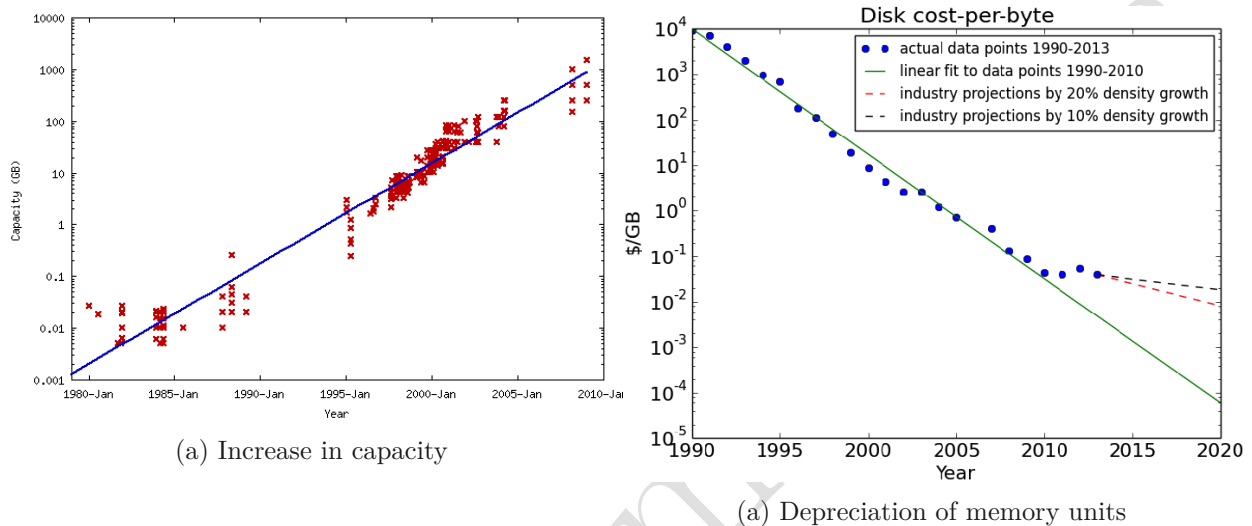
Figure 2.3.: Magnetic tape library at CERN

The reading itself is very fast — the high latency is exclusively attributable to the mechanical retrieval process described above. The main advantages of this format are its low production cost and its robustness: even in extreme cases such as flooding, the data would remain intact.

2. Storage

2.2. Kryder's Law

A factor to be taken into account for the long-term storage of data is the constant development of technologies in this sector. Analogous to Moore's Law for transistors, **Kryder's Law** (Mark Kryder, 2005) states that storage density doubles roughly every 13 months — implying that the cost per GB halves on a similar timescale. A side effect is the constant *depreciation* over time of the same amount of memory.



Kryder's Law: capacity growth and memory depreciation over time

⚠ Kryder's Law Has Significantly Slowed

The original 2005 prediction has **not held for well over a decade**. Areal density improvements in HDDs — the primary driver of the law — stalled around 2010 as magnetic bit cells approached fundamental physical limits. Progress since then has relied on incremental engineering advances (heat-assisted magnetic recording, shingled magnetic recording) rather than the exponential density scaling Kryder envisioned.

Practical consequences:

- The **cost-per-GB gap** between HDD and flash/SSD has narrowed far more slowly than Kryder's Law predicted.
- **Magnetic tape** remains cost-competitive for cold storage at scale precisely because HDD prices have not fallen as quickly as expected — tape continues to offer the lowest cost per GB for write-once archival workloads.
- Long-term capacity planning for large experiments cannot safely assume exponential cost reduction; actual procurement costs should be benchmarked against current market prices.

This law, first proposed in 2005 on an empirical basis, has not been strictly valid for about two decades. Nevertheless, its intrinsic meaning remains: technologies in this field evolve over time, and it is important to take this into account in the context of datasets that grow over the years.

2.3. Scalability

How do we manage a constantly growing dataset? How will performance change as a result of the increase in memory? This problem is called *scalability*, and there are two different approaches:

- **Vertical scalability:** the increase in memory is performed by replacing the current resources with more modern and advanced resources.
- **Horizontal scalability:** the increase in memory is performed by connecting new resources in a chain, forming a *distributed system*.

Vertical	Horizontal
No real change in the way the user accesses the resources	Possible to increase space consistently with a relatively low price
If performance is satisfactory, it can only improve	Scalability is facilitated at the hardware level
Expensive	Relies on the network connection — bandwidth can be limiting
Limit to the amount of memory available for a single device	Laborious to access resources distributed on different local systems
Operations must be interrupted during the update and data transfer	

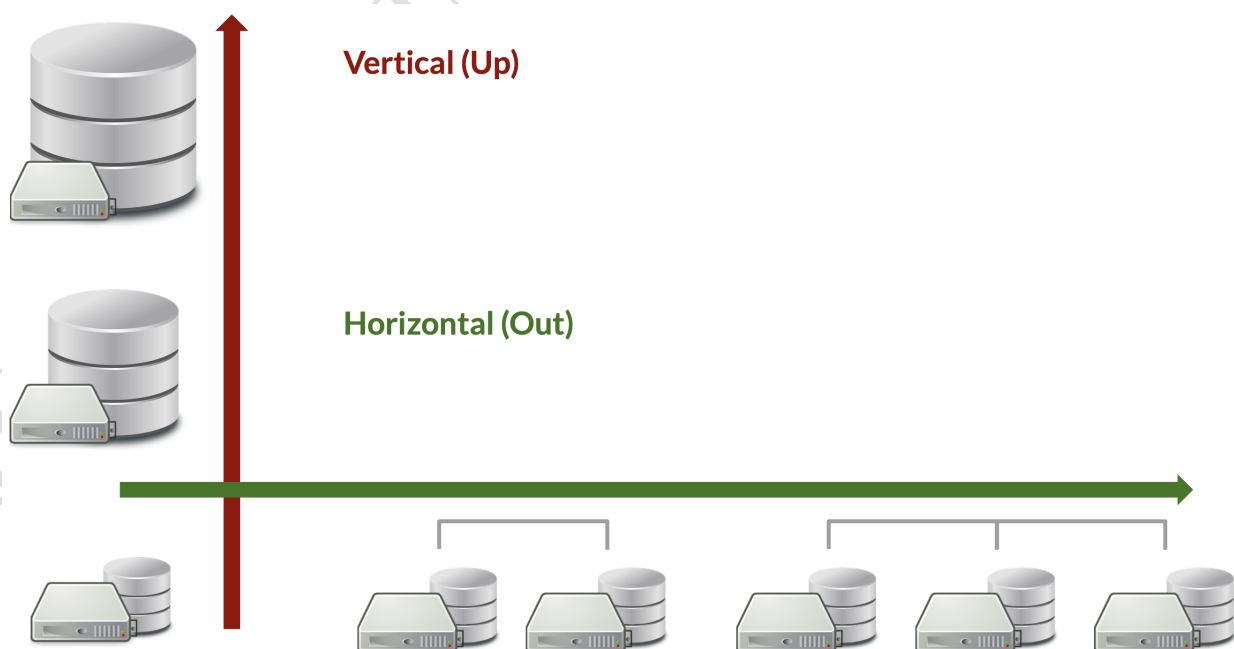


Figure 2.6.: The two possible scalability approaches starting from a given storage

2.4. Storage Media at a Glance

Medium	Typical Latency	Typical Throughput	Capacity	Cost/GB	Persistence
DRAM	~100 ns	~50 GB/s	GBs	Very high	Volatile
NVM / Persistent Memory	~1 μ s	~10–40 GB/s	GBs–TBs	High	Non-volatile
SSD (NVMe)	~100 μ s	~3–7 GB/s	TBs	Medium–high	Non-volatile
SSD (SATA)	~500 μ s	~500 MB/s	TBs	Medium	Non-volatile
HDD	~10 ms	~100–200 MB/s	TBs–PBs	Low	Non-volatile
Magnetic Tape	~100 s	~250–400 MB/s (sequential)	PBs	Very low	Non-volatile

The right storage tier depends on the access pattern: latency-sensitive workloads require DRAM or NVMe; archival workloads with infrequent access are well served by tape.

2.5. Measuring Storage Performance

The `dd` command-line utility is a standard tool for benchmarking raw storage throughput on Linux systems.

Write test (write 1 GB of zeros to a file, bypassing the OS page cache with `oflag=direct`):

```
dd if=/dev/zero of=/tmp/test_write bs=1M count=1024 oflag=direct
```

Read test (read the file back, discarding output):

```
dd if=/tmp/test_write of=/dev/null bs=1M iflag=direct
```

The `dd` output reports the elapsed time and measured throughput.

i Why `/dev/zero` and `/dev/null`?

In the write test, the input file is `/dev/zero` — a special Linux *device file* that produces an unlimited stream of null bytes (all zeros) without reading any real storage. This ensures the benchmark measures **write performance only**, not how fast the source medium can be read. In the read test, the output file is `/dev/null` — a special device file that silently discards everything written to it. This ensures the benchmark measures **read performance only**,

not how fast the destination medium can be written.

Both devices are effectively instantaneous from the kernel's perspective, so the elapsed time is dominated by the storage under test.

2.5.1. fio — Production-Grade I/O Benchmarking

`dd` is useful for a quick sequential-throughput estimate, but it cannot measure **IOPS**, **random access patterns**, **queue depth effects**, or **latency distributions**. The industry-standard tool for comprehensive storage characterisation is **fio** (Flexible I/O Tester).

Key capabilities of `fio`:

- Configurable block sizes (e.g., 4 kB for IOPS-focused tests, 1 MB for throughput tests)
- Sequential and random read/write patterns (`--rw=read`, `--rw=randread`, `--rw=randrw`, ...)
- Adjustable I/O queue depth (`--iodepth`) to stress the device's internal parallelism
- Multi-job concurrency (`--numjobs`) to simulate real-world workloads
- Detailed output: IOPS, bandwidth, and latency percentiles (p50, p99, p99.9)

Minimal fio example — random 4 kB reads (typical IOPS benchmark):

```
fio --name=randread --ioengine=libaio --rw=randread --bs=4k --numjobs=4 \
    --size=1G --runtime=30 --group_reporting
```

This runs four parallel jobs, each performing random 4 kB reads over a 1 GB working set for 30 seconds, and reports aggregated IOPS, throughput, and latency.

2.6. Exercises

Exercise 2.1 (*Evolving Physics Systems*)

In physics, a large portion of data is structured. What issues might arise in an evolving system (e.g., an experiment adding more sensing devices over time), and what alternative data management solutions could be implemented to address these challenges?

Solution

The main problem with structured data is that the schema is completely fixed, so if you add more sensor that are measuring something that is not already in the schema, you will have to change the schema and update all previous records. Another problem might be the scalability of the system, the new sensors might produce a bigger amount of data than expected. The first problem has not an easy solution, but apart from updating the schema you can change to a semi-structured data format which is more flexible. The second problem can be solved if it was foreseen before making measurements, if the system was designed to scale up (updating the hardware) then the fix is just to update the hardware, if not, it means that it was designed to scale horizontally, so the solution is to add more nodes to the system and distribute the

2. Storage

data among them.

Exercise 2.2 (*Scaling Up Storage*)

When scaling up a storage system, what types of issues are you likely to encounter? Discuss both technical and organisational challenges.

Solution

Depending on the type of scalability, the issues might be different. In vertical scalability, the main problem is that there is a limit to the amount of memory that can be added to a single device, so if the dataset grows too much, it might not be possible to scale up anymore. In horizontal scalability, the main problem is that the network connection can become a bottleneck, especially if the data needs to be accessed frequently and from multiple nodes. Organisational challenges include coordinating updates across multiple nodes, ensuring data consistency, and managing the increased complexity of a distributed system.

Exercise 2.3 (*Throughput vs Bandwidth*)

Explain the concept of storage throughput and highlight how it differs from bandwidth. Why is the measured throughput always lower than the theoretical bandwidth?

Solution

Storage throughput is the amount of data that can be transported (or used in I/O operations) per second. The bandwidth is the theoretical maximum of the throughput, which assumes optimal conditions for the device, and that's why it is always bigger than the throughput.

Exercise 2.4 (*Throughput Calculation*)

A storage device transfers 100 blocks of 4 kB in 9.5 ms. Compute the throughput (in MB/s) and the IOPS. Based on these values, identify the most likely type of storage device being used.

Solution

The throughput is:

$$\frac{100 \times 4 \times 10^{-3} \text{ [MB]}}{9.5 \times 10^{-3} \text{ [s]}} = 42.105 \text{ [MB/s]}$$

and the IOPS are:

$$\frac{100}{9.5 \times 10^{-3} \text{ [s]}} = 10526.31579 \text{ [IO/s}^{-1}\text{]}$$

In order to determine the device we should compute the latency, which is:

$$IOPS^{-1} = \frac{1}{10526.31579} = 95 \times 10^{-6} \text{ [s]} = 95 \text{ [\mu s]}$$

Given that the latency is closer to the 100 μs we could say that the device is an SSD.

Exercise 2.5 (*dd Command Exercise*)

The following Unix `dd` command is used to benchmark a storage element:

```
dd if=/dev/zero of=./test bs=16M count=64 oflag=direct
```

This command performs 64 consecutive writes of 16 MB blocks and measures the total elapsed time. If the overall time is measured at 0.436 seconds, what is the throughput of the storage element? From this throughput value, what kind of storage element is most likely being used?

Solution

The throughput is:

$$\frac{64 \times 16 \text{ [MB]}}{0.436 \text{ [s]}} = 2348.624 \text{ [MB/s]} \approx 2.35 \text{ [GB/s]}$$

and the IOPS are:

$$\frac{64}{0.436 \text{ [s]}} = 146.789 \text{ [IO/s]}.$$

With 16 MB blocks $1/\text{IOPS} = 6.8 \text{ ms}$ is time-per-block, not a device latency suitable for classification. The relevant clue is the **throughput**: a sustained $\sim 2.3 \text{ GB/s}$ on streaming writes is well above SATA SSD ($\sim 500 \text{ MB/s}$) and HDD ($\sim 150 \text{ MB/s}$), so the device is an **NVMe SSD**.

Exercise 2.6 (*HDD Latency from IOPS*)

Under ideal conditions, a given HDD is capable of 85 IOPS. What is the latency of the device? What are the main physical sources of this latency in a mechanical hard drive?

Solution

The latency is:

$$\text{IOPS}^{-1} = \frac{1}{85} = 11.764 \times 10^{-3} \text{ [s]} = 11.764 \text{ [ms]}$$

Which is very high compared with other devices, this high latency it's due to 2 main factors:

- * Seek time: Is the time the disk takes to align with the sector where the data lies
- * Rotation time: Is the time it takes the disk to rotate into the sector where the data lies

Exercise 2.7 (*Latency and Storage Media Identification*)

Given a storage device with the following characteristics: block size = 512 B, throughput = 3.2 GB/s. Calculate the IOPS and the latency of the device. Based on the obtained latency value, suggest what type of storage media (DRAM, NVMe SSD, SATA SSD, HDD, tape) is suitable.

2. Storage

Solution

With $3.2 \text{ [GB/s]} = 3.2 \times 10^9 \text{ [B/s]}$, the IOPS are throughput divided by block size:

$$\frac{3.2 \times 10^9 \text{ [B/s]}}{512 \text{ [B]}} = 6.25 \times 10^6 \text{ [IOPS]}.$$

Latency:

$$\text{IOPS}^{-1} = \frac{1}{6.25 \times 10^6} = 160 \times 10^{-9} \text{ [s]} = 160 \text{ [ns]}.$$

Sub-microsecond random latency combined with multi-GB/s throughput is in the **DRAM** range — neither SATA SSD ($\sim 100 \mu\text{s}$) nor NVMe SSD ($\sim 10 \mu\text{s}$) can hit it.

Exercise 2.8 (MTTF and Data Centre Reliability)

How does the Mean Time To Failure (MTTF) of storage devices influence the overall reliability of large-scale data centres? What strategies can be adopted to mitigate the risk of data loss due to device failures?

Solution

The Mean Time To Failure (MTTF) measures what is the average time that a device will work before it can loss part or all of the data. It influences the reliability of large-scale data center because they need to ensure that the data loss is reduced to the minimum and the more devices you manage, the more devices will brake and will have to be raplaced. One solution can be implemented by schedueling replacement of the devices before the MTTF, other solution can be to implement a RAID system to ensure that if there is a malfunctioning device the data is not lost, then replace that device, in this approach the device will only be replaced after failure.

Exercise 2.9 (CERN Failure Rate Calculation)

In 2013, the CERN Data Centre hosted approximately 52,000 tape cartridges and 17,000 HDD disks. Assuming a realistic MTTF of 150,000 hours for HDDs and 300,000 hours for tape cartridges, how many combined disks and tapes should be expected to fail every week?

Solution

For the tape cartridges (MTTF = 300,000 [h]):

$$\frac{52,000 \times 24 \text{ [h/day]} \times 7 \text{ [day/week]}}{300,000 \text{ [h]}} \approx 29.1 \text{ tapes per week.}$$

For the HDDs (MTTF = 150,000 [h]):

$$\frac{17,000 \times 24 \text{ [h/day]} \times 7 \text{ [day/week]}}{150,000 \text{ [h]}} \approx 19.0 \text{ disks per week.}$$

So on average ≈ 48 devices fail every week at the CERN Data Centre.

3. Reliability and Security

Physical media contain our data, but what matters is the integrity of the data, not of the memory units themselves. The threats from which we must protect data are of three types:

1. **Data loss:** if a memory device is subject to a failure (electrical or mechanical), usually all the data contained within it are lost.
2. **Data corruption:** certain environmental factors (lowering of the input voltage, increase in ambient temperature, cosmic radiation) can cause a *bit-flip*, that is the inversion of a bit. For HDDs, on average there is a bit-flip every 10^{14} bits involved in read or write cycles. A 10 TB read therefore spans roughly 10^{14} bits — meaning a single full scan of a 10 TB disk could produce one silent, undetected data corruption.
3. **Transfer errors:** occur when there are no adequate checks in the copy or transfer of data.

Definition 3.1 (Reliability). **Reliability** corresponds to the probability that a system will continue to operate correctly beyond a certain period of time or a certain number of operations.

In quantitative terms:

- **MTTF** (*Mean Time To Failure*): the average time that elapses between the first power-on of the device and the moment a failure occurs.
- **MTBF** (*Mean Time Between Failure*): defined for *repairable* systems, the average time that elapses between one failure and the next.

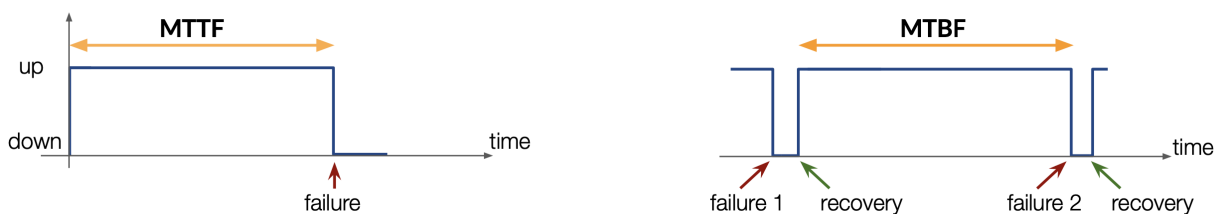
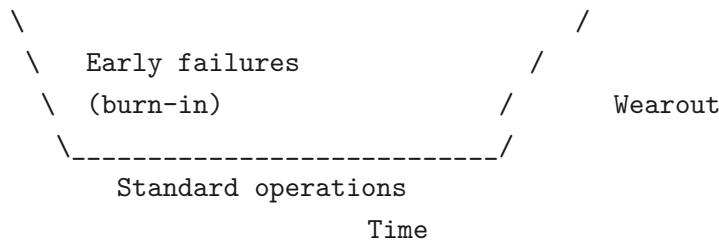


Figure 3.1.: MTTF and MTBF as a function of time

3.0.1. Failure Rate Lifecycle: The Bathtub Curve

The failure rate of hardware components is not constant over time. It follows a characteristic pattern known as the **bathtub curve**:

Failure
Rate



- **Early failure period (burn-in):** newly deployed components have an elevated failure rate due to manufacturing defects; these typically manifest within the first hours or days of operation.
- **Useful life period:** failure rate is low and roughly constant; this is the expected operating lifetime quoted by manufacturers.
- **Wearout period:** as components age, mechanical wear, electromigration, and cell degradation cause the failure rate to rise again.

MTTF and MTBF statistics are measured during the useful life period and do not capture burn-in or wearout behaviour.

Realistically, the MTTF of an HDD is about ten years. Assuming an equally distributed probability of failure:

$$p = \frac{1}{10 \cdot 365} = 0.00027$$

Considering 5000 independent HDDs:

$$5000 \cdot 0.00027 = 1.35$$

So, on average, it is expected that every day an HDD will fail and have to be replaced. At CERN, with $\sim 100\,000$ HDDs in daily use, that translates to about 27 damaged memory units per day.

3.1. Reliability Strategies

To store data securely, three main strategies are used:

1. **Mirroring:** the data are replicated on different devices, sometimes located in different places.
2. **Striping:** the data are divided into “pieces” and distributed on different memory elements.
3. **Checking:** errors relating to the consistency of the data within the devices are identified (and possibly corrected), such as a *checksum* or a *parity check*.

The basis on which to apply these strategies is the so-called *disk redundancy*, indicated by the acronym **RAID** (*Redundant Array of Independent Disks*), in which multiple devices are combined to form a single *logical* device in order to improve reliability and performance.

i Software RAID

RAID can be implemented entirely in software without dedicated hardware. On Linux, the `mdadm` utility manages software RAID arrays:

```
# Create a RAID 5 array across three devices
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb /dev/sdc
      ↪ /dev/sdd

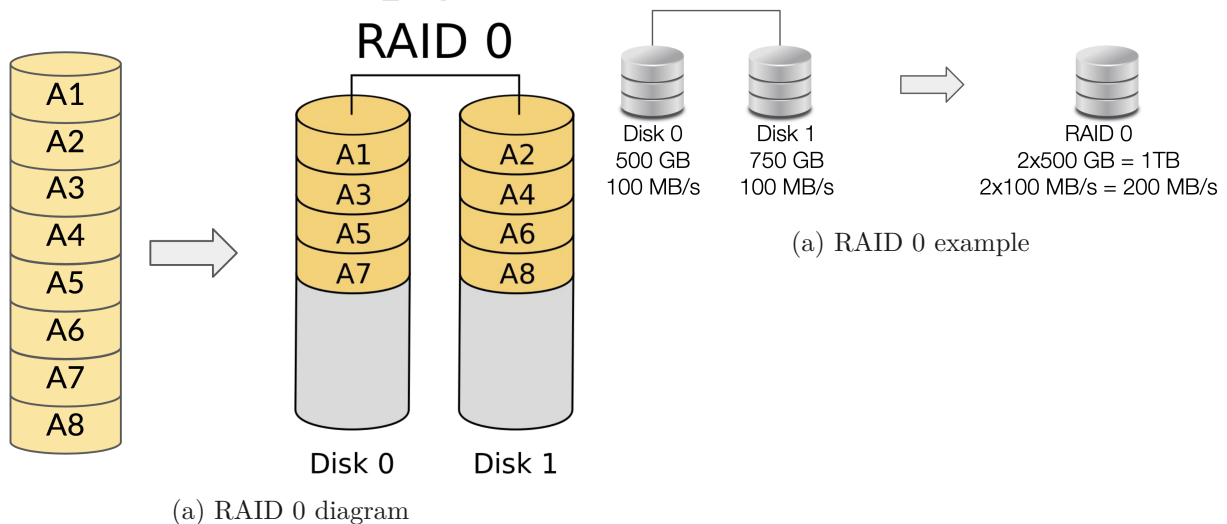
# Check array status
mdadm --detail /dev/md0
```

Software RAID is less expensive than hardware RAID controllers and is widely used in commodity server deployments.

3.1.1. RAID 0

Property	Value
Strategies	Striping (no mirroring, no checking)
Min. disks	2
Total capacity	Lowest single disk capacity × number of disks
Throughput	Conditioned by simultaneous read/write from all disks

Prioritizes performance over reliability — losing a disk implies losing all data.

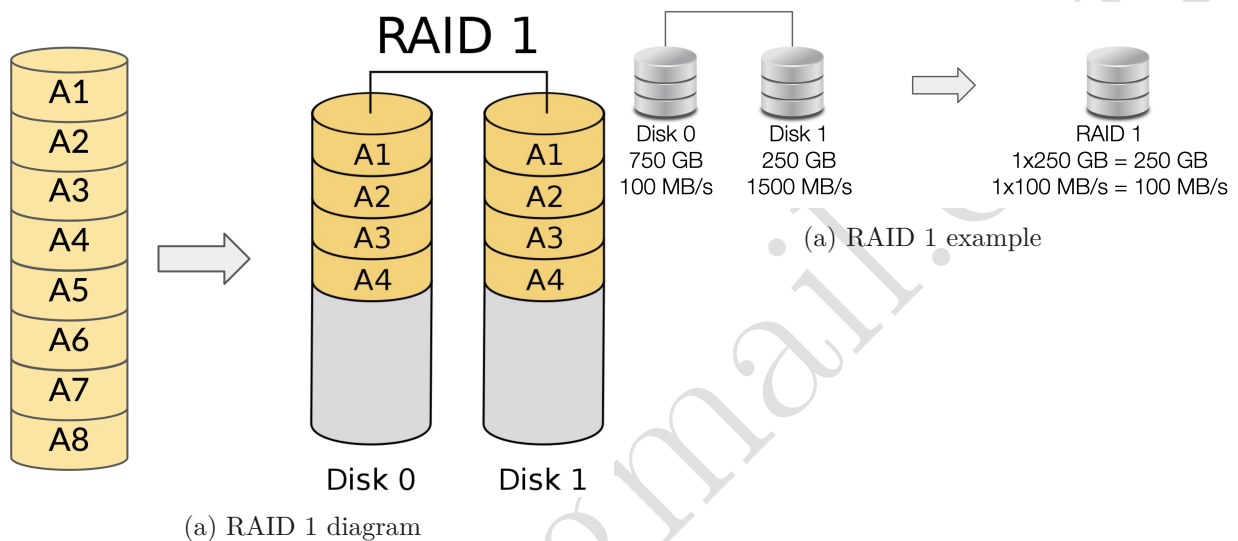


3.1.2. RAID 1

3. Reliability and Security

Property	Value
Strategies	Mirroring (no striping, no checking)
Min. disks	2
Total capacity	Lowest single disk capacity
Throughput	Throughput of the single lowest disk

Prioritizes reliability over performance — data is safe as long as at least one disk is operational.



3.1.3. RAID 1+0

Property	Value
Strategies	Mirroring and striping (no checking)
Min. disks	4
Total capacity	Lowest RAID 1 unit capacity $\times$ number of RAID 1 units
Throughput	Read: RAID 0 speed (fast); Write: RAID 1 speed (slow)

Maintains the high performance of RAID 0 while improving reliability with mirroring.

3.1.4. RAID 5

Property	Value
Strategies	Striping and parity checking (no mirroring)
Min. disks	3

Property	Value
Total capacity	Lowest single disk capacity $\times$ (number of disks $- 1$)
Throughput	Non-trivial due to parity read/write overhead

A portion of each disk is “sacrificed” in favor of parity information, so the data can be recovered if a single disk fails. The parity is calculated through a chain of XORs:

$$p = D_1 \oplus D_2 \oplus \dots \oplus D_n$$

Parity recovery by XOR inversion:

XOR is self-inverse: $A \oplus B \oplus B = A$. If disk D2 fails, the missing data is recovered as:

$$D2 = p \oplus D1 \oplus D3$$

Concrete 4-bit example:

Block	Bits
D1	1010
D2	1100 (failed)
D3	0111
$p = D1 \oplus D2 \oplus D3$	0001

Recovery: $p \oplus D1 \oplus D3 = 0001 \oplus 1010 \oplus 0111 = 1100$

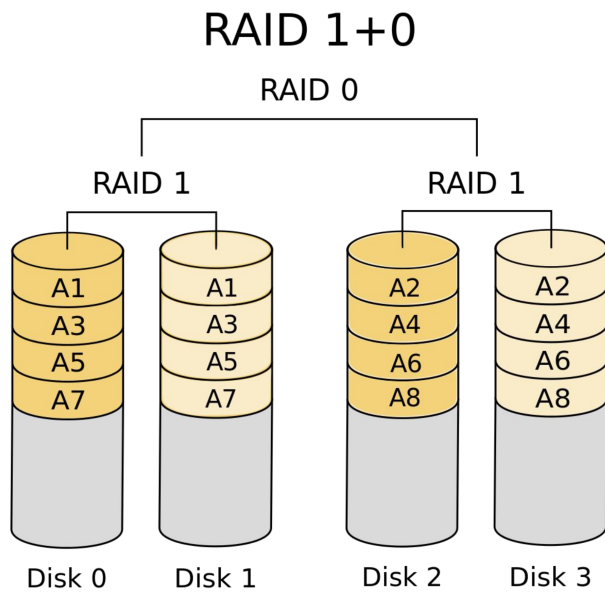
3.1.5. RAID 6

Property	Value
Strategies	Striping and parity checking (no mirroring)
Min. disks	5
Total capacity	Lowest single disk capacity $\times$ (number of disks $- 2$)
Throughput	Non-trivial due to parity read/write overhead

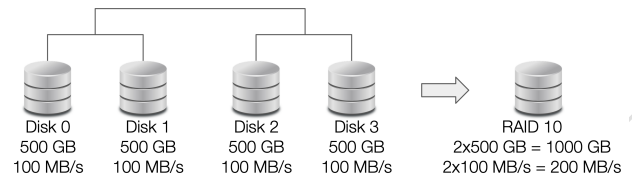
A portion of two disks is “sacrificed” in favor of the parity information, encoded in two bits per section. In order for the data to be recoverable after the failure of two disks, these parity bits must be *linearly independent* — the Reed-Solomon code is the most famous solution:

$$p = D_1 \oplus D_2 \oplus D_3$$

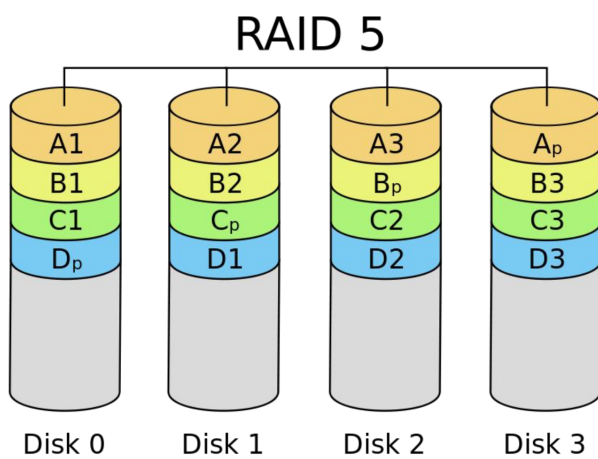
$$q = D_1 \oplus (D_2 \ll 1) \oplus (D_3 \ll 2)$$



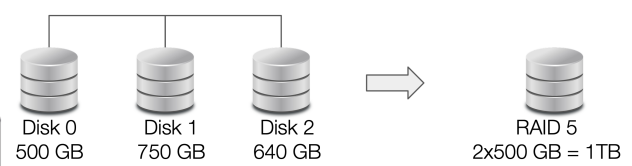
(a) RAID 1+0 diagram



(a) RAID 1+0 example



(a) RAID 5 diagram



(a) RAID 5 example

where p is the standard XOR parity (identical to RAID 5) and q is a second independent parity computed using Galois-field bit-shifts (). Because p and q are linearly independent, the system has enough information to solve for any **two simultaneously failed disks** — RAID 6 tolerates any two disk losses without data loss.

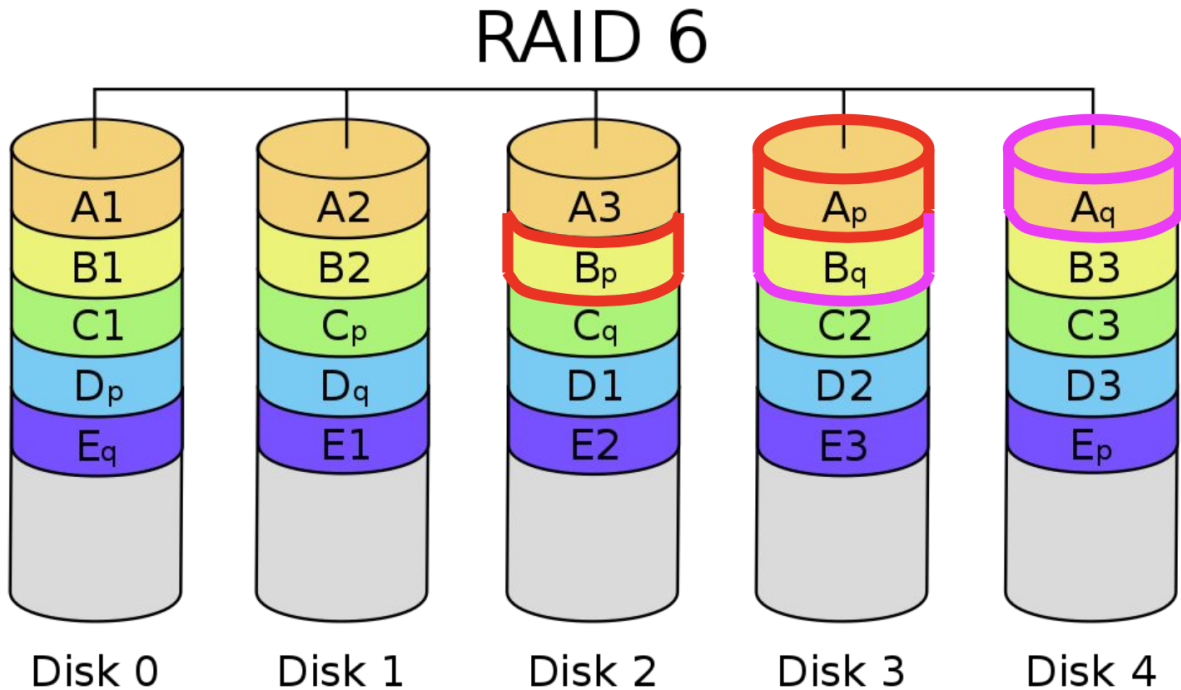


Figure 3.10.: RAID 6 diagram

3.2. Cryptography

Cryptography is a field of mathematics that deals with encoding information in order to make it secure and intelligible only to people authorized to access it.

The main actions involving security are:

- Protect the data saved on a device
- Establish secure connections with remote servers
- Exchange information between two *peers*
- Prevent malicious users from tampering with the data

For a message exchange to be considered *secure*, we must answer the following questions:

- Am I sure I know who I'm talking to?
- Can someone else intercept the data? (*Yes — it is **always** possible.* This is the basis of the security systems to be implemented.)
- Is the message I received identical to the one sent?
- Is the user I am talking to responsible for the message sent?

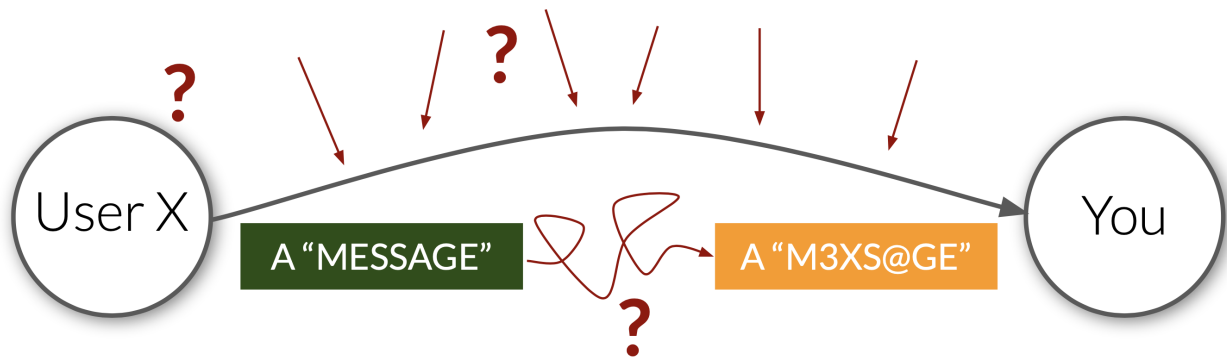


Figure 3.11.: Schematic of the message exchange with all the dangers that threaten its integrity and reliability

The main concepts of cryptography in data management:

1. **Confidentiality:** ensuring that no one can extract information even by intercepting the entire conversation.
2. **Integrity:** ensuring that the message has not been modified during the transfer.
3. **Authenticity:** verifying that you are communicating with the entity you think is at the other end.
4. **Identity:** verifying the identity of the specific individual.
5. **Non-repudiation:** ensuring that the individual responsible for a certain activity cannot deny being associated with it.

The five properties can be summarised as follows:

Property	Description
Confidentiality	Information is accessible only to authorised parties
Integrity	Data has not been altered without authorisation
Authenticity	The identity of the communicating party is verified
Identity	The ability to uniquely identify an entity in the system
Non-repudiation	A party cannot deny having performed an action

3.3. Authentication and Authorization

- **Authentication:** identifying users and validating their identities. Methods include username/password, certificates, keys, SSO, and biometric data (fingerprints, facial recognition, etc.).
- **Authorization:** granting a user permissions to access or use certain resources.

Authentication typically takes place through an *authorization server*, which certifies the user's identity; only after authentication does the system verify the user's permissions through an *Access Control List*, contained in the *resource server*.

3.4. Encoding and Decoding of Information

All messages exchanged through computer tools are constantly exposed. The only way to prevent information from being extracted is to modify the message so that an interceptor cannot understand it.

The most basic idea is to apply a *mask* — called a **key** — to the message, which shuffles and reassembles it. In cryptography these actions are called **encrypting** and **decrypting**.

Symmetric key encryption (*private-key cryptography*) uses a single shared key (applied for example via XOR or AES-128/192/256). It is simple to implement but both parties must share the same key, which is a security risk.

Asymmetric key encryption (*public-key cryptography*) uses two keys: a *public* key for encrypting and a *private* key for decrypting. They are produced together by the same algorithm (RSA, ECC, etc.) so that from one key it is impossible to derive the other. The public key is made openly available; only the intended recipient can decrypt with their private key. This is slower computationally but far more secure, and is today considered the standard.

Standard Cryptographic Algorithm Families

Category	Algorithms
Symmetric	AES-128, AES-192, AES-256
Asymmetric	RSA, ECC (Elliptic Curve Cryptography)
Hashing	MD5 (<i>deprecated</i>), SHA-1 (<i>deprecated</i>), SHA-2 family (SHA-256, SHA-512)

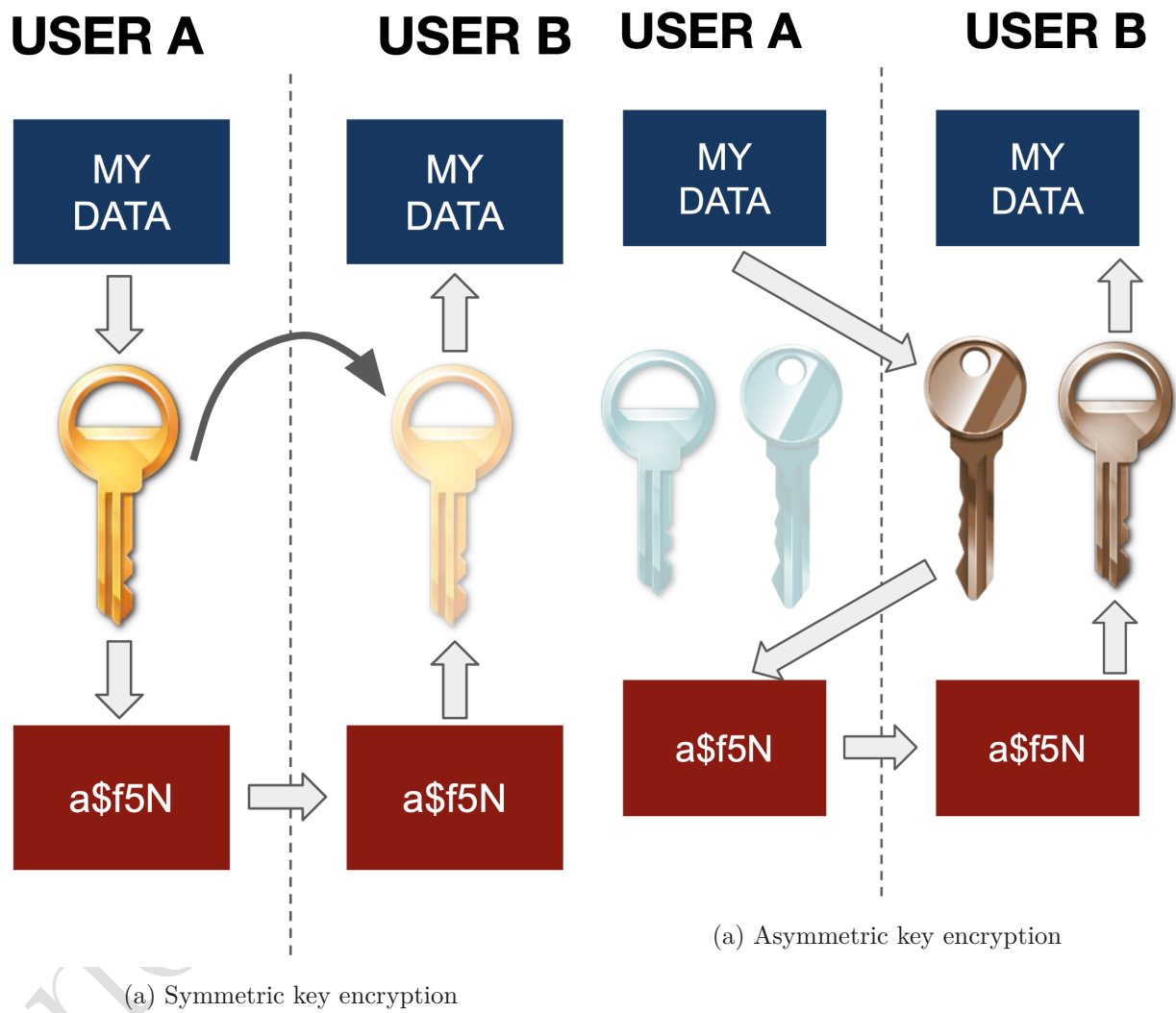
Symmetric algorithms are fast and suitable for bulk data encryption once a secure channel is established. Asymmetric algorithms are slower but allow secure key exchange without a pre-shared secret. Hashing algorithms are one-way and produce a fixed-length digest regardless of input size.

SSH in Practice

Asymmetric key authentication is the standard mechanism for connecting to remote Linux servers. A typical connection to a cloud VM using a private key:

```
ssh -i ~/private/my_private_key.pem ubuntu@10.67.22.231
```

When the server is behind a gateway (jump host), use the `-J` flag:



```
ssh -J gate.cloudveneto.it -i ~/private/my_private_key.pem
↪ ubuntu@10.67.22.231
```

The private key file must have restricted permissions (`chmod 400`). The server stores only the corresponding public key in `~/.ssh/authorized_keys`.

3.5. Hash Functions

Hash functions apply *hashing* to the data — scrambling them to the point that the original form *can no longer be reproduced*. Hashing algorithms (MD5, SHA-1, SHA-2) produce a **digest** of fixed length for all inputs. Modifying the input even slightly will completely change the final digest.

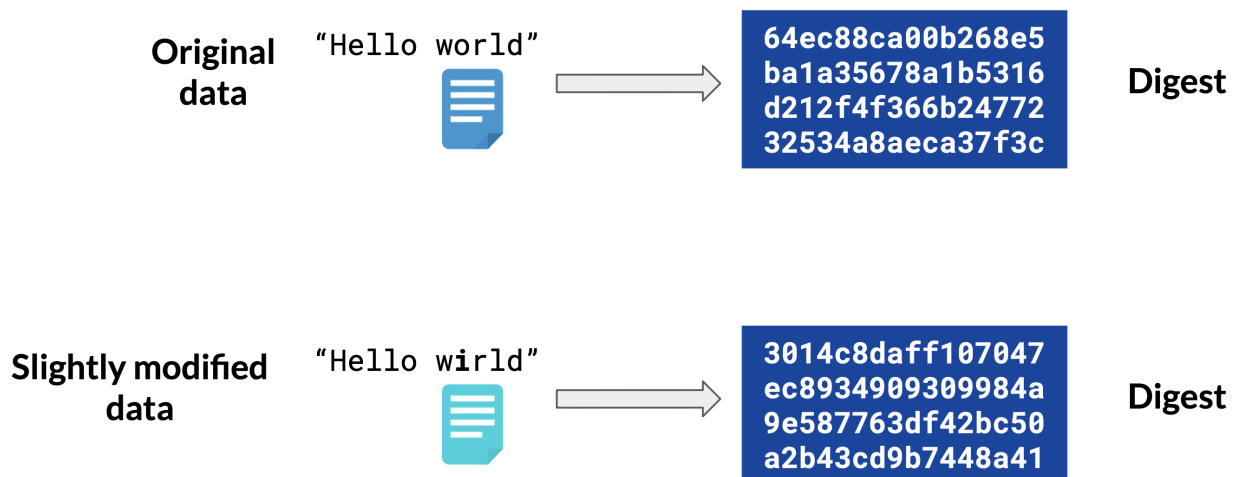


Figure 3.14.: Digest of two very similar data sets

A hash function maps an **infinite input space** to a **finite output space** (for example, 2^{256} possible digests for SHA-256). By the **pigeonhole principle**, collisions must therefore exist in theory: given enough distinct inputs, at least two will always map to the same digest. A cryptographically secure hash function does not eliminate collisions — it makes finding them computationally infeasible, meaning no practical algorithm can produce a collision in a tractable amount of time.

What the algorithms must prevent are **hash collisions** — cases in which two different data sets give rise to the same digest — since the uniqueness of each digest is their primary purpose.

The three main uses of hashing:

- **Data protection:** sensitive data such as passwords are stored as hashes, not in plaintext. Authentication systems compare input hashes with stored hashes.
- **Data integrity check:** comparing the hash of a received file with the expected hash confirms that the file was not tampered with during transfer.
- **Digital signatures:** due to their uniqueness and irreproducibility, hashes are used to certify documents stored in digital systems.

3.6. Encryption vs Hashing: Summary

	Encryption	Hashing
Reversible?	Yes (with key)	No (one-way function)
Purpose	Confidentiality — protect data in transit/at rest	Integrity — verify data has not changed
Output size	Same order as input	Fixed size (e.g. 256 bits for SHA-256)
Key required?	Yes (symmetric) or key pair (asymmetric)	No
Use when	Storing sensitive data, securing communications	Storing passwords, verifying file integrity, digital signatures
Examples	AES-256, RSA, ECC	SHA-256, SHA-3, bcrypt

3.7. Exercises

Exercise 3.1 (*Mirroring, Striping, Parity*)

Define mirroring, striping, and parity check. Discuss the pros and cons of each technique, and compare them in terms of reliability in RAID configurations.

Solution

Mirroring: Duplicates the data in to different devices. Pros: In case of disk failure the data is still available. Cons: You duplicate the data using mor storage
Striping: Divides the data into smaller portions and the portions are distributed in different devices. Pros: It increases throughput (it can be parallelized). Cons: If there is disk failure the data is lost
Parity checks: It assigns a parity to the data and checks the data with XOR operations. Pros: It makes sure the data is not corrupted and it can reverse data loss in some cases. Cons: It reduces throughput due to performing checkings.

In terms of reliability the best methods are mirroring and parity checking since are the only ones where disk failure does not necessarilly mean data loss. Stripping is only prefered when throughput needs to be high.

Exercise 3.2 (*RAID 0 vs RAID 1*)

What is the performance impact of using RAID 0 vs RAID 1? What factors should be considered when choosing between them?

Solution

The two main factors are the reliability and throughput. RAID 0 is prefered when multiple operations needs to be done at the same time and fast, as it uses striping. RAID 1 is prefered when reliability is main factor to guarantee, as it uses mirroring. A third factor would be

storage capacity, as it is a limiting factor, specially in a RAID 1 configuration since you need twice or more storage capacity than data.

Exercise 3.3 (*RAID 0 Throughput and Risks*)

How does RAID 0 increase data throughput, and what are the risks associated with this configuration?

Solution

The RAID 0 throughput is increased via striping (dividing the data into multiple disks), since this make reading and writting faster via parallelization of the task. The main risk affecting the RAID 0 configuration is data loss, as there is no back up copies of the data when a disk fails all data is lost.

Exercise 3.4 (*RAID 5 Capacity*)

You have five disks with sizes x_1, x_2, x_3, x_4, x_5 . If you arrange them in a RAID 5 system, what will be the usable storage capacity of the system? Express the answer in terms of the individual disk sizes.

Solution

The usable storage capacity will be $4 \times$ the lowest storage capacity as one disk is kept for allocating the parity of the data.

Exercise 3.5 (*Parity Placement*)

Can you store the parity of a data block on the same disk as the data block itself? Explain your answer in the context of RAID reliability.

Solution

The answer is NO. If we stored the data in the same disk as the data it would be impossible to recover the data as we would have lost the data and the parity of the data. If the parity is stored in a different disk is possible to compute the parity of the data and compared with the stored parity in order to recover it, but if we lose both parts we can not computed or know which parity had. That is why is important in terms of reliability to store them in different disks

Exercise 3.6 (*RAID 6 Configuration*)

You have a RAID 6 configuration with 5 disks. What is the usable storage capacity of the filesystem (expressed as a fraction of total raw capacity)? How many disks can fail simultaneously without data loss?

Solution

In a RAID 6 configuration we can lose up to 2 disks. The total capacity (assuming all disks have equal capacity) would be $3/5$ of the raw capacity as we use 2 disks for parity checking. If the disks have different capacity the usable storage would be smaller than $3/5$.

Exercise 3.7 (RAID 10 vs RAID 6)

Compare RAID 10 and RAID 6 in terms of pros and cons regarding storage efficiency and reliability (fault tolerance).

Solution

- **Storage efficiency.** RAID 6 wins: $(n - 2)/n$ usable vs $1/2$ for RAID 10. With $n = 6$: $4/6$ vs $3/6$.
- **Write performance.** RAID 10 wins: mirror + stripe with no parity computation. RAID 6 pays double-parity overhead on every write.
- **Fault tolerance.** RAID 6 tolerates *any* 2 simultaneous failures. RAID 10 also tolerates 2 in the worst case (and up to one per mirror in the best), but loses data if both halves of any single mirror fail together.

Rule of thumb: RAID 6 for capacity-bound archives, RAID 10 for write-heavy databases.

Exercise 3.8 (RAID Configuration Exercise (1))

You are tasked with choosing a RAID configuration to store a 3.5 TB dataset. You have 6 disks available with the following capacities:

Disk	Capacity (TB)
Disk-0	1.0
Disk-1	1.0
Disk-2	1.2
Disk-3	1.5
Disk-4	1.5
Disk-5	1.8

The dataset needs to be stored with high reliability and fault tolerance. Which RAID configuration would you choose, and why? Consider both the storage requirements and fault tolerance.

Solution

The RAID 6 configuration is the best choice as it has an storage capacity of $(6 - 2) \times 1.0 \text{ [TB]} = 4.0 \text{ [TB]}$ (RAID 5 would give us more storage, but is less reliable) where we could lose 2 disks before data loss. RAID 0 would give us more storage (6.0 [TB]) but no reliability and RAID 1 or RAID 10 do not meet the storage requirements.

Exercise 3.9 (RAID Configuration Exercise (2))

Arrange the following 6 disks in a RAID 10 configuration:

Disk	Capacity (GB)	Throughput (MB/s)
0	750	120
1	640	200
2	500	250
3	640	150
4	1000	120
5	720	160

- How can the 6 disks be arranged into mirrored pairs?
- What is the estimated overall usable storage capacity?
- How many disks can fail without incurring data loss in this configuration?

Solution

Pair similar capacities so the mirror loss is small: $(1000 + 750)$, $(720 + 640)$, $(640 + 500)$, giving mirror sizes 750, 640, 500 GB.

Striping across the three pairs is limited by the smallest pair-min, so the usable capacity is $3 \times 500 \text{ [GB]} = 1.5 \text{ [TB]}$.

Up to three disks can fail (one per pair); if both halves of any single mirror fail, data is lost.

Exercise 3.10 (Encryption vs Hashing)

What is the difference between encryption and hashing? Why are both techniques important and useful for ensuring data security? Give an example use case for each.

Solution

Encryption is used in secure communication as data can be coded and decoded if the users have the keys or methods to do it. Hashing instead can not be decoded and is used to check and store data.

We use encryption we want to have a controled aces to data, like when we send a message in an app to a person, we want only that person to be able to read it and replying. Instead we would use hashing when we want to check the data (the same data will produce the same digest, so if we know the digest and want to check the data we can compare it) or to store data (like when we enter a password into a web service the platform will check the digest, so they don't know or have your password but they can authenticate you by the digest)

Exercise 3.11 (Symmetric vs Asymmetric Encryption)

What are the differences between symmetric and asymmetric encryption? In what scenarios is each type preferable?

Solution

In symmetric encryption only one key is generated and is meant to be shared only with the entity we want to communicate, in asymmetric encryption 2 keys are generated and one is meant to be public for anyone and the other one is meant to be completely private, in that way entities can communicate with you but only you can read what they send to you. Asymmetric encryption is always preferable, since you don't need to share anything that is meant to be private. Symmetric key encryption can be used in private communications and asymmetric encryption can be used when you want to share data with other people but have control about who can read it.

Exercise 3.12 (*Asymmetric Encryption Vulnerabilities*)

How can data be compromised in an asymmetric key encryption scheme? Describe at least two attack vectors or points of failure.

Solution

- **Man-in-the-middle.** Without a trusted PKI / certificate authority, an attacker can substitute their own public key for the recipient's and intercept all subsequent traffic.
- **Private-key compromise.** If the private key is stolen, copied, or generated with a weak RNG, every past and future message encrypted to it becomes decryptable. RSA is also vulnerable to factorisation attacks if the key length is too small (below ~2048 bits today).

Exercise 3.13 (*Data Confidentiality in Network Communications*)

What common methods are used for protecting data confidentiality in network communications? Name and briefly describe at least three techniques.

Solution

- **TLS / HTTPS.** An asymmetric handshake establishes a session key, then the payload is symmetrically encrypted (AES). Used in essentially all modern web traffic.
- **VPN / IPsec.** Encrypts the whole IP packet between two endpoints, hiding source/destination as well as the payload.
- **End-to-end encryption** (PGP, Signal, age). Application-level encryption so the transport provider cannot read content; only the recipient's private key decrypts.

Exercise 3.14 (*Authentication vs Authorisation*)

What is the difference between authentication and authorisation? Provide a concrete example illustrating the distinction.

Solution

In authentication you want to verify the person who is logging in and in authorisation is where you manage what permission (what the authenticated user can do) they have. An example could be a university website, where students and professors are authenticated in order to access, but they have different authorisations as the students can not alter grades.

Exercise 3.15 (Data Integrity)

What is integrity in data security, and what strategies can be used to ensure it? Describe at least two mechanisms used in practice.

Solution

Integrity is the guarantee that data has not been altered — accidentally or maliciously — since it was written. Two practical mechanisms:

- **Cryptographic hashes / checksums** (SHA-256, CRC32). The producer publishes the digest; the consumer recomputes it on receipt and compares.
- **Digital signatures**. The producer signs the hash with their private key; anyone with the matching public key verifies *integrity* and *authenticity* in one step.

Exercise 3.16 (Confidentiality in Cryptography)

Describe how confidentiality can be ensured through cryptographic techniques. What properties must a secure encryption algorithm have?

Solution

Confidentiality is achieved by encrypting data so only holders of the right key can recover the plaintext. A secure encryption algorithm must be:

- **Hard to invert without the key** — no efficient decryption beyond brute force.
- **Resistant to chosen-plaintext attacks** — many (*plaintext*, *ciphertext*) pairs must not leak the key.
- **Semantically secure** — ciphertexts of two chosen plaintexts must be indistinguishable to an attacker.
- **Randomised** via fresh IVs / nonces so identical plaintexts yield different ciphertexts.

Examples: AES-256-GCM (symmetric), RSA-OAEP or ECIES (asymmetric).

Exercise 3.17 (Cloud Confidentiality for Patient Data)

How could confidentiality be ensured in a Cloud system that contains sensitive data about patients? What technical and organisational measures would you implement?

Solution

Technical.

- **Encryption at rest** (AES-256) and **in transit** (TLS).
- **Client-side encryption** so the provider stores only ciphertext; keys held in a separate KMS or on-premise HSM.
- **Pseudonymisation** of identifying fields (name, ID, date of birth) before upload.

Organisational.

- **Role-based access control** with audit logs of every read.
- **Vetted provider** under a compliant data-processing agreement (HIPAA, GDPR).
- **Geographic restrictions** so the data never leaves the legal jurisdiction.

Exercise 3.18 (*Managing a Sensitive Physics Dataset*)

You are tasked with defining the strategy to manage and share with others (e.g., physicists) a large dataset of potentially sensitive data, such as records of hospital patients. What techniques and strategies would you consider effective? Consider data access control, anonymisation, encryption, and audit trails.

Solution

- **Access control.** Authenticated users only, with RBAC: clinicians see identifiable records, researchers see pseudonymised ones.
- **Anonymisation / pseudonymisation.** Strip direct identifiers before release and replace with stable tokens so re-linking requires a separate key held by the data custodian.
- **Encryption.** AES-256 at rest, TLS in transit, plus client-side encryption for highly sensitive fields.
- **Audit trails.** Immutable logs of every access (who, when, what record) for compliance review.
- **Data-sharing agreements** with collaborators specifying allowed analyses and prohibiting re-identification.

Exercise 3.19 (*Hash Collisions*)

What is a hash collision? Why do we want to avoid it, and is it totally avoidable? How do modern hash functions (e.g. SHA-256) minimise the probability of collisions?

Solution

A **collision** is two distinct inputs $x \neq y$ with the same digest $h(x) = h(y)$. We want to avoid them because an attacker who finds a collision can substitute one input for another — e.g. a malicious file for a benign one — without changing the published hash.

Not totally avoidable: any fixed-length hash maps an infinite input space into 2^n outputs, so collisions exist by pigeonhole. Modern hashes like SHA-256 make them *computationally infeasible*: the birthday-bound cost of finding one is $2^{n/2} = 2^{128}$, well beyond any feasible

compute.

Exercise 3.20 (Parity Checks and Checksums)

What is the role of parity checks and checksums in data management? In what contexts are they used, and what are their limitations?

Solution

Both detect accidental data corruption.

- **Parity bit.** A single bit added so the count of 1s in a word is even (or odd). Detects any odd number of bit-flips but cannot locate or correct them, and is useless against bursts. Used in RAM ECC, RAID 5/6, serial links.
- **Checksum** (CRC, Adler-32, MD5). A short fixed-size digest of a longer block. Detects most random errors and many bursts. Used in TCP/IP packets, `md5sum` file checks, ZFS / ext4 block validation.

Limitations. Neither resists *intentional* tampering — an attacker who modifies the data can recompute the checksum. For that you need a cryptographic hash or a MAC.

Exercise 3.21 (Brute-Force Cracking Exercises)

Answer the following two sub-questions:

(a) Imagine your password is hashed via the SHA-256 algorithm and it contains 10 characters drawn from a standard alphanumeric character set (letters a–z, A–Z, digits 0–9: 62 characters total). A hacker can try 1,000 different hash combinations per hour. How much time does it take for the hacker to crack your password in the worst case?

(b) If you use SHA-256 hashing on a text and an algorithm can guess at a rate of 2×10^{18} guesses per second, how long will the algorithm take in the worst case to find the original input? (Recall that SHA-256 produces a 256-bit digest, giving 2^{256} possible hash values.)

Solution

(a) Search space = $62^{10} \approx 8.4 \times 10^{17}$. At 10^3 hashes/hour,

$$\frac{62^{10}}{10^3 \text{ h}^{-1}} \approx 8.4 \times 10^{14} \text{ h} \approx 9.6 \times 10^{10} \text{ years.}$$

About a hundred billion years — astronomically safe at that hash rate.

(b) With $2^{256} \approx 1.16 \times 10^{77}$ possible digests and 2×10^{18} guesses/s,

$$\frac{2^{256}}{2 \times 10^{18} \text{ s}^{-1}} \approx 5.8 \times 10^{58} \text{ s} \approx 1.8 \times 10^{51} \text{ years.}$$

Brute force on the digest space is infeasible regardless of how short the input was; this is why pre-image resistance of cryptographic hashes is the load-bearing property in password hashing.

4. File System

There are three main solutions for storing and working with datasets:

1. **File:** the best known and most used. *Files* are sets of data blocks managed by a system called a *File System*, which distributed over multiple memory units (horizontal scalability) will act as a *Distributed File System*.
2. **DataBase:** each single element is accessible at a *granular* level and modifiable through systems called *DataBase Management System*; if distributed over multiple memory units, Databases are called *Distributed Databases*.
3. **ObjectStore:** the data are saved in any arbitrary form (files, blocks of specific length, entire operating systems) inside containers called *buckets*; they are mainly used in *cloud storage*.

This chapter deals with file systems; the next chapter covers databases.

4.1. Characteristics

The role of a file system is to manage and keep track of the position of data saved in memory in the form of files through a **hierarchical** structure, making use of folders (also called *directories*) and subfolders. Its main characteristics can be summarized in three aspects:

1. **Data saving:** the data are physically stored in the form of *blocks* of fixed length, distributed within the memory devices. The file system both divides and allocates the files in memory, and keeps track of all the positions of the blocks.
2. **Abstraction:** the file system identifies the data through the *name* of the file, the *path* organized according to a hierarchical structure, and the *metadata* attributed to the file.
3. **Low-level operations:** the file system interfaces with the operating system to allow *low-level* operations on the storage blocks, such as opening, closing, reading, writing, and searching. This interface follows precise standards, the most widespread of which is **POSIX** (*Portable Operating System Interface for Unix*).

The **POSIX (Portable Operating System Interface)** standard defines the minimum set of file operations that a conforming OS must provide:

Operation	Description
OPEN	Open or create a file; returns a file descriptor
CLOSE	Release the file descriptor
READ	Read bytes from the current position
WRITE	Write bytes at the current position

Operation	Description
SEEK	Move the current position within the file

POSIX compliance ensures that programs written for one Unix-like OS (Linux, macOS, BSD) will work on another without modification.

4.2. File Access Modes

File systems support three fundamental access patterns:

Access Mode	Description	Use Case
Sequential	Records are read/written in order from beginning to end	Log files, streaming data, tape archives
Direct (Random)	Any record can be accessed directly by its position/address	Databases, indexed data
Indexed	An index structure maps keys to physical locations; lookups use the index	B-tree indexes in databases, directory listings

Most operating systems implement direct access at the OS level; sequential and indexed access are typically layered on top by applications or database engines.

4.3. Space Allocation

The main distinction between the various file systems concerns the way in which space is allocated to save files in memory. Below are three types of allocation, proceeding by scale of efficiency.

4.3.1. Contiguous Allocation

The various blocks that make up the individual files are physically arranged next to each other. Access to the files is of a **sequential** type.

The limit of this type of allocation is evident: when you want to expand the content of a file, beyond a certain point you will encounter an already occupied block from another file that cannot be overwritten. For this reason, a file system based on this allocation is suitable for magnetic tapes, where files are saved to be kept long-term without ever being modified.

File	Start	Length
count	0	2
f	6	2
tr	14	3

File	Start	Length
mail	19	6
list	28	4

Example directory table for contiguous allocation with five files

4.3.2. Linked Allocation

Through linked allocation it is possible to “jump” from one block to another to store a single file. This is done by inserting in each block the information on where the next data block is located, so a small part of the memory is “sacrificed” to allow the file system to move and retrieve information.

Compared to contiguous allocation, a better optimization of the available space is possible, not being bound to a series of contiguous blocks to compose the file.

File	Start	End
jeep	9	25

Example directory table for linked allocation

- **Advantage:** no external fragmentation; files can grow dynamically without needing contiguous free space.
- **Disadvantage:** random access is $O(n)$ — to reach block k you must follow k pointers sequentially. Additionally, a single corrupt pointer loses the rest of the file.

i FAT: Linked Allocation with an In-Memory Table

FAT (File Allocation Table) is a variant of linked allocation where the linked list of block pointers is stored in a separate table kept in memory rather than within the data blocks themselves. This avoids the per-block pointer overhead and allows faster traversal, but the FAT table must fit in memory and becomes a single point of failure.

4.3.3. Indexed Allocation

Linked allocation optimizes space on the disk but not the time in operations involving the files: every time an operation is requested, the system must start from the initial block and “bounce” between all subsequent blocks until it reaches the desired destination.

Indexed allocation optimizes time through the use of direct and indirect blocks, with the latter having the function of indicating the positions of the data.

The first block is constituted as follows:

- The initial part contains the *metadata* relating to the file.
- Then follow the *direct blocks* — addresses pointing directly to the data.

4. File System

- Optionally, *single indirect blocks* point to blocks that in turn point to the data; *double indirect blocks* extend this one level further, and so on.

An inode's data block pointers are organized in tiers:

1. **Direct blocks** (e.g., 12 pointers) — each pointer points directly to a data block.
2. **Single indirect block** — one pointer to a block that contains N pointers, each pointing to a data block.
3. **Double indirect block** — one pointer to a block of pointers, each of which points to a block of pointers to data blocks.
4. **Triple indirect block** — one further level of indirection beyond double indirect.

This tiered structure keeps the inode small while supporting very large files via indirection: small files use only direct blocks (fast access), while large files extend through indirect blocks as needed.

Example (Berkeley Fast File System): 12 direct blocks + 1 indirect block, each 4 kB; memory addresses occupy 4 bytes.

The 12 direct blocks contain:

$$12 \times 4 \text{ kB} = 48 \text{ kB}$$

The indirect block contains $\frac{4 \text{ kB}}{4 \text{ B}} = 1000$ addresses, pointing to $1000 \times 4 \text{ kB} = 4 \text{ MB}$ of data.
Total:

$$48 \text{ kB} + 4 \text{ MB} \approx 4 \text{ MB}$$

Adding a *double* indirect block would yield $1000 \times 1000 \times 4 \text{ kB} = 4 \text{ GB}$ more. Increasing the layer of indirect blocks exponentially increases the memory dedicated to a file.

4.4. Limits

A file system is mainly limited by the space it can dedicate to a single file, and by the total space it can manage for a single memory unit (due to counters with a limited number of bits).

Another limitation is that the file system cannot give information on the content within a single file — it is up to the user to exploit the hierarchical tree organization to order the data conveniently. This does not allow operations such as filtering based on the content of the files.

Pros	Cons
Can handle a large amount of data	Scaling a growing amount of data according to the hierarchical view is complex
Low-level operations via POSIX are very fast	No structure underlying that of the files is accessible
Offers a hierarchical view of all data	It is not possible to group, select and filter data based on file content

Pros	Cons
Single entities (files) can be easily transferred, copied, shared and processed	

4.5. Common File Systems Compared

File System	OS	Max File Size	Max Volume Size	Max Filename	Permissions	Journal
FAT32	Win-dows/cross-platform	4 GB	8 TB	255 UCS-2 characters	No	No
exFAT	Win-dows/cross-platform	128 PB	128 PB	255 UTF-16 characters	No	No
NTFS	Windows	16 TB	256 TB	255 characters	Yes	Yes
HFS+	macOS	8 EB	8 EB	255 UTF-16 characters	Yes	Yes
ext3	Linux	2 TB	32 TB	255 bytes	Yes	Yes
ext4	Linux	16 TB	1 EB	255 bytes	Yes	Yes
XFS	Linux	8 EB	8 EB	255 bytes	Yes	Yes

ext4 is the default file system on most modern Linux distributions and is suitable for the majority of workloads. **FAT32/exFAT** are used for cross-platform compatibility (USB drives, SD cards).

File System Paths in Practice

A file path represents the full location of a file within the hierarchy. In the context of physics data, paths can encode experiment, run, and file type information:

```
/eos/cms/store/data/Run2012D/SingleMu/RAW-RECO/22Jan2013-v1/20000/
00259D38-8A6F-E211-8E23-003048F118C4.root
```

Each component — experiment (**cms**), data type (**data**), run period (**Run2012D**), stream (**SingleMu**), format (**RAW-RECO**) — is a directory level, making the hierarchy self-documenting.

4.6. Exercises

Exercise 4.1 (*Indirect Blocks and Maximum File Size*)

How does increasing the number of indirect blocks impact the maximum file size in an indexed file system? Explain the relationship between indirect block levels and file size limits.

Solution

Indirect blocks optimize the space used to store data as we store the pointer and not the data itself (which has a lower size in memory), so the more pointers we have the bigger the file we can store. This comes with a trade-off as the number of pointers increase the time to read the data also increases. If we use double indirected blocks, which is an indirect block that points to another indirect block that then points to the data, it increases exponentially as it would n^2

Exercise 4.2 (*File System Size Calculation*)

You have a simple indexed file system with the following parameters:

- Block size: 512 bytes
- Pointer size: 4 bytes
- 3 direct blocks
- 2 singly-indirect blocks
- 1 doubly-indirect block

What is the maximum size of a file that can be stored in this file system? Show all calculation steps.

Solution

First let's compute the number of pointers per block, that is:

$$\frac{512 [B]}{4 [B]} = 128 \text{ pointers}$$

The maximum size for a file would be:

$$3 \times 512 [B] + 2 \times 128 \times 512 [B] + 128 \times 128 \times 512 [B] = 8,521,216 [B] \approx 8.5 [MB]$$

As we account for the direct blocks, the 2 singly-indirect blocks and the doubly-indirect block. The space used to store that file would be:

$$3 \times 512 [B] + 2 \times 128 \times 4 [B] + 128 \times 128 \times 4 [B] = 68,096 [B] = 68.096 [kB]$$

Exercise 4.3 (*Types of File Access*)

What are the types of memory access in a local file system? Describe each type and give an example use case where it is most appropriate.

Solution

The first type is the sequential access, where the data is stored in order so the data is stored in order and can be accessed by simply moving forward in the memory. Like a movie tape where we can see each fotogram of the movie displayed in order. You just need to unroll the tape to see everything but you can not go directly to the end as you would have to see everything before. The second type is direct access, in this case you only store the information of where is the data and can be accessed from anywhere in the system. The last one is indexed access, here the information is stored in an index table (or key-value table) which contains pointers to the data. You can see it as a menu, where the file system (you) select what file you want (the dish) and then that is retrieved for you to use (eat).

5. Distributed File System

To apply a file system to a horizontally scaled system we must move from *local file systems* to **distributed file systems**, indicated with **DFS** (*Distributed File Systems*). In technical terms, a *cluster* is a set of computing resources — computers called *nodes* of the cluster. The user typically connects from outside the cluster through a *client* program. The individual nodes are connected to each other through a very fast *communication network*, which the external user also accesses directly, meaning the user does not have an *atomic* view of the individual nodes in the cluster.

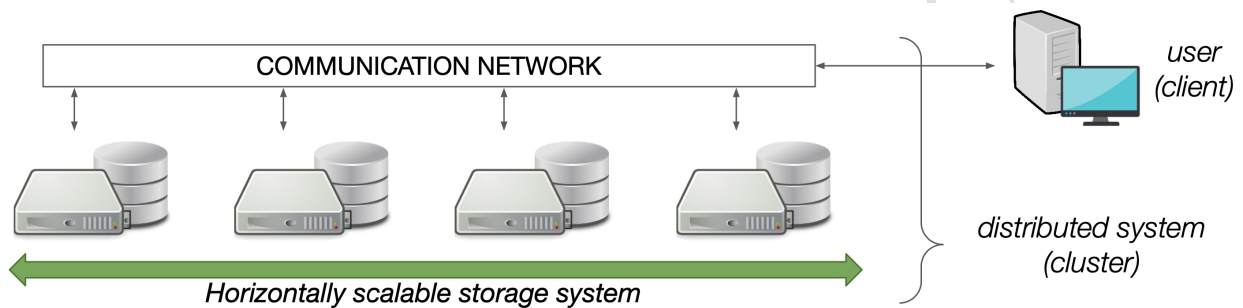


Figure 5.1.: Diagram of a horizontally scaled storage system

5.1. Operation

Some well-known DFS implementations:

- **NFS** (*Network File System*): built for UNIX operating systems, the most widespread at the user level.
- **AFS** (*Andrew File System*): suitable for academic purposes in research environments.
- **Lustre**: specifically designed for the HPC (*High Performance Computing*) field to maximize throughput and manage huge amounts of data.
- **EOS** (*Elastic Object Storage*): developed at CERN to mainly manage large amounts of unstructured data.

The table below summarises the major DFS implementations and their release years:

DFS	Year	Notes
NFS (Network File System)	1984	Sun Microsystems; still widely used
AFS (Andrew File System)	1986	Carnegie Mellon; predecessor to OpenAFS

5. Distributed File System

DFS	Year	Notes
Lustre	2003	High-performance computing clusters
GFS (Google File System)	2003	Inspired HDFS design
HDFS	2006	Apache Hadoop; based on GFS paper
EOS	2010	CERN; used for LHC data

All DFS share the same basic operation: data (together with metadata) are divided into files, in turn broken into memory blocks of fixed size distributed in various positions but always traceable through indexes and pointers.

The main differences with respect to local file systems are three:

1. The block sizes are divided on different machines, so for the position not only the memory address must be specified but also the machine itself.
2. Since operations must act on different machines to work on the same file, the latency inevitably increases, adding the communication network latency to the inherent mechanical latency.
3. Due to the previous point, block sizes are larger than those typical of local file systems — on the order of megabytes — in order to maximize the amount of data extracted from each position.

Three *ideal* requirements for a DFS:

1. **Transparency:** the user should not feel any difference working with a DFS compared to a local file system; the dispersion of data on multiple computers must be as little evident as possible.
2. **Fault tolerance:** the ability to keep all operations active despite the failure of some infrastructures.
3. **Scalability:** extensibility to new nodes in the cluster in a simple and practical way.

5.2. Example: HDFS

The **HDFS** (*Hadoop Distributed File System*) is an open-source DFS created in 2006 to emulate through reverse engineering the *GFS* (*Google File System*) introduced by Google three years earlier. It was created to manage and process large amounts of data on *commodity hardware* — relatively cheap and easily available devices — using standard TCP/IP protocols.

In HDFS there is a special node called the **namenode**, which contains the metadata of all the files and has the role of accepting or rejecting requests from users. All other nodes, called **datanodes**, host file blocks of 128 MB each.

i Block size: local vs distributed file systems

Local file systems (e.g. ext4, NTFS) typically use a block size of **4 KB**. HDFS uses a dramatically larger default block size of **128 MB** (increased from 64 MB in Hadoop 2.x; the value is configurable). The large block size serves two purposes:

- **NameNode memory:** the NameNode must keep an entry in memory for every block across the entire cluster. Fewer, larger blocks means far fewer metadata entries to track, reducing NameNode RAM pressure.
- **Seek overhead:** for the large sequential reads that are typical in data-analysis workloads, spending time seeking to the start of a 128 MB block is amortised over much more useful data transfer than it would be for a 4 KB block.

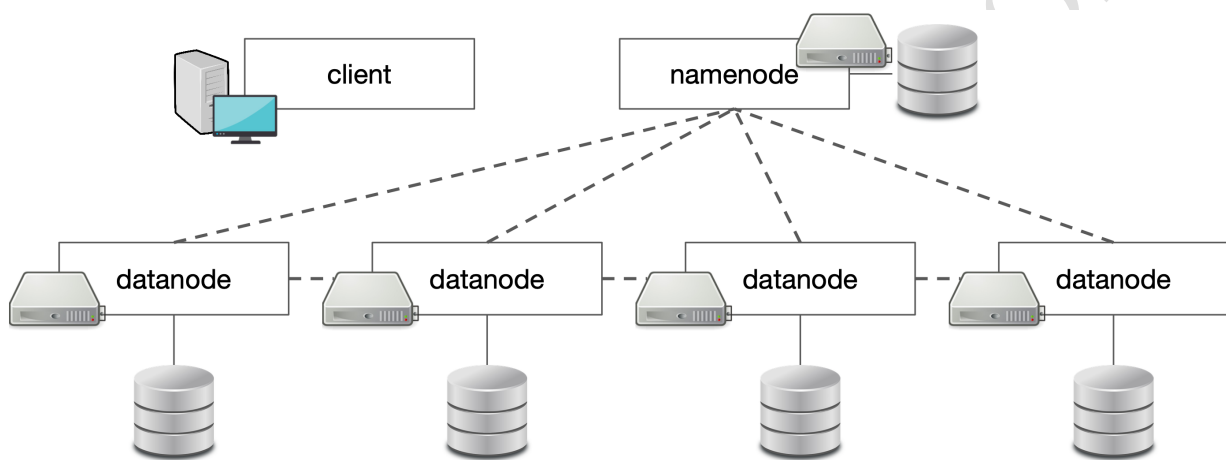


Figure 5.2.: HDFS system diagram

Each operation is divided into four steps:

1. The client sends its request (read, write, delete, etc.) directly to the namenode.
2. The namenode verifies authorizations and permissions, then provides the client with the positions of the relevant data blocks.
3. The client acts directly on the datanodes containing those blocks, without going through the namenode.
4. The datanodes signal to the client that the operation is complete.

Fault tolerance in HDFS involves three types of failures:

- **Network instability:** each datanode repeatedly sends a *heartbeat* signal to the namenode. When this heartbeat is missing, the namenode excludes that node until the heartbeat is restored.
- **Datanode failure:** data blocks are copied to different nodes (by default, three copies) so that if a datanode fails, the data is not lost.
- **Namenode failure:** this is the worst possible failure — a *Single Point Of Failure* (SPOF). The data itself is not lost, but the information on how to compose files from blocks is lost. To mitigate this, HDFS introduces a **Secondary NameNode**.

i Secondary NameNode — what it is and what it is not

The Secondary NameNode is **not** a hot standby. If the Primary NameNode crashes, HDFS does **not** automatically recover — manual intervention is required.

Its actual job is *checkpointing*: it periodically fetches the Primary NameNode's **fsimage** (a snapshot of the entire namespace) together with the **edit log** (a journal of every change made since the last snapshot), merges them into a new, up-to-date fsimage, and sends the result back to the Primary NameNode.

Without this merging the edit log would grow without bound, and restarting the Primary NameNode after a crash would require replaying an arbitrarily long sequence of edits — making startup extremely slow. Periodic checkpointing keeps the edit log short and the startup time bounded.

5.3. CAP Theorem

For distributed systems, a theorem mathematically demonstrated by Eric Brewer — known as the **CAP theorem** — states the following:

Theorem 5.1. *It is **impossible** for a distributed system to provide simultaneously more than **two** of the three CAP guarantees (consistency, availability and partition tolerance).*

The three guarantees:

1. **Consistency (C)**: *every* request from *every* client to *any* datanode must result in the *same* copy of the saved information. To apply this, a “waiting line” is introduced: after a client modifies a file, all other copies are updated before any further operation is allowed.

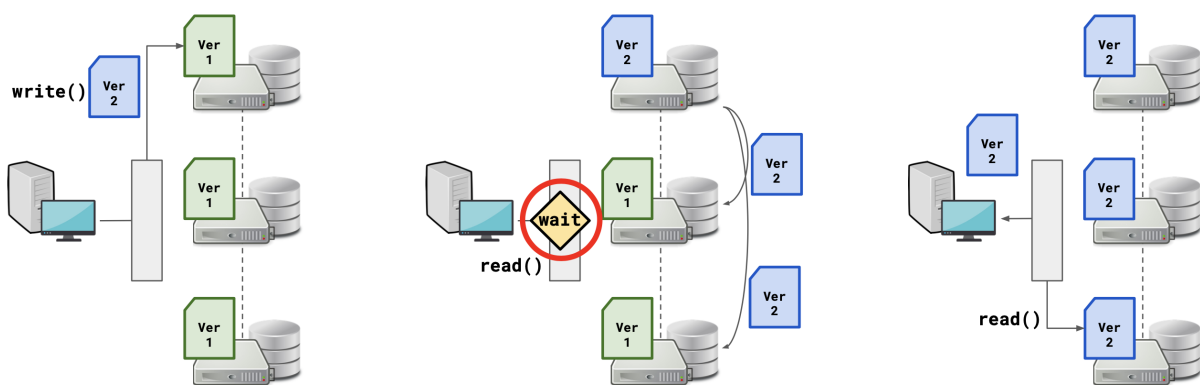


Figure 5.3.: Example of consistency in case of writing to a file

2. **Availability (A)**: *every* request from *every* client must correspond to an *immediate* response, *regardless* of the state of the data and the non-functioning of some network nodes. This means minimizing latency and supporting multiple simultaneous clients. However, this creates a *race competition* — a client may read an outdated version right after writing.

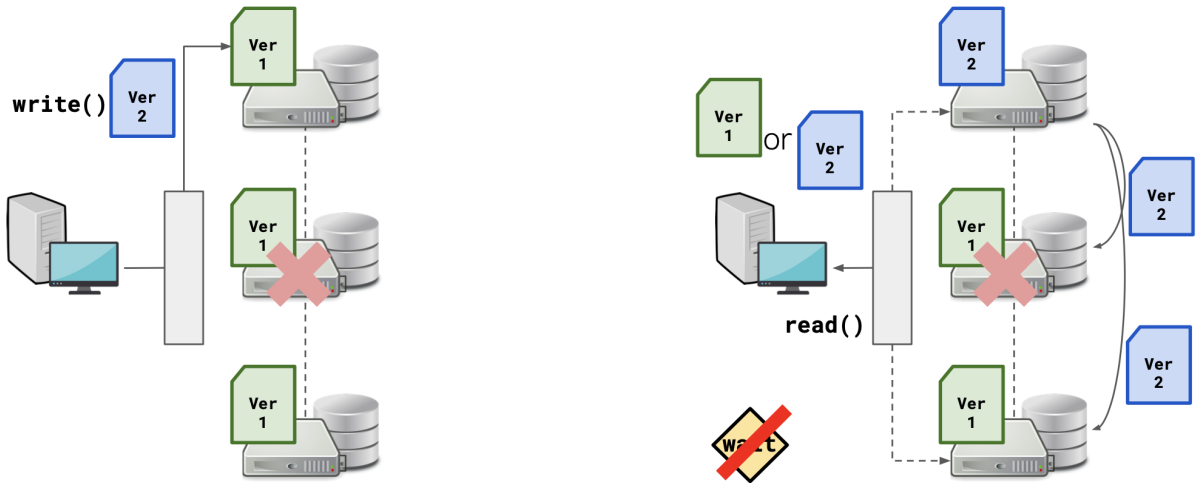


Figure 5.4.: Example of availability in case of writing and subsequent reading

3. **Partition tolerance (P)**: the system *must* continue to operate *regardless* of cuts to the communication lines between nodes (*network partitions*).

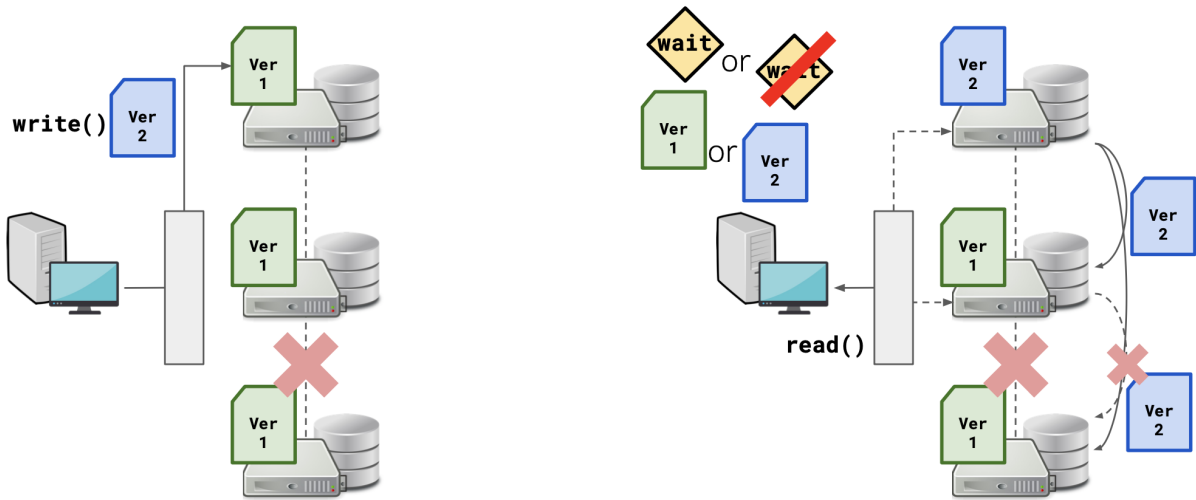


Figure 5.5.: Example of partition tolerance in case of writing to a file

In a distributed system it is not possible to avoid partition tolerance, so the practical choice reduces to:

- **CP system**: tendentially slower in response but guarantees consistency in reading by multiple clients.
- **AP system**: fast in reading but no guarantee that the available data is updated to the latest version.

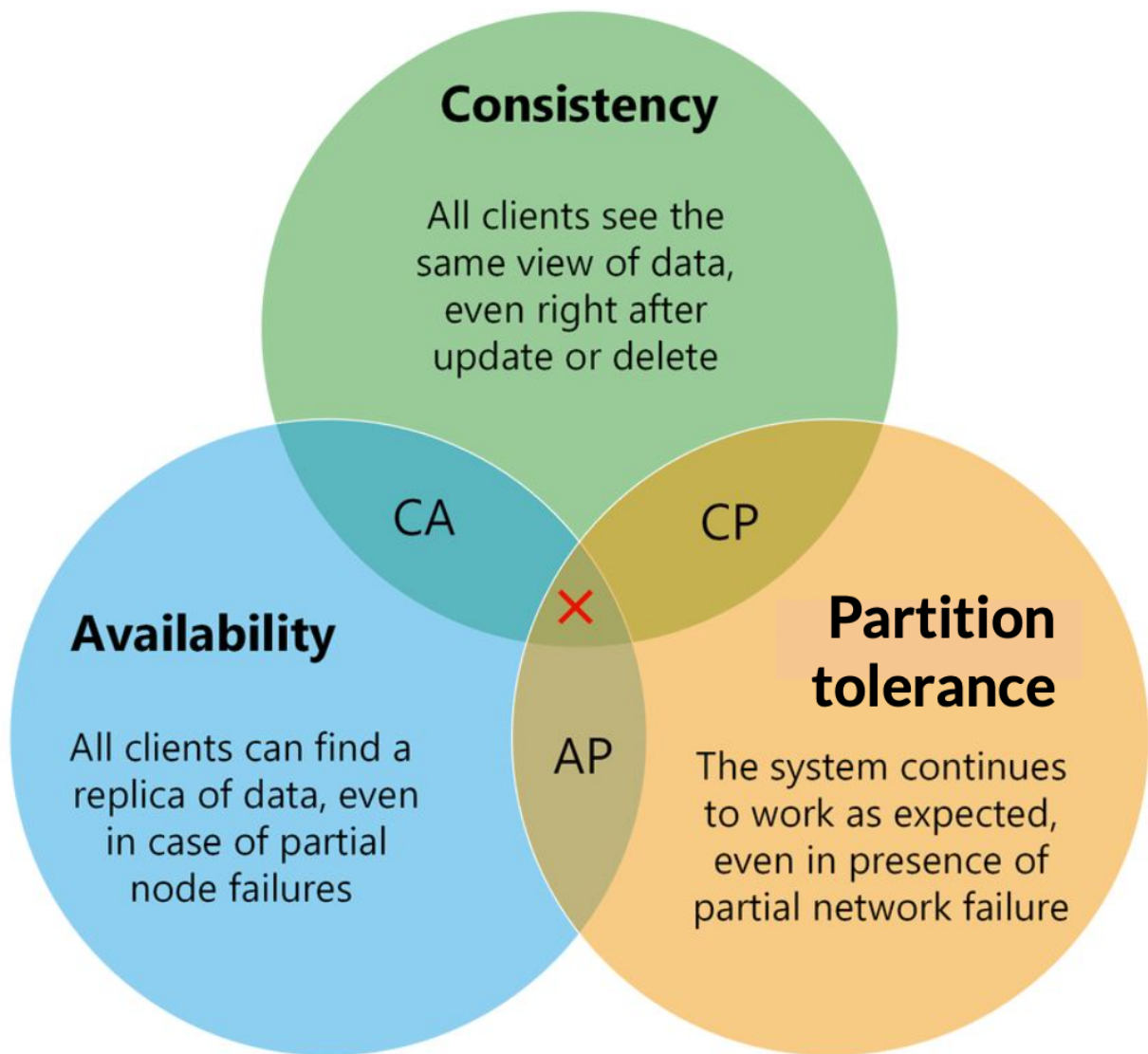


Figure 5.6.: CAP theorem diagram

The choice depends on the use case: high-level computing systems like *Lustre* prefer AP for rapid access to constantly updated data; banking systems prefer CP for guaranteed consistency. The AC combination is not viable for distributed systems but is used for vertically scalable systems where partition tolerance is not needed.

5.4. Exercises

Exercise 5.1 (*Writing a File to HDFS*)

Explain how a client interacts with HDFS when it wants to write a file. Describe the roles of the NameNode and DataNodes in this process.

Solution

The client first contacts the **NameNode** with the write request. The NameNode checks authorisation and returns the list of DataNodes that should host each block. The client then connects **directly** to those DataNodes — data is **not** routed through the NameNode. Each block is written through a **replication pipeline**: client → DN1 → DN2 → DN3 (default replication factor 3). Acknowledgements propagate back to the client and the DataNodes report the new block to the NameNode.

Exercise 5.2 (Heartbeat in HDFS)

What is meant by a heartbeat in a distributed system such as HDFS? What is its main role, and what happens if heartbeats stop?

Solution

It is a periodic signal the DataNodes send to the NameNode to make sure the DataNodes are working and available. Its main functionality is to prevent Network Failures, so in case that “heartbeat” stops the NameNode excludes that DataNode until it is available and redirects to another DataNode containing the same data.

Exercise 5.3 (DataNode Failure)

What happens in HDFS when a DataNode fails? How does HDFS detect the failure and what corrective actions does it take?

Solution

The NameNode detects the failure when heartbeats from the DataNode stop. The data on the dead node is “lost” from that copy, but other replicas survive on different DataNodes, so the NameNode redirects clients to those. Crucially, the NameNode also triggers **re-replication**: it identifies every block whose replication factor dropped below the configured value and instructs surviving DataNodes to copy those blocks to other live DataNodes, restoring the target factor. Without this step durability would degrade with every subsequent failure.

Exercise 5.4 (Fault Tolerance in HDFS)

How is fault tolerance implemented in HDFS? Describe the mechanisms used to ensure data durability in the presence of hardware failures.

Solution

Fault tolerance is divided in three categories:

- Network instability: The NameNode can not interact with a DataNode, this is avoided by having multiple DataNodes with copies of the data, as DataNodes are continuously sending signals to the NameNode when the signal fails that Node is excluded and redirected to another working node while the signal is restored.
- DataNode failure: The data of a DataNode is lost, to avoid that multiple copies of the

data is stored in different DataNodes, so the NameNode uses one of the copies while the failed one is being restored

- NameNode failure: The worst case, this can be avoided by having a secondary NameNode that is activated, but needs to be done manually.

Exercise 5.5 (*AC in a Distributed File System*)

Can we create a system that behaves as AC (Available + Consistent) in a distributed file system? Relate your answer to the CAP theorem.

Solution

No. The CAP theorem says a distributed system can guarantee at most two of {C, A, P}. **Partition tolerance cannot be designed away** — network links between nodes will eventually fail (cable cuts, switch failures, congestion), and when they do the system is partitioned whether we like it or not. Since P is forced upon us, the practical choice is **CP** or **AP**. AC is only viable for non-distributed (single-node, vertically scaled) systems where there is no inter-node network to fail.

Exercise 5.6 (*CAP Theorem Explained*)

Explain the CAP Theorem in the context of a distributed computing system. Define each of the three properties and explain why only two can be fully guaranteed simultaneously.

Solution

- **Consistency (C)**. Every read returns the most recent write — all nodes see the same value at the same time.
- **Availability (A)**. Every request receives a (non-error) response, regardless of the state of any individual node.
- **Partition tolerance (P)**. The system continues to operate correctly even when the inter-node network is split into disconnected groups (a *partition*).

Only two can hold simultaneously because during a partition the system must choose: either refuse some requests on the minority side to keep the data consistent (sacrifices A) or accept divergent writes on each side and reconcile later (sacrifices C). Since real networks always eventually partition, the practical choice in any distributed system is **CP** or **AP**.

Exercise 5.7 (*CAP Theorem and Implementation*)

Describe the CAP theorem and discuss how it guides the design and implementation of distributed databases.

Solution

CAP says a distributed database must pick **CP** or **AP**. That choice dictates the architecture:

- **CP designs** use leader-based replication with quorum writes; during a partition the minority side refuses writes to preserve consistency (e.g. HBase, MongoDB default, Spanner).
- **AP designs** use multi-master replication with eventual consistency and conflict resolution (last-write-wins, vector clocks, CRDTs) — every node keeps serving requests during a partition and reconciles state afterwards (e.g. Cassandra, DynamoDB, Riak).

So CAP doesn't just classify databases — it shapes the replication protocol, the consistency model, and the failure-mode behaviour the application must handle.

Exercise 5.8 (CAP and Databases)

Does the CAP theorem apply to databases? Discuss with examples of real database systems.

Solution

Yes — any database that is replicated across nodes is subject to CAP.

- **CP**: HBase, MongoDB (default), Google Spanner, etcd, ZooKeeper.
- **AP**: Cassandra, DynamoDB, Riak, CouchDB.
- **Single-node RDBMS** (PostgreSQL, MySQL on one host) are effectively **CA** because there is no inter-node partition to tolerate; clustering them turns them into CP systems.

Some systems are **tunable** per operation (Cassandra: $R + W > N$ gives stronger consistency; MongoDB: read-concern levels), letting the application slide along the CP–AP spectrum.

Exercise 5.9 (CAP and ATM Booths)

Describe how the CAP theorem applies to ATM booths of a banking system. Which two properties does this system typically prioritise, and why?

Solution

Banking systems prioritise **CP**. During a network partition between an ATM and the central ledger, the ATM will **refuse** the withdrawal rather than risk inconsistent balances or overdrafts — i.e. availability is sacrificed to preserve consistency. Money has integrity constraints that cannot tolerate divergent replicas: a brief “sorry, please try again later” is acceptable; a double-spend is not.

Exercise 5.10 (CAP and Live Chat Systems)

Describe how the CAP theorem applies to a live chat system that wants to deliver messages as quickly as possible. Which CAP combination does this system choose?

Solution

Live chat picks **AP**. Users care that messages are delivered fast and that the service stays up under network turbulence; they tolerate brief inconsistencies (messages temporarily showing in a different order on different devices, late delivery on a slow link). Strong consistency would force every message through a single coordinator and add latency the users would notice. Eventual consistency is fine here.

Exercise 5.11 (*Social Media DBMS*)

How would you classify the database management system of a social media platform that wants to upload messages as quickly as possible? Describe it using the CAP theorem framework (as an AP system).

Solution

AP. Posts must be accepted quickly and the timeline must stay reachable at all times. It is acceptable that a new post appears on followers' feeds with a few seconds' lag, or that the like-count is briefly stale on different servers — eventual consistency suffices. The cost of refusing a write (lost engagement) is much higher than the cost of a slightly stale read. Hence the typical storage layer is a multi-master, eventually consistent system like Cassandra or DynamoDB.

Exercise 5.12 (*AP Systems*)

Discuss Availability–Partition tolerance (AP) distributed systems according to the CAP theorem. What consistency guarantees do AP systems provide, and what do they sacrifice?

Solution

AP systems guarantee that every node keeps responding to requests during a partition, sacrificing strict consistency. They typically provide **eventual consistency**: replicas may diverge temporarily but converge to the same state once the partition heals and writes propagate. Conflicts are resolved by mechanisms such as **last-write-wins** timestamps, **vector clocks** or **CRDTs**. The application has to be prepared to read stale data and, sometimes, see anomalies like read-your-own-writes failures unless extra session guarantees are layered on top.

Exercise 5.13 (*Consistency vs Availability Trade-off*)

Discuss the consistency and availability trade-off in distributed NoSQL databases. How do different systems position themselves along this spectrum?

Solution

NoSQL stores span the whole spectrum:

- **Strongly consistent (CP)**. HBase, Spanner, ZooKeeper, MongoDB (default). Slow writes during partitions, may refuse requests, but never serve stale data.

- **Tunable.** Cassandra (per-query consistency level: $R+W > N$ for quorum), DynamoDB (eventual or strong reads), MongoDB (write-concern levels). The operator picks the trade-off per workload.
- **Eventually consistent (AP).** Riak, Voldemort, CouchDB. Lowest latency, highest availability, reads may be stale, conflicts resolved post-hoc.

The choice is essentially how much **staleness** / **divergence** the application tolerates in exchange for **latency** / **availability**.

Exercise 5.14 (*CAP in Relational and NoSQL DBs*)

Explain the CAP theorem in the context of both relational and NoSQL databases. How do traditional RDBMS systems typically position themselves compared to NoSQL systems?

Solution

Traditional single-node RDBMS (PostgreSQL, MySQL) are effectively **CA** — there is no partition because there is only one node. As soon as they are clustered with synchronous replication they become **CP**: writes are blocked on the minority side of a partition to preserve consistency.

NoSQL systems were created precisely to give up some consistency for **horizontal scale and availability**, so most lean **AP** (Cassandra, DynamoDB, Riak). A few NoSQL systems remain CP (HBase, MongoDB-default) when consistency is non-negotiable. The shift from RDBMS to NoSQL was, in CAP terms, the explicit acceptance of eventual consistency as the price of always-on, geo-distributed scale.

Exercise 5.15 (*"C" in CAP vs "C" in ACID*)

What is the difference between the “C” (Consistency) in the CAP theorem and the “C” (Consistency) in the ACID model? Are they the same concept?

Solution

No, they are different concepts.

- **CAP-C** is *linearizability*: every read sees the most recent write, and all replicas agree on the value at every instant. It is a property of how replicas are kept in sync across the network.
- **ACID-C** is *integrity*: a transaction moves the database from one valid state to another, preserving constraints (foreign keys, uniqueness, business rules). It is a property of a single transaction on a single logical database, independent of replication.

A single-node RDBMS satisfies ACID-C without CAP ever applying; a distributed AP store may violate CAP-C while still respecting ACID-C within each replica.

Exercise 5.16 (*CAP Consistency*)

Explain the concept of Consistency from the perspective of the CAP theorem. How does it differ from eventual consistency?

Solution

CAP-Consistency (linearizability) requires that **every read returns the most recent committed write** — as if there were a single global copy of the data. To uphold it on N replicas, every write must be propagated and acknowledged before any later read can succeed. **Eventual consistency** is weaker: it only promises that **if no new writes occur, all replicas will converge to the same value**. In the meantime, reads may return stale data, and different clients may briefly see different values. Eventual consistency is the consistency model of AP systems; CAP-C is the one CP systems uphold.

Exercise 5.17 (*Partition Tolerance*)

What is the meaning of “Partition” in the context of the CAP theorem? Why is partition tolerance generally considered non-negotiable in distributed systems?

Solution

A **partition** is a state of the network in which some subset of nodes can no longer communicate with the rest — caused by cable cuts, switch failures, congestion, routing problems, or whole-datacentre outages. **Partition tolerance** is the property of continuing to operate correctly when this happens.

It is non-negotiable because partitions are not a design choice — they will happen in any sufficiently long-lived networked system. A “non-P” design would simply fail (or worse, silently corrupt data) on the first network blip, which is unacceptable.

Exercise 5.18 (*Why Partition Tolerance is Crucial*)

Why is partition tolerance crucial in distributed systems? What would happen to a distributed system that does not tolerate network partitions?

Solution

Because network failures (link cuts, switch faults, congestion, dropped packets) are **routine**, not exotic, in any system that spans more than one host. A system that did not tolerate partitions would either become completely unavailable on the first link failure, or — worse — keep accepting writes on both sides without reconciliation, silently producing inconsistent state. Neither is acceptable in production, which is why P is mandatory and the real CAP choice is between **CP** and **AP**.

Exercise 5.19 (*Ways to Partition Databases*)

What are the different ways to partition (shard) databases? Describe at least three partitioning strategies and their trade-offs.

Solution

- **Range partitioning.** Each shard owns a contiguous key range (rows 0–1M on node A, 1M–2M on node B). Simple, range scans are cheap, but **hotspots** appear on popular ranges.
- **Hash partitioning.** $\text{Shard} = \text{hash}(\text{key}) \bmod N$. Distributes load evenly, but adding a node reshuffles every record unless you use **consistent hashing**, and range scans become expensive (a scan visits every shard).
- **Directory / lookup-based partitioning.** A separate metadata service maps each key to a shard, allowing arbitrary placement. Maximally flexible, but the lookup service is a SPOF and an extra hop.
- **Vertical partitioning.** Different *columns* or *tables* live on different nodes (split hot vs cold attributes). Improves cache locality but joins across partitions become expensive.

Choosing a strategy trades off **load balance**, **range-query locality**, **rebalance cost** and **operational complexity**.

Exercise 5.20 (*MongoDB and CAP*)

MongoDB is a NoSQL database focused on consistency. Does it mean it is CA or CP (or perhaps somewhere in the middle)? Motivate your answer with reference to MongoDB's replication and failover mechanisms.

Solution

MongoDB is **CP** (not CA — CA is impossible for any real distributed system). A replica set has a single **primary** that accepts writes and replicates them to secondaries via the oplog. On a partition, only the side holding a majority of nodes can elect a primary and continue serving writes; the minority side **rejects writes** to preserve consistency — that is the CP trade-off, sacrificing availability on one side of the partition.

It is “in the middle” only in the sense of being **tunable on reads**: reading from the primary gives linearizable behaviour, while reading from secondaries (`readPreference: secondary`) gives faster but potentially stale data — an AP-like operation point on top of the otherwise CP architecture.

6. Databases

Definition 6.1 (Database). A **database** is a set of *records* (generally related to each other) organized and processed by a dedicated *management system*.

The term *database* encompasses three concepts: the *records*, the *collections* that group the individual records, and the *relations* that exist both between records of the same collection and between records of different collections.

The dedicated management system is called a **database management system** (DBMS), whose main functionalities are:

1. *Create* and *define* data
2. *Select* and *query* data
3. *Manipulate* and *process* data

The main differences between management through a file system and through a DBMS:

1. With DBMS, operations can take place at the level of the *single records*, not just whole files.
2. The data are no longer “scattered” between different files and directories, but all located within a single entity that also includes the relationships between the data.
3. In data analysis, different algorithms applied to the same data set do not require a new copy of the file — it is possible to select a subset of data without replicating it.

6.1. Database Models

In databases we distinguish two aspects:

1. **Model** (or **schema**): the *description* of the data and the *relations* between the various parts of the dataset.
2. **Instance** (or **state**): the actual content in the database, that is the *data*.

In general, we distinguish two types of models: *relational* and *non-relational*; the former are the standard for structured data, and are also called *SQL databases*; the latter are typically made for semi-structured and unstructured data, and are referred to as *NoSQL* (“Not Only SQL”).

6.2. Relational Databases

The foundations of relational databases were laid in 1970 by **Edgar F. “Ted” Codd** at IBM, in his landmark paper “*A Relational Model of Data for Large Shared Data Banks*”. Codd proposed

organising data into **relations** (tables) and defined **relational algebra** — a set of formal operations on those relations, including *select* (filter rows), *project* (choose columns), *join* (combine tables), *union*, *intersection*, and *difference*. Every SQL query can be understood as a composition of these algebraic operations. The most widely used RDBMS today is *MySQL*, an open source management system.

Relational databases require a predefined schema and are perfectly suited to structured data. The collections of RDBs (*Relational DataBases*) are **tables** organized in columns (categories) and rows (individual records).

Three elements are clearly distinguished:

- The *records*, corresponding to the single rows of the table, also called *tuples*.
- The **attributes**, corresponding to the single columns.
- The **primary key**: a specific attribute (or combination of attributes) that must be *unique* (different tuples cannot have the same key) and *minimal* (the minimum number of attributes required to define the key).

To link the various tables, **foreign keys** are used: attributes or combinations of attributes in one table that act as a primary key in another table.

Each set of operations performed on a relational database is called a **transaction**, and each must respect four criteria summarized in the acronym **ACID**:

- **Atomicity** (*all-or-nothing*): all operations in a transaction are either performed completely or the entire transaction is aborted.
- **Consistency**: each transaction must bring a database from a *valid state* to *another valid state*, preserving all imposed constraints.
- **Isolation**: each transaction must generate the same results that would have been obtained in a single-user environment, avoiding conflicts when multiple users act simultaneously.
- **Durability**: following a transaction, the state of the database must remain the same until it is modified by another transaction.

6.2.1. Transaction Control in SQL

ACID properties are enforced through **transactions**. SQL provides three transaction control statements:

```
START TRANSACTION;                                -- begin a transaction

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;                                            -- make changes permanent
```

If an error occurs, roll back all changes made since **START TRANSACTION**:

```
START TRANSACTION;
DELETE FROM Orders WHERE order_date < '2020-01-01';
ROLLBACK;                                -- undo the DELETE
```

SAVEPOINT allows partial rollbacks within a transaction:

```
START TRANSACTION;
INSERT INTO log VALUES ('step 1');
SAVEPOINT sp1;
INSERT INTO log VALUES ('step 2');
ROLLBACK TO SAVEPOINT sp1;              -- undo only step 2
COMMIT;                                -- commits step 1
```

6.2.2. Transaction Isolation Levels

SQL defines four isolation levels that control the trade-off between consistency and concurrency:

Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

MySQL's default is **REPEATABLE READ**. Higher isolation levels reduce anomalies but increase lock contention. Set the level with:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

6.3. SQL

The standard language for any DBMS related to relational databases is **SQL** (pronounced *sequel*, from the original name **SEQUEL** — *Structured English Query Language* — given to it by IBM researchers in **1974**). Development at IBM began that year, and SQL was adopted as an **ISO/ANSI standard in 1987**. The standard has been revised many times since: SQL:1992 added important query features, SQL:1999 introduced recursive queries and triggers, SQL:2003 added window functions and XML support, and SQL:2016 added JSON support, among other revisions.

SQL is a **declarative** language: what is written describes *what* must be done, rather than *how* it should be done. It is structured in three parts:

- **DDL** (*Data Definition Language*): defines and modifies the structure of the database.
- **DML** (*Data Manipulation Language*): inserts, updates, deletes and queries records.
- **DCL** (*Data Control Language*): manages access permissions and security.

6.3.1. Creating Databases and Tables

Each SQL instruction must end with a semicolon.

```
-- View all databases
SHOW DATABASES;

-- Create a database
CREATE DATABASE IF NOT EXISTS MyDataBase;

-- Select a database to work with
USE MyDataBase;

-- View tables in the current database
SHOW TABLES;

-- Create a table
CREATE TABLE Users (
    UserID    varchar(30),
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);
```

SQL variable types include: int, float, varchar(n), char(n), date, datetime, boolean, text, and more.

6.3.2. Keys

```
-- Single-attribute primary key
CREATE TABLE Users (
    UserID    varchar(30) PRIMARY KEY,
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);

-- Multi-attribute primary key
CREATE TABLE Users (
    UserID    varchar(30),
```

```

    BadgeNum    int,
    FirstName   varchar(255),
    LastName    varchar(255),
    Age         int,
    OtherAttr   float,
    PRIMARY KEY (UserID, BadgeNum)
);

-- Foreign key
CREATE TABLE Authorizations (
    BadgeID int PRIMARY KEY,
    Since   date,
    Level   int,
    Sector  int
);

CREATE TABLE Users (
    UserID      varchar(30) PRIMARY KEY,
    BadgeNum    int,
    FirstName   varchar(255),
    LastName    varchar(255),
    Age         int,
    OtherAttr   float,
    FOREIGN KEY (BadgeNum) REFERENCES Authorizations(BadgeID)
);

```

6.3.2.1. AUTO_INCREMENT / SERIAL

When using a simple integer surrogate key, you rarely want to assign IDs manually. The `AUTO_INCREMENT` attribute (MySQL syntax) tells the database to generate a unique, incrementing integer value automatically whenever a new row is inserted:

```

CREATE TABLE users (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    name  VARCHAR(100)
);

```

Every `INSERT` that omits `id` (or passes `NULL`) receives the next available integer. PostgreSQL uses the `SERIAL` shorthand (or the standard `GENERATED ALWAYS AS IDENTITY`) for the same effect.

⚠ Common mistake

Do **not** write `PRIMARY KEY` next to each individual attribute for a multi-attribute key — SQL will interpret each as a separate primary key and return `ERROR 1068 (42000): Multiple`

primary key defined. Use the PRIMARY KEY (attr1, attr2) syntax at the end of the table definition instead.

6.4. Relationships in Relational Databases

A relational database models **relationships** between entities. There are three fundamental relationship types, each requiring a different schema design.

The table below summarises the notation commonly used in **Entity-Relationship (ER) diagrams** alongside the corresponding SQL implementation:

Relationship	Crow's foot notation	SQL implementation
One-to-one (1:1)	\ --\	FOREIGN KEY with UNIQUE constraint
One-to-many (1:N)	\ ---<	FOREIGN KEY on the “many” side
Many-to-many (N:M)	>---<	Junction/bridge table with two FOREIGN KEYs

6.4.1. One-to-One (1:1)

Each record in table A corresponds to **at most one** record in table B. The Foreign Key is placed on either side, but must have a UNIQUE constraint to enforce the 1:1 cardinality.

Example — Student Thesis: each student has at most one thesis.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);

CREATE TABLE Theses (
    thesis_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    student_id INT UNIQUE,           -- UNIQUE enforces 1:1
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

6.4.2. One-to-Many (1:N)

Each record in table A (the “one” side) corresponds to **many** records in table B (the “many” side). The Foreign Key must be placed on the **“many” side** — never on the “one” side.

Example — Course Lectures: one course has many lectures.

```
CREATE TABLE Courses (
  course_id  INT          PRIMARY KEY,
  name       VARCHAR(100) NOT NULL
);

CREATE TABLE Lectures (
  lecture_id INT          PRIMARY KEY,
  title      VARCHAR(200) NOT NULL,
  course_id  INT,          -- FK on the "many" side
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

i Note

Placing the FK on the “one” side (e.g., storing a list of lecture IDs inside the Course row) violates First Normal Form and is not possible in standard SQL. The FK always belongs on the table that can have multiple records per parent.

6.4.3. Many-to-Many (N:M)

Records in table A can relate to **many** records in table B and vice versa. SQL cannot express this directly with a single FK — a **junction table** (also called association or bridge table) is required.

Example — Student Course: students enrol in many courses; each course has many students.

```
CREATE TABLE Enrollments (
  student_id  INT NOT NULL,
  course_id   INT NOT NULL,
  enrollment_date DATE,
  PRIMARY KEY (student_id, course_id),          -- composite PK ensures
  ↪ uniqueness
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

The composite Primary Key (student_id, course_id) guarantees that a student cannot be enrolled in the same course twice, and the combination is the minimal unique identifier for each enrolment record.

6.4.4. Constraints

Constraints are keywords written after the variable type:

- **UNIQUE**: each value in the column must be unique.
- **NOT NULL**: no NULL values allowed.
- **DEFAULT <value>**: sets a default value when none is provided.
- **CHECK (<condition>)**: returns an error if the condition is not respected.

```
CREATE TABLE Users (
    UserID    varchar(30) PRIMARY KEY,
    BadgeNum  int          NOT NULL UNIQUE,
    FirstName varchar(255) NOT NULL DEFAULT '',
    LastName  varchar(255) NOT NULL DEFAULT '',
    Age       int          DEFAULT 18,
    OtherAttr float        CHECK (OtherAttr > 0)
);
```

To inspect a table's structure:

```
DESCRIBE Users;
```

Field	Type	Null	Key	Default	Extra
UserID	varchar(30)	NO	PRI	NULL	
BadgeNum	int	NO	UNI	NULL	
FirstName	varchar(255)	NO			
LastName	varchar(255)	NO			
Age	int	YES		18	
OtherAttr	float	YES		NULL	

6.4.5. Altering Tables

Altering a table's structure involves all the records present in it. This is one of the main challenges in working with relational databases — maintaining consistency is critical (a process called **database normalization**).

```
-- Add a column
ALTER TABLE Users
ADD COLUMN Role char(1) DEFAULT 'A' CHECK (Role IN ('A','B','C'));

-- Change default value
ALTER TABLE Users
ALTER Role SET DEFAULT 'B';

-- Add a constraint (redefines the attribute)
ALTER TABLE Users
```

```

MODIFY Role char(1) NOT NULL;

-- Add a primary key after creation
ALTER TABLE AddressBook
ADD PRIMARY KEY (TelephoneNumber, User);

-- Add a foreign key
ALTER TABLE AddressBook
ADD CONSTRAINT FK_user
FOREIGN KEY (User) REFERENCES Users(UserID);

-- Remove a constraint
ALTER TABLE AddressBook
DROP CONSTRAINT FK_user;

-- View table creation info
SHOW CREATE TABLE AddressBook;

-- Drop a column
ALTER TABLE Users
DROP COLUMN Role;

-- Drop a table
DROP TABLE IF EXISTS Users;

```

Warning

Dropping a table deletes all records within it **and** all foreign key connections to other tables, potentially making the entire database unusable. Use with caution.

6.4.6. Data Insertion

```

INSERT INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)
VALUES ('usr:00001', 100, 'Rei', 'Ayanami', 1.8);

-- Multiple records at once
INSERT INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)
VALUES
    ('usr:00002', 101, 'Asuka', 'Soryu Langley', 3.5),
    ('usr:00003', 102, 'Shinji', 'Ikari', 2.9),
    ('usr:00004', 103, 'Toji', 'Suzuhara', 4.7),
    ('usr:00005', 104, 'Kaworu', 'Nagisa', 3.0);

```

6.4.7. Modifying Records

```
UPDATE Users
SET    Role = 'C'
WHERE  (LastName = 'Ayanami');
```

Wildcards extend WHERE conditions for strings: `_` is a char wildcard (any single character) and `%` is a string wildcard (any sequence of characters). They are used with `LIKE`:

```
-- All records where FirstName contains 'o'
UPDATE Users
SET    Role = 'B'
WHERE  (FirstName LIKE '%o%');

-- All records where the third character of FirstName is 'i'
UPDATE Users
SET    OtherAttr = 5.0
WHERE  (FirstName LIKE '__i%');
```

6.4.8. Deleting Records

```
DELETE FROM Users
WHERE  (LastName LIKE '%p_');
```

6.4.9. Conditional Expressions: CASE

The `CASE` expression evaluates conditions row-by-row and returns a value, similar to an `if-else` chain:

```
SELECT name,
       salary,
       CASE
         WHEN salary < 30000 THEN 'Low'
         WHEN salary < 70000 THEN 'Medium'
         ELSE 'High'
       END AS salary_band
FROM Employees;
```

`CASE` can also be used in `ORDER BY`, `GROUP BY`, and aggregate functions.

6.4.10. Data Selection

i How to Read (and Write) a SELECT Statement: Execution Order

Although you write **SELECT** first, the database executes the clauses in this order:

Step	Clause	What it does
1	FROM / JOIN	Identify the source tables and combine them
2	WHERE	Filter rows before grouping
3	GROUP BY	Group the filtered rows
4	HAVING	Filter groups (like WHERE but on aggregate results)
5	SELECT	Evaluate expressions and select columns
6	ORDER BY	Sort the result
7	LIMIT / OFFSET	Restrict the number of returned rows

This order explains why you **cannot** use a column alias defined in **SELECT** inside a **WHERE** clause — **WHERE** is evaluated before **SELECT**.

The **SELECT** command is the most powerful and widely used in SQL, enabling the extraction of records according to conditions specified by the user — the so-called **queries**.

```
-- View entire table
SELECT * FROM Users;

-- Limit displayed records
SELECT * FROM Users
LIMIT 3;

-- Select specific attributes
SELECT FirstName, BadgeNum FROM Users;

-- Distinct values (no repetitions)
SELECT DISTINCT FirstName FROM Users;

-- Sort (ascending by default; DESC for descending)
SELECT FirstName FROM Users
ORDER BY FirstName;

-- Filter with WHERE
SELECT FirstName, BadgeNum FROM Users
WHERE (OtherAttr > 0);

-- Create a VIEW
CREATE VIEW OneThree AS
```

```

SELECT UserID, BadgeNum, LastName
FROM Users
WHERE (OtherAttr BETWEEN 1 AND 3);

SELECT * FROM OneThree;
SHOW CREATE VIEW OneThree;

-- Mathematical operations on attributes
SELECT OtherAttr * 2, POWER(BadgeNum, 2) FROM Users
WHERE (OtherAttr > 1.8);

```

Built-in aggregate functions: COUNT(), MAX(), MIN(), SUM(), AVG().

Aliases for attributes and tables:

```

SELECT DISTINCT FirstName AS UniqueNames
FROM Users;

-- AS can be omitted
SELECT FirstName NamesEndingI
FROM Users U
WHERE (FirstName LIKE '%i');

```

GROUP BY and HAVING:

```

SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
ORDER BY TotalCities DESC;

-- HAVING filters on aggregated results
SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
HAVING TotalCities > 1
ORDER BY TotalCities DESC;

```

 Implicit JOIN = Cartesian Product + Filter

An **implicit join** uses a comma-separated FROM clause and a WHERE condition to filter the result:

```
SELECT c.name, o.total
FROM Customers c, Orders o
WHERE c.customer_id = o.customer_id;
```

This is equivalent to an explicit `INNER JOIN`, but the database engine first computes the full **Cartesian product** (every row of `Customers` $\times$ every row of `Orders`) and then filters. For large tables this is highly inefficient and easy to make incorrect by forgetting the `WHERE` condition. Always prefer explicit `JOIN ... ON` syntax.

JOINS combine tables through matching attribute values:

```
-- INNER JOIN: only records with matching keys in both tables
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
INNER JOIN Orders o ON s.Code = o.Code;

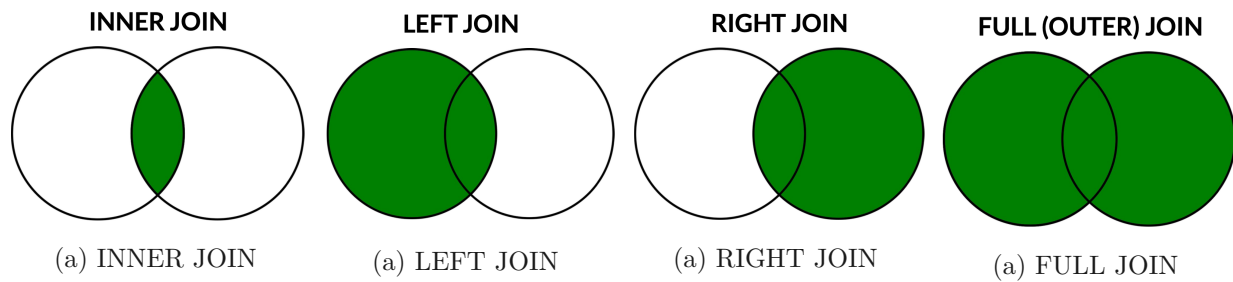
-- LEFT JOIN: all records from the left table; NULLs for non-matching right
-- records
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code;

-- RIGHT JOIN: all records from the right table; NULLs for non-matching left
-- records
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;

-- FULL (OUTER) JOIN: all records from both tables; NULLs where no match
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code
UNION
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;
```

Graphic illustration of the join processes

6.4.11. Other JOIN Types



JOIN Type	Description
CROSS JOIN	Returns the Cartesian product of two tables — every combination of rows. No <code>ON</code> condition. Rarely used directly; equivalent to implicit join without a <code>WHERE</code> clause.
SELF JOIN	Joins a table to itself. Useful for hierarchical data (e.g. an employee table with a <code>manager_id</code> column referencing the same table).
NATURAL JOIN	Automatically joins on all columns that share the same name and data type in both tables. No <code>ON</code> clause is written — the database infers the join condition.

```

-- CROSS JOIN example
SELECT a.colour, b.size FROM Colours a CROSS JOIN Sizes b;

-- SELF JOIN example (employees and their managers)
SELECT e.name AS employee, m.name AS manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.employee_id;

-- NATURAL JOIN example
SELECT * FROM orders NATURAL JOIN customers;

```

⚠ NATURAL JOIN is fragile

`NATURAL JOIN` infers its join condition from column names at query time. If either table later gains a new column that happens to share a name with a column in the other table, the join condition **silently changes** — the query continues to run but returns different (wrong) results with no error or warning. For this reason, explicit `JOIN ... ON` clauses are strongly preferred in any production or long-lived code.

6.5. Subqueries

A **subquery** (or nested query) is a `SELECT` statement embedded inside another SQL statement. Subqueries can appear in the `WHERE`, `FROM`, or `SELECT` clauses.

```
-- Find customers who have placed at least one order over €1000
SELECT customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE total > 1000
);
```

Subqueries can also use `ANY` or `ALL`:

```
-- Products more expensive than ALL products in category 5
SELECT name FROM Products
WHERE price > ALL (SELECT price FROM Products WHERE category_id = 5);
```

Warning

When possible, rewrite correlated subqueries as explicit `JOIN`s — a well-optimised `JOIN` is typically faster because the planner can choose the join algorithm. Subqueries that are executed once per row of the outer query (“correlated subqueries”) can be very slow on large tables.

6.5.1. Data Control (DCL)

The DCL (*Data Control Language*) manages access to databases, user permissions and data security.

```
-- Create a user
CREATE USER 'new_user'@'localhost'
IDENTIFIED BY 'new_password';

-- Grant privileges
GRANT SELECT, UPDATE ON Orders.Name TO 'new_user'@'localhost';

-- Revoke privileges
REVOKE UPDATE ON Orders.Name FROM 'new_user'@'localhost';
```

6.6. Exercises

6.6.1. Conceptual Questions

Exercise 6.1 (*Schema vs Instance*)

Explain the difference between the database schema and a database instance. Give an example to illustrate the distinction.

Solution

The database schema is how the data is organized, it affects the structure and the dataset and can constrain the values of the data is stored. An instance is the data itself that follows the schema. We could see it as a party with a dress code, the organizers set the dress code and ensures everyone follows (the schema), each person in the party follows those rules (each person is an instance) and if they don't follow it they can not attend.

Exercise 6.2 (*File System vs Database*)

Compare file system-based data management with database management systems. What are the main advantages of using a DBMS over raw file storage?

Solution

In file systems you can not impose rules to the data, they can store more data but there is no control about what is being stored, in DBMS you can store less data, but you can control what is stored and you can access the data directly by applying filters. The main advantage is that you have the fixed schema that ensures the data has the proper format and you can search and make operations directly in the data.

Exercise 6.3 (*Primary Key*)

What is a primary key and why is it important? Can a table have more than one primary key? What properties must a primary key satisfy?

Solution

A primary key is an attribute (or set of attributes) that uniquely identifies each row of a table. A table has **exactly one** primary key. It can be **composite** (made of several columns), but there is only one PK per table. Other column-sets that could also serve as a unique identifier are called **candidate keys**; the DBA picks one and promotes it to PK.

Properties a PK must satisfy: **uniqueness**, **non-null**, **minimality** (no proper subset is itself unique) and **stability** (the value should not change over the row's lifetime).

Exercise 6.4 (*Foreign Key*)

What is a foreign key? Provide a concrete example showing a foreign key relationship between two tables.

Solution

A foreign key is a key that is related to another table and can be used to link tables, the key that relates to needs to be a primary key of the other table. For example in a table named Cities we have information of each city in a country and there is a Country_Code key and in another table named Country we have the information at the Country level, and there is a Code key. In this case Country_code is a foreign key in the Cities table so now both are linked and we can get the information of each city in a country.

Exercise 6.5 (DDL and DML)

What is the meaning of DDL (Data Definition Language) and DML (Data Manipulation Language) in SQL? Provide at least one example statement for each category.

Solution

- **DDL** defines or modifies the *schema* — the structure of the database. Examples: CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX.
- **DML** manipulates the *data* inside existing tables. Examples: INSERT INTO, UPDATE, DELETE, and (in some texts) SELECT — though SELECT is sometimes split off into a separate **DQL** (Data Query Language) category.

Exercise 6.6 (ALTER vs UPDATE)

Briefly explain the difference between ALTER and UPDATE in SQL. When would you use each?

Solution

The ALTER statements is used when you want to alter the tables schema, the UPDATE statement is used when you want to update the records in the table.

Exercise 6.7 (DELETE vs DROP)

Briefly explain the difference between the DELETE and DROP statements in SQL. What are the consequences of each?

Solution

The DELETE statements is used to delete specific records and the DROP statements is used to delete the table from the schema, so the entire atble is erased from the DBMS and all that dta is lost. A DELETE statement with a weak WHERE condition can have the same effect but it wont lose the schema of the table

Exercise 6.8 (CRUD Operations)

What are the CRUD operations and their corresponding SQL commands? Provide a simple example for each command.

Solution

CRUD = **Create, Read, Update, Delete** — the four basic operations on persistent data.

```
-- Create
INSERT INTO Students (FirstName, LastName, Email)
VALUES ('Ada', 'Lovelace', 'ada@example.com');

-- Read
SELECT * FROM Students WHERE LastName = 'Lovelace';

-- Update
UPDATE Students SET Email = 'ada.l@example.com' WHERE BadgeID = 1;

-- Delete
DELETE FROM Students WHERE BadgeID = 1;
```

Exercise 6.9 (Database Normalisation)

Why is it important to avoid storing all records in a single table in relational databases, and why should databases be normalised across multiple tables? What problems does normalisation solve?

Solution

A single fat table duplicates information and creates three classes of *anomalies*:

- **Update anomaly.** Changing one fact (e.g. a course's name) requires updating every row that mentions it; missing one row leaves the data inconsistent.
- **Insertion anomaly.** Cannot insert a course until at least one student is enrolled (or vice versa) without nullable filler columns.
- **Deletion anomaly.** Removing the last student of a course also erases the course itself.

Normalisation (1NF → 2NF → 3NF → BCNF) splits the data into linked tables so each fact lives in exactly one place, joined back at query time via foreign keys.

6.6.2. ACID Questions**Exercise 6.10 (Atomicity)**

What is Atomicity in the ACID model? Provide an example of a database transaction where atomicity is critical.

Solution

Atomicity means a transaction is all-or-nothing: either every statement succeeds and the result is committed, or none of them take effect (rollback).

Bank transfer example. Moving \$100 from account A to account B requires two updates: $A -= 100$ and $B += 100$. If the system crashes between them and atomicity is not enforced, money disappears (debit applied, credit lost) or is created (credit applied, debit lost). Atomicity guarantees the pair either both happen or both do not.

Exercise 6.11 (*Isolation*)

What does “I” (Isolation) refer to in the ACID database transaction model? Describe a scenario where insufficient isolation causes a data integrity problem.

Solution

Isolation means concurrent transactions do not see each other’s intermediate state — each runs as if it were alone on the database. Without it, anomalies appear: dirty reads, lost updates, non-repeatable reads, phantoms.

Lost-update example. Two cashiers process refunds against an account with balance 100.

1. Cashier 1 reads balance = 100.
2. Cashier 2 reads balance = 100.
3. Cashier 1 writes $100 - 30 = 70$.
4. Cashier 2 writes $100 - 20 = 80$.

Both refunds happened but only one was deducted. Proper isolation (serializable or row-level locking) forces the second read to wait, giving the correct final balance 50.

Exercise 6.12 (*Durability vs BASE*)

What is the meaning of Durability in the ACID model? How does ACID Durability compare to the approach taken by BASE models?

Solution

Durability means that once a transaction has been acknowledged as committed, the result survives any subsequent crash — power loss, OS panic, hardware failure. It is enforced via write-ahead logs (WAL) and `fsync` of the log to non-volatile storage before the commit returns.

BASE (**B**asically **A**vailable, **S**oft state, **E**ventual consistency) is the NoSQL counterpart that explicitly relaxes ACID for horizontal scale: writes may be acknowledged after being stored on only a subset of replicas, soft state may evolve in the background, and the system promises only that replicas will eventually converge. Durability is weakened in exchange for higher availability and throughput.

6.6.3. Schema Design Exercises

Exercise 6.13 (*University Schema: Students and Courses*)

The University is preparing a database to store all Students and Courses.

- Each Course can be attended by multiple Students.
- Each Student can be enrolled in multiple Courses.

Students table:

```
BadgeID (INT, AUTO_INCREMENT, PK),
FirstName (VARCHAR(30), NOT NULL),
LastName (VARCHAR(30), NOT NULL),
Email (VARCHAR(30), NOT NULL).
```

Courses table:

```
CourseID (CHAR(10), UNIQUE NOT NULL, PK),
CourseName (VARCHAR(255), NOT NULL),
Degree (VARCHAR(30), NOT NULL),
B_or_M (CHAR(1),
CHECK (B_or_M IN ('B','M'))).
```

Given this schema, what modifications are necessary (new tables, constraints, etc.) to relate the Students and Courses tables? If creating a new table, identify its Primary and Foreign Keys.

Solution

A *many-to-many* relationship requires a **junction (associative) table** linking the two existing primary keys. (*The schema below is illustrative — not a real University database.*)

```
CREATE TABLE Enrollments (
    BadgeID INT NOT NULL,
    CourseID CHAR(10) NOT NULL,
    Year INT NOT NULL,
    Grade DECIMAL(3,1),
    -- Composite primary key: a student can take the same course in
    -- different academic years, so the year is part of the PK.
    PRIMARY KEY (BadgeID, CourseID, Year),
    FOREIGN KEY (BadgeID) REFERENCES Students(BadgeID)
        ON DELETE CASCADE,
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
        ON DELETE CASCADE
);
```

- **Primary key:** composite (BadgeID, CourseID, Year).
- **Foreign keys:** BadgeID → Students(BadgeID) and CourseID → Courses(CourseID).
- Optional payload columns on the link (e.g. Grade, Year) capture facts that belong to the *pair*, not to a student or a course alone.

Exercise 6.14 (*Library Schema: Books and Authors*)

A library is preparing a relational database to manage all Books and Authors.

- Each Author can write multiple Books.
- Each Book can have multiple Authors.

What modifications are necessary to relate the Books and Authors tables? If creating a new table, what are the Primary and Foreign Keys of that table?

Solution

Same many-to-many pattern as the University case — add a junction table. (*Hypothetical schema, not a real library database.*)

```
CREATE TABLE Authors (
    AuthorID INT AUTO_INCREMENT PRIMARY KEY,
    Name     VARCHAR(100) NOT NULL
);

CREATE TABLE Books (
    ISBN     CHAR(13) PRIMARY KEY,
    Title    VARCHAR(255) NOT NULL,
    PubYear  INT
);

CREATE TABLE BookAuthors (
    ISBN     CHAR(13) NOT NULL,
    AuthorID INT NOT NULL,
    PRIMARY KEY (ISBN, AuthorID),
    FOREIGN KEY (ISBN) REFERENCES Books(ISBN)
        ON DELETE CASCADE,
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
        ON DELETE CASCADE
);
```

- **PK** of BookAuthors: composite (ISBN, AuthorID).
- **FKs:** ISBN → Books(ISBN), AuthorID → Authors(AuthorID).

Exercise 6.15 (*Twitter: User and Follows*)

A Twitter user can follow multiple people. How must the **User** table relate to the **Follows** table? Write the SQL statement to alter the **Follows** table to define the attributes that constitute the primary and foreign keys.

Solution

Follows is a **self-referencing** many-to-many link on **Users**: both the follower and the followee are users. (*Sample schema, not the real Twitter one.*)

```
CREATE TABLE Users (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Handle VARCHAR(30) UNIQUE NOT NULL
);

-- Assume Follows(FollowerID, FolloweeID) already exists.
ALTER TABLE Follows
    ADD CONSTRAINT pk_follows
        PRIMARY KEY (FollowerID, FolloweeID),
    ADD CONSTRAINT fk_follower
        FOREIGN KEY (FollowerID) REFERENCES Users(UserID)
        ON DELETE CASCADE,
    ADD CONSTRAINT fk_followee
        FOREIGN KEY (FolloweeID) REFERENCES Users(UserID)
        ON DELETE CASCADE,
    ADD CONSTRAINT chk_no_self_follow
        CHECK (FollowerID <> FolloweeID);
```

- **PK** of **Follows**: composite (**FollowerID**, **FolloweeID**) — each directed follow relationship is unique.
- **FKs**: both columns reference **Users(UserID)**.
- The **CHECK** prevents users from following themselves.

Exercise 6.16 (*Many-to-Many: Actors and Plays*)

How do you implement a many-to-many relationship between two existing SQL tables, **Actors** and **Plays**? Write the SQL statements required to create the necessary structure.

Solution

Create a junction table whose primary key is the composite of the two foreign keys. (*Illustrative schema, not a real one.*)

```

-- Existing tables (sketched here for context):
CREATE TABLE Actors (
    ActorID INT AUTO_INCREMENT PRIMARY KEY,
    Name    VARCHAR(100) NOT NULL
);
CREATE TABLE Plays (
    PlayID  INT AUTO_INCREMENT PRIMARY KEY,
    Title   VARCHAR(255) NOT NULL,
    Year    INT
);

-- Junction table linking them:
CREATE TABLE ActorPlays (
    ActorID INT NOT NULL,
    PlayID  INT NOT NULL,
    Role    VARCHAR(100),
    PRIMARY KEY (ActorID, PlayID),
    FOREIGN KEY (ActorID) REFERENCES Actors(ActorID)
        ON DELETE CASCADE,
    FOREIGN KEY (PlayID)  REFERENCES Plays(PlayID)
        ON DELETE CASCADE
);

```

The optional `Role` column on the link captures information that belongs to the *pair* (the actor played a specific role in that play), not to the actor or the play individually.

7. NoSQL Databases

The SQL language is perfectly suited to relational databases, which were predominant until the 1990s, when the amount of data generated by the explosion of the internet and web applications became increasingly large. In 1998, computer scientist Carlo Strozzi coined the term *NoSQL* by creating a database management system that did not require the use of SQL, promoting the idea of open source databases that departed from the traditional relational model.

7.1. Database Partitioning

A first important obstacle in managing a relational database arises with the growth of the quantity of records, which forces the **partitioning** of the databases — their subdivision into smaller parts.

Vertical partitioning: all records are saved on each disk so that the attributes are divided between disks. This is not a problem for the relational model, since the various partitions can be seen as different tables with the same primary key.

Horizontal partitioning (also called **sharding**): the records are divided among the different disks so that all attributes associated with the same record are maintained together. This case is not supported by traditional RDBMSs, which require each table to be entirely included in a single memory element.

The idea is to apply a hash function to the primary key, obtaining a number, then perform integer division with the number of available partitions P : the remainder — a number between 0 and $P - 1$ — indicates the disk on which to store the record. Formally, given a key k :

$$\text{partition} = \text{hash}(k) \bmod P$$

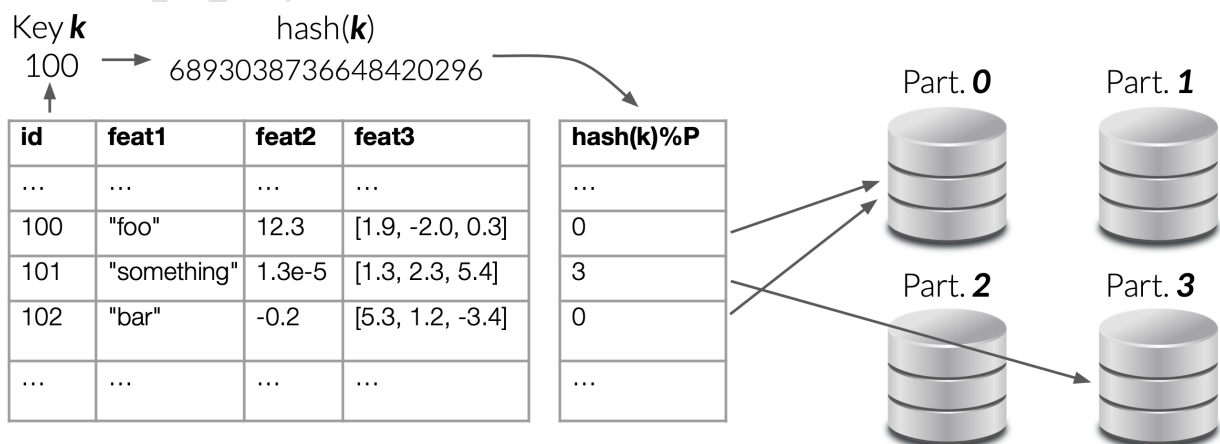


Figure 7.1.: Example of sharding via hashing with integer division and $P = 4$

This method does not respond well in terms of scalability: adding a new disk means going from P to $P + 1$, requiring all integer divisions to be recalculated and all records moved according to the new indexing. Because the modulus changes, nearly **all** K keys must be remapped to different partitions — an $O(K)$ data movement that is prohibitively expensive in large clusters.

7.1.1. Consistent Hashing

The practical solution is **consistent hashing**: instead of using modulo arithmetic, both keys and nodes are mapped to positions on a **circular hash ring** — a continuous space of values from 0 to $2^N - 1$ (for SHA-256, $N = 256$, giving 2^{256} possible positions).

Each node (partition/disk) is assigned one or more positions on the ring. To look up where a key k belongs, compute $\text{hash}(k)$ and walk **clockwise** around the ring; the first node encountered is responsible for that key.

A “sector” of the ring is associated with each disk, and each digest in the ring is “rounded” to the largest value within the sector where the association with the partition takes place.

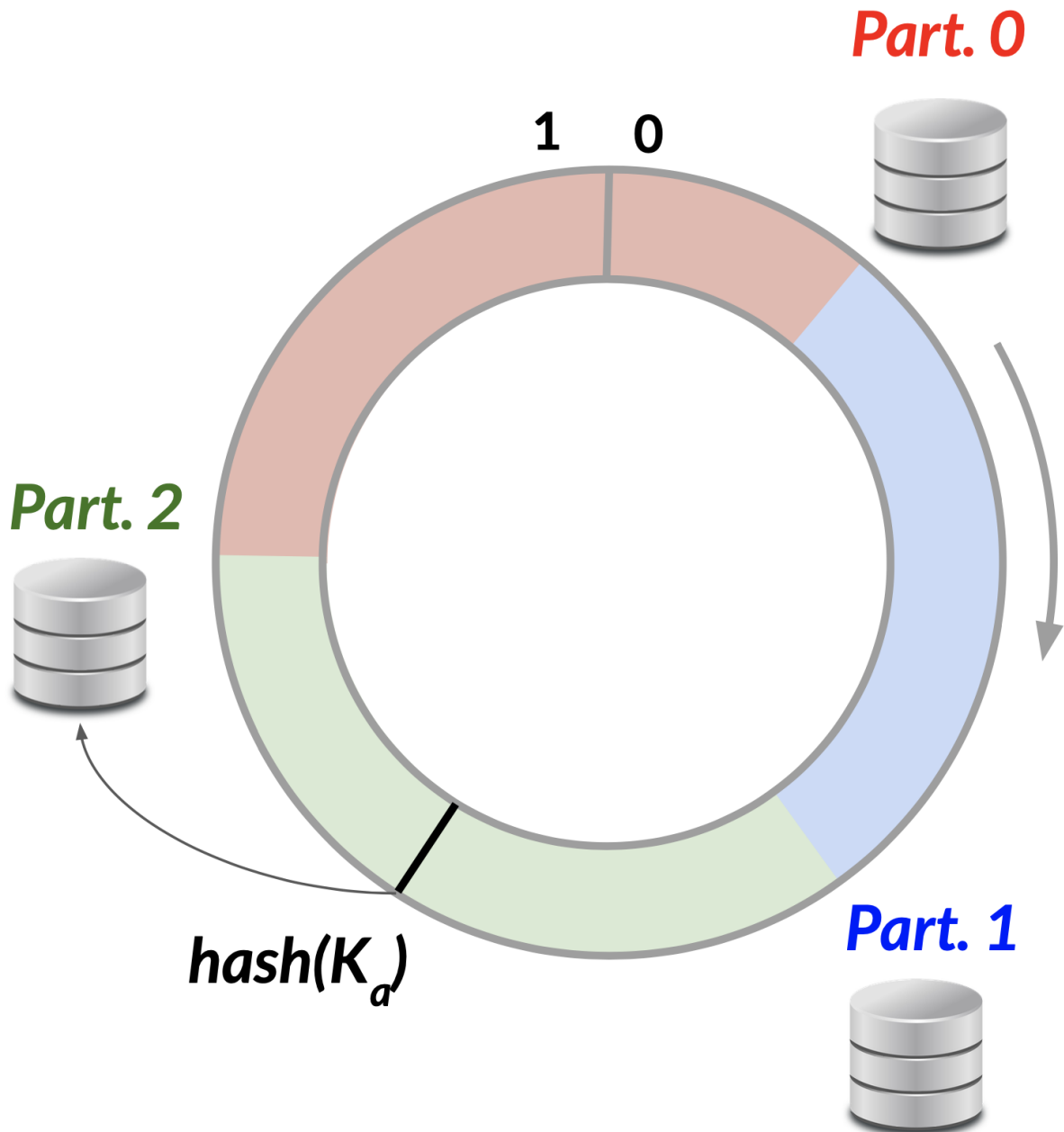


Figure 7.2.: Example of consistent hashing with three disks

When introducing a new disk, it occupies part of a sector previously occupied by another disk. Only the records in that sector need to be transferred — not the entire database. Concretely, only the keys that fall between the new node and its predecessor on the ring need to move — roughly $O(K/N)$ remappings instead of the $O(K)$ required by naive modular hashing.

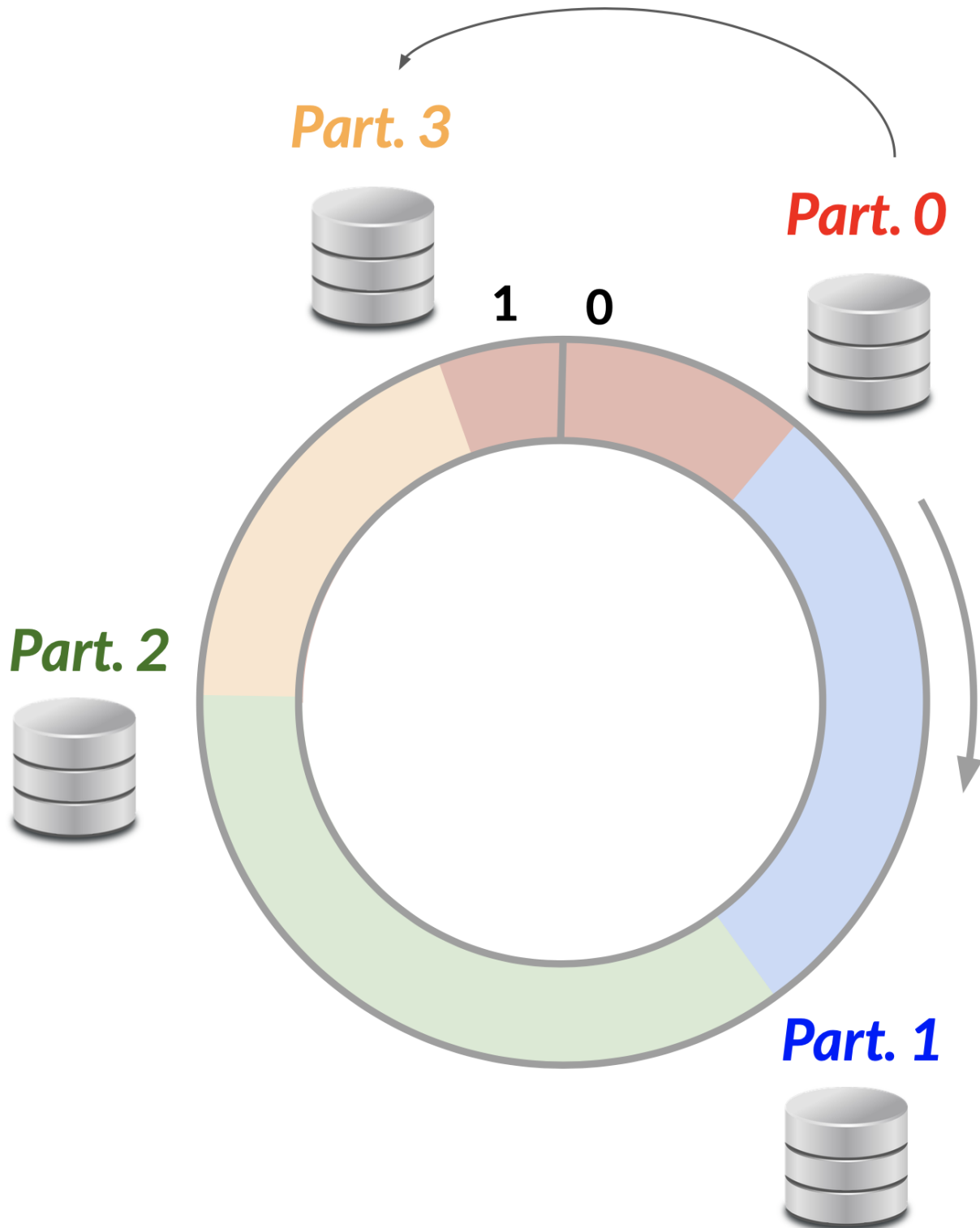


Figure 7.3.: Introduction of a new partition with consequent transfer of records

Virtual nodes: to distribute load more uniformly, a single physical node is mapped to *multiple* positions on the ring (virtual nodes). This smooths out hash-space imbalances and makes rebalancing even more granular when nodes join or leave.

Consistent hashing also lends itself well to **data replication**: the user chooses a replication factor

R ; a record stored at its primary node is additionally copied to the next $R - 1$ nodes clockwise on the ring, ensuring fault tolerance without a separate replication mechanism.

7.2. Characteristics of NoSQL Databases

The four main points that distinguish NoSQL databases from traditional relational ones:

1. **Flexible models:** support for the storage and management of semi-structured and unstructured data, departing from the rigid structured nature of relational databases.
2. **Horizontal scalability:** distribution of databases on multiple nodes of a cluster without significant degradation of performance.
3. **Simplified queries:** languages that do not use complex operations such as JOIN, favoring faster response times especially in distributed environments.
4. **Models with less consistency:** adherence to ACID properties is reduced in favor of the scalability and *availability* necessary in distributed systems.

7.2.1. BASE vs ACID

The transaction model followed by most NoSQL databases is **BASE** — **B**asically **A**vailable, **S**oft state, **E**ventually consistent (a deliberate contrast with ACID as in chemistry):

- **Basically Available:** the individual nodes in a network are generally accessible *almost* always, unlike the ACID model where access could be interrupted due to *isolation* during a read or write operation.
- **Soft state:** the state of a database is not rigidly *persistent* — it may change without a specific intervention by the user, as in the case of data replication across cluster nodes.
- **Eventually consistent:** *consistency* is guaranteed only after a certain time, not necessarily at the exact moment a transaction is concluded.

Where ACID prioritises correctness and strict consistency (at the cost of availability under network partitions), BASE trades strict consistency for higher availability and partition tolerance — occupying the *AP* corner of the CAP theorem. This makes BASE-compliant systems well suited to large-scale distributed deployments where brief inconsistency windows are acceptable.

Note

The last point does not imply that all NoSQL databases are *AP*. Both availability *A* and consistency *C* are guaranteed along a “gradient” — from more stringent conditions (*high availability*) to more labile conditions (*eventual availability*). This flexibility allows the same DBMS to accommodate different services and use cases.

C in CAP $\neq$ C in ACID

The letter **C** appears in both acronyms but means something completely different:

- **C in ACID** (*Consistency*): every transaction leaves the database in a **valid state**

— all integrity constraints (foreign keys, uniqueness, check constraints) are preserved before and after the transaction.

- **C in CAP** (*Consistency*): every **read** receives the **most recent write** or an error. All nodes in the distributed system see the same data at the same time — this is the property of *linearisability* (or *serializability* in a broader sense).

A system can satisfy ACID consistency while violating CAP consistency (e.g. a single-node RDBMS that cannot guarantee up-to-date reads in a network-partitioned cluster), and vice versa.

7.3. NoSQL Database Categories

NoSQL databases are not a monolithic family — they cover several distinct data models:

Category	Core abstraction	Typical strengths
Key-Value Store	Opaque value per key	Ultra-fast reads/writes, easy sharding, in-memory caching
Document Store	Self-describing JSON/BSON document	Flexible schema, rich queries on nested fields
Wide-Column Store	Sparse row with variable columns per row	High-throughput writes, efficient column-family scans
Graph Database	Nodes and labelled edges with properties	Relationship traversal, multi-hop queries
Time Series Database	Value indexed by timestamp	Append-heavy workloads, time-window aggregations; optimised for metrics, sensor readings, and financial ticks. Examples: InfluxDB, Prometheus
Full-Text Search Engine	Inverted index over text documents	Ranked keyword/phrase search over unstructured text. Examples: Elasticsearch, Solr
Vector Database	High-dimensional embedding vectors	Approximate nearest-neighbour (similarity) search

7.4. Examples of NoSQL Databases

7.4.1. Key-Value Databases

The simplest way to store data: a key is associated with each record, similarly to Python dictionaries. Each key can be associated with a different type of data (a number, a string, a list, etc.). This

type of storage is *schema-less*, not requiring metadata to illustrate the structure of the data.

The absence of a schema allows for easy and rapid distribution among the nodes of a network. The key-value model is particularly suitable for datasets requiring a large number of reads and writes in rapid succession, so data are often saved directly in RAM for significantly higher performance. Databases that manage data located in main memory are called **in-memory databases**.

Redis — common commands:

```
# Store a value (string)
SET user:1001 "Alice"

# Retrieve a value
GET user:1001

# Delete a key
DEL user:1001

# Set a key with an expiry time (TTL in seconds)
SET session:abc123 "token_data" EX 3600

# Work with a list
LPUSH myqueue "job1"
RPOP myqueue

# Work with a hash (like a mini-document)
HSET user:1001 name "Alice" age 30
HGET user:1001 name
```

Redis supports five core data types:

Type	Description / typical use case
String	Binary-safe byte sequence; counters, cached HTML fragments, session tokens
Hash	Map of field→value pairs inside one key; user profiles, configuration objects
List	Ordered sequence with O(1) push/pop at both ends; task queues, activity feeds
Set	Unordered collection of unique strings; tag systems, unique visitor tracking
Sorted Set	Set where every member has a numeric score; leaderboards, rate-limiting windows

These types make Redis suitable for caching, session management, rate limiting, leaderboards, and pub/sub messaging.

Notable implementations: Redis, Amazon DynamoDB, Memcached, Apache Ignite.

7.4.2. Document Databases

Instead of saving key-value pairs, document databases save *documents* composed of field-value pairs. These are essentially information encoded in JSON format that are perfectly suited to semi-structured data. The various documents can also be organized into “collections” through a field — usually called *id* — that uniquely identifies the document, just like primary keys in relational databases. Queries are performed using the fields of the documents.

Notable implementations: MongoDB, CouchDB, Amazon DocumentDB, Couchbase.

7.4.3. Time Series Databases

They are designed to store data indexed by timestamps and to group them according to time windows. They are suitable for empirical data produced by sensors and provide functionality to calculate metrics and statistics that help identify trends and patterns.

Notable implementations: InfluxDB, Prometheus, TimescaleDB, OpenTSDB.

7.4.4. FullText Search Engine

Suitable for unstructured data collected as textual documents. The queries — similarly to what search engines like Google do — return a “ranking” of documents on the basis of the matches with the keywords specified by the user.

Notable implementations: Elasticsearch, Apache Solr, OpenSearch, Meilisearch.

7.5. Wide-Column Databases

Wide-column databases (also called column-family stores) organise data as rows with a row key, but unlike relational tables, **each row can have a different set of columns**. Columns are grouped into “column families” and sparse rows are stored efficiently without null padding. This means that different rows genuinely store different columns — absent columns consume no space at all — which is the key advantage over a fixed-schema RDBMS, where every row must carry every column (or an explicit NULL) regardless of whether the value exists.

Row Key	Name	Age	Email	Temperature	Pressure
sensor_001				23.5	1013.2
sensor_002				24.1	
user_001	Alice	30	alice@example.com		

The `sensor_*` rows store only measurement columns; the `user_*` rows store only profile columns. Both coexist in the same table without wasted storage.

Strengths:

- Excellent for **time-series and event data** where columns vary per record
- High write throughput across distributed nodes
- Efficient for queries that scan a column family rather than full rows

Typical use cases: IoT sensor data, financial ticks, clickstream logs, audit trails.

Notable implementations: Apache Cassandra, Apache HBase, Google Bigtable.

7.6. Graph Databases

Graph databases model data as a network of **nodes** (entities) and **edges** (relationships), each of which can carry arbitrary properties. This structure is natural for highly interconnected data where the relationships themselves are first-class objects.

```
(Alice:Person {age:30}) -[:KNOWS]-> (Bob:Person {age:25})
(Bob:Person)           -[:WORKS_AT]-> (Acme:Company {since:2020})
(Alice:Person)         -[:ACTED_IN]-> (Matrix:Movie {year:1999})
```

When to prefer a graph DB over a relational DB:

Relational databases can model graph-like data via self-joins and junction tables, but performance degrades with query depth (e.g., “friends of friends of friends”). Graph databases store adjacency natively and traverse relationships in $O(1)$ per hop.

Applications: social networks, knowledge graphs, recommendation engines, fraud detection, molecular biology (protein interaction networks), epidemiology (contact tracing).

Query language: Cypher (neo4j), SPARQL (RDF graphs), Gremlin (Apache TinkerPop).

Notable implementations: neo4j, Dgraph, Amazon Neptune, ArangoDB (multi-model).

7.7. Vector Databases

Vector databases store **high-dimensional embedding vectors** and are optimised for **similarity search** — finding the k nearest vectors to a query vector, typically using cosine similarity or Euclidean distance.

Motivation: machine learning models (language models, image encoders) represent objects as dense vectors in a high-dimensional space (e.g. 768 or 1536 dimensions). Traditional RDBMS and even standard NoSQL databases do not support efficient approximate nearest-neighbour (ANN) search.

Typical workflow:

1. Encode documents, images, or records as embedding vectors using a neural model.
2. Store vectors in the vector database with their metadata.
3. At query time, encode the query and retrieve the top- k most similar vectors.

Applications: semantic search, Retrieval-Augmented Generation (RAG) for LLMs, recommendation systems, image similarity, anomaly detection.

Notable implementations: Pinecone, Weaviate, ChromaDB, Milvus, pgvector (PostgreSQL extension).

7.8. Exercises

Exercise 7.1 (*Non-relational DBMS Examples*)

List at least four examples of non-relational (NoSQL) database management systems. For each, identify the NoSQL category it belongs to (Key-Value, Document, Wide-Column, Graph, etc.).

Solution

- **Key-Value** — Redis, DynamoDB, Riak. Fast lookups by opaque key; values are blobs.
- **Document** — MongoDB, CouchDB, Elasticsearch. Self-describing JSON / BSON records, queryable by field.
- **Wide-Column** — Cassandra, HBase, ScyllaDB. Sparse rows with very many columns, column families grouped on disk.
- **Graph** — Neo4j, Amazon Neptune, ArangoDB. First-class nodes + edges with traversal queries (Cypher, Gremlin).

Some systems (Elasticsearch, Solr) are sometimes split off as **full-text search engines**, and time-series stores (InfluxDB, TimescaleDB) as a category of their own.

Exercise 7.2 (*ACID vs BASE*)

Describe the differences between the ACID and BASE consistency models. In what scenarios is each model preferable?

Solution

- **ACID** = **A**tomicity, **C**onsistency, **I**solation, **D**urability. Strict, transactional guarantees. Every committed write is immediately visible to all subsequent reads and survives crashes. Cost: latency and limited write throughput under contention.
- **BASE** = **B**asically **A**vailable, **S**oft state, **E**ventually consistent. Relaxed guarantees in exchange for availability and horizontal scale. Reads may return stale data; replicas converge in the background.

ACID is preferable when correctness is non-negotiable: banking and accounting, inventory, order management, anything with money or audit constraints. **BASE** wins in high-throughput, eventually consistent workloads: social media feeds, CDNs, IoT telemetry, large-scale caches and analytics ingestion.

Exercise 7.3 (*Horizontal vs Vertical Scaling*)

What are horizontal and vertical scaling (partitioning)? Discuss their pros and cons, and argue which one is generally preferable for large-scale data systems and why.

Solution

- **Vertical scaling (scale-up).** Bigger single machine: more CPU, RAM, faster disks. Pros: simple — software is unchanged, no distributed-systems complexity, no consistency trade-offs. Cons: hard ceiling on hardware, exponential price per unit performance at the top end, single point of failure.
- **Horizontal scaling (scale-out).** More machines, data sharded and replicated across them. Pros: near-unlimited growth, commodity hardware, built-in fault tolerance via replication. Cons: needs partitioning, replication, distributed transactions or eventual consistency; operational complexity is much higher.

For **large-scale data systems**, horizontal scaling is generally preferred. Vertical scaling cannot keep up with terabyte- or petabyte-scale workloads at any reasonable cost, and a single super-server is a single point of failure. NoSQL and modern distributed RDBMS were designed precisely to make horizontal scaling tractable.

Exercise 7.4 (*Consistent Hashing for Sharding*)

What are the advantages of using consistent hashing for sharding distributed databases compared to a “standard” partitioning technique (e.g. via modulo-operations)? What problem does consistent hashing solve when nodes are added or removed?

Solution

With **plain hash-mod sharding** ($\text{shard} = \text{hash}(\text{key}) \bmod N$), changing the number of nodes from N to $N \pm 1$ changes the modulus and therefore **reshuffles almost every key** — i.e. roughly $\mathcal{O}(K)$ key movements for K keys. That stalls the cluster during any resize.

Consistent hashing places nodes and keys on a virtual ring; each key is owned by the next node clockwise. Adding or removing a node only changes ownership of the keys in its **immediate neighbourhood** on the ring, so on average only $\mathcal{O}(K/N)$ keys move. Combined with **virtual nodes** (each physical node owns many ring positions), the load stays balanced. The problem it solves: **graceful, low-impact rebalancing** when nodes join or leave a dynamic cluster — exactly the day-to-day reality of large NoSQL and CDN deployments (DynamoDB, Cassandra, Riak, memcached).

Part II.

Processing

8. Processing and Concurrency

8.1. Introduction to Data Processing

Processing refers broadly to the set of operations performed on a dataset to extract higher-level information. In a very simplified view:

$$\text{DATA(SET)} \xrightarrow{\text{PROCESSING}} \text{RESULTS}$$

This encompasses a wide variety of tasks such as analytics, data reduction, data cleansing, and feature enrichment. In practice, processing often involves multiple sequential stages, each consuming the output of the previous one:

$$\text{DATA} \xrightarrow{\text{PROCESSING}} \text{DATA} \xrightarrow{\text{PROCESSING}} \text{DATA} \xrightarrow{\text{PROCESSING}} \text{RESULTS}$$

The fundamental question that runs through all of these stages is: **how does the processing scale with the data size, task complexity, and available resources?**

8.2. CPU Processing

At its core, processing is the sequential execution of instructions by the CPU, operating on data held in memory. The CPU follows the classic **fetch–decode–execute** cycle for each instruction:

- **Fetch** — retrieve the instruction from memory
- **Decode** — interpret the instruction
- **Execute** — carry out the operation (Von Neumann architecture)

8.2.1. Processing Time

The **CPU processing time** is the time needed to execute the entire instruction set. Two complementary metrics matter:

- **Wall-time** (response time / elapsed time): the total latency to complete a task, including all I/O operations (memory and storage access), network communication, and OS overhead.
- **CPU time**: the time attributed solely to instruction processing by the CPU, excluding time spent waiting for other activities.

CPU time can be modelled as:

$$t_{\text{CPU}} = \sum_i \frac{IC_i \cdot CPI_i}{CF}$$

where:

Symbol	Meaning
IC_i	Instruction count — number of instructions in the program
CPI_i	Clock cycles per instruction
CF	Clock frequency (Hz)

8.2.2. Historical Context: Moore's Law

In 1965, Gordon Moore observed that the number of transistors on a chip doubled approximately every 18–24 months — a trend that held for nearly five decades and became known as **Moore's Law**.

Era	Key trend
1970s–1990s	Transistor count and clock frequency both doubled regularly
Early 2000s	Clock frequency plateau: thermal limits prevented further frequency scaling
2004 onwards	Industry shifted to multi-core CPUs rather than faster single cores
Today	Transistor scaling continues (though slowing) but single-thread performance improvements are incremental

The plateau in single-thread performance — roughly 3–4 GHz since the mid-2000s — is why parallel and distributed computing became the dominant paradigm for improving application throughput.

8.2.3. The Power Wall

A straightforward way to increase processing performance is to raise the clock frequency. This strategy drove chip performance for many years, alongside transistor scaling.

However, from the early 2000s onwards, processors started hitting a fundamental limit: **power dissipation**. The power consumed by a chip scales as:

$$P \propto n \cdot C \cdot V^2 \cdot f$$

where n is the number of transistors, C is transistor capacitance, V is the supply voltage, and f is the clock frequency. Early 32-bit processors such as the Intel 80386 consumed around 2 W; a 2021 4.0 GHz Intel Xeon Gold 5318Y consumes more than 150 W. Continuing to scale frequency without also reducing voltage became thermally and energetically unsustainable.

8.2.4. Pipelining and Alternative Strategies

As performance gains from raw clock-frequency scaling became harder to sustain, chip designers turned to **instruction pipelining**, which allows multiple instructions to overlap in their fetch–decode–execute stages on the same ALU. This achieves a speedup without raising the clock frequency.

A broader change of paradigm was therefore required to continue improving performance.

8.3. Multi-Processing Architectures

The industry response to the power wall was **multi-core** CPUs: placing multiple independent processing units (cores) on the same die, and further combining multiple CPUs (sockets) on the same motherboard.

This can be realised in several ways:

- **Symmetric Multiprocessing (SMP)** — homogeneous CPUs, all with equal access to shared memory.
- **Asymmetric Multiprocessing** — heterogeneous CPUs with different roles (e.g. big.LITTLE mobile architectures).
- **Master/Slave** — one master CPU assigns tasks to subordinate CPUs.

A modern laptop (e.g. a Lenovo X1 Gen 8 with an Intel Core i7-10510U) might show the following `lscpu` output:

```
Architecture:      x86_64
CPU(s):            8          ← 1 socket × 4 cores × 2 threads/core = 8 logical PUs
Thread(s) per core: 2        ← Intel Hyper-Threading: 2 independent "feeding lanes" per core
Core(s) per socket: 4        ← 4 physical processing units (cores) per CPU
Socket(s):         1          ← 1 CPU socket on the motherboard
Model name:        Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz
CPU max MHz:       4900.0000
L1d cache:         128 KiB
L2 cache:          1 MiB
L3 cache:          8 MiB
```

Hyper-Threading

Intel's Hyper-Threading technology provides two independent **fetch/decode** lanes per physical core, but only one execution unit (ALU). The OS sees 8 logical CPUs on a 4-core processor, but at most 4 instructions can execute truly in parallel at any moment.

The total number of logical CPUs visible to the OS is:

$$\text{CPUs} = \text{Sockets} \times \text{Cores per Socket} \times \text{Threads per Core}$$

A production-grade server (e.g. a Dell PowerEdge R750 with two Intel Xeon Gold 5318Y) might expose 96 logical CPUs to the OS (2 sockets $\times$ 24 cores $\times$ 2 threads). Multi-processor machines are now the standard — the question is how to take advantage of them.

8.4. Parallelism

Parallelism means exploiting multi-processing architectures to solve problems faster. Two orthogonal forms of parallelism exist:

1. **Data-Level Parallelism (DLP)** — many data items can be operated on simultaneously. The same operation is applied to all elements in parallel.
2. **Task-Level Parallelism (TLP)** — multiple sub-tasks of a process operate independently and in parallel.

In practice, the most efficient solutions combine both.

Parallel processing is the simultaneous execution of a single job split into n sub-tasks across m CPUs/cores. The primary goal is to reduce the total execution time (**speedup**).

8.5. Flynn's Taxonomy

Flynn's taxonomy (1966) classifies computer architectures by the number of simultaneous instruction streams and data streams:

	Single Data	Multiple Data
Single Instruction	SISD	SIMD
Multiple Instruction	MISD	MIMD

- **SISD** — one instruction on one data stream at a time. No parallelism; e.g. a single-threaded process on a single-core CPU.
- **SIMD** — one instruction performed on multiple data streams simultaneously. Enables data-level parallelism. Found in modern CPU AVX/SSE/MMX instructions and most GPUs.
- **MISD** — multiple instructions on a single data stream. Rare in practice; used in fault-tolerant applications.
- **MIMD** — multiple instructions on multiple data streams simultaneously. Supports both DLP and TLP; each processing unit runs independently. Most modern parallel computers fall into this category.

i SPMD

A widely used sub-category of MIMD is **SPMD (Single Program Multiple Data)**: the same entire program (not just single instructions) runs on multiple “chunks” of data across several processing units. This is the natural model for **embarrassingly parallel** problems.

8.6. Speedup: Amdahl's Law

Is it possible to speed up a computation indefinitely by adding more processing units?

Given a process with a fraction p that can be parallelised and a fraction $(1 - p)$ that must remain serial, and with n processing units, the speedup predicted by **Amdahl's law** is:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

As $n \rightarrow \infty$:

$$S_{\max} = \frac{1}{1 - p}$$

No matter how many processing units are available, the **serial fraction** of the computation remains the bottleneck. To maximise performance, the process itself must be redesigned to reduce its serial portion.

Amdahl's law is often considered somewhat pessimistic, as it assumes the serial fraction grows proportionally with problem size.

8.7. Gustafson's Law

Gustafson's **scaled speedup** addresses parallelism from a different angle. It observes that in practice the “problem size” often scales with the number of available processors: as more processors are added, a larger problem is tackled, so the parallelisable component grows more than the serial component.

$$S_{\text{scaled}}(n) = 1 + (n - 1) \cdot p$$

Either way, **heterogeneous computing** — integrating CPUs, GPUs, and specialised accelerators — aims to maximise both the serial and parallel components of a computation.

8.8. Processes and Threads

8.8.1. Definitions

i Process

A **process** is an instance of a program in execution. Every process is an independent entity with its own executable code and memory space. Multiple instances of the same program can run simultaneously — they share the same code but are distinct processes.

i Thread

A **thread** is the basic unit of CPU utilisation. Every thread consists of a program counter, a set of registers, and its own execution stack. A thread is part of a process, and each process can contain one or more threads.

Within a multi-threaded process:

- All threads **share** the process memory and common resources (“global” for all threads).
- Each thread has an **independent** set of registers, memory, and execution state.

A word processor, for example, might use one thread to check spelling, another to process keystrokes, a third to load images, and a fourth to perform periodic automatic saves.

8.8.2. Concurrency vs Parallelism

These two concepts are often confused:

- **Concurrency** — multiple tasks are *in progress* at the same time. Tasks can start, run, and stop independently and in any order. On a single-core CPU, concurrency is achieved through rapid context switching.
- **Parallelism** — multiple tasks are *executing* at the same time. True simultaneous execution, requiring multiple processing units.

8.8.3. Multi-Tasking

Multi-tasking is the OS-enabled execution of multiple concurrent tasks. The OS instructs the CPU to share its time between tasks via a **scheduler**:

- **Pre-emptive scheduling** — the scheduler alternates between N tasks, sharing CPU time approximately equally (with priority adjustments).
- **Collaborative scheduling** — the system switches processes only when the active one voluntarily releases control.

A **context switch** saves the state of the currently running process or thread so it can be resumed later.

8.8.4. Multi-Threading

Multi-threading is the execution of multiple threads within a process. The execution mode depends on the architecture:

- **Concurrent**, on single-core CPUs with a multi-tasking OS (the CPU alternates between threads via context switching).
- **Parallel**, on multi-core CPUs (different threads run simultaneously on different cores).

Context switching between threads is lighter than between full processes, since threads within the same process share memory. Multiple sub-components of a single application can thus proceed independently while sharing global resources.

8.9. Race Conditions

Multi-threaded execution can lead to subtle bugs when multiple threads access the same shared memory simultaneously.

Race Condition

A **race condition** occurs when a shared resource (e.g. a variable in memory) is accessed concurrently by multiple threads, and the outcome depends on the particular order in which the accesses take place.

Consider `var_x = 5` shared between two threads:

- Thread 1 reads `var_x` (value 5) and computes $5 * 2 = 10$.
- Thread 2 **also** reads `var_x` (still 5, before Thread 1 has written back) and computes $5 * 3 = 15$.
- Thread 1 writes 10 back to `var_x`.
- Thread 2 writes 15 back to `var_x`.
- Final value: 15 — but the correct sequential result (multiply by 2 then by 3) should be 30.

8.10. Thread Synchronisation

Thread synchronisation mechanisms prevent race conditions by controlling access to shared resources:

- **Lock / Mutex (Mutual Exclusion)** — only one thread at a time can hold the lock and access the protected resource. Other threads block until the lock is released.
- **Binary semaphore** — similar to a mutex, with a 0/1 counter.
- **Counting semaphore** — controls access to a pool of resources.

Deadlock

A **deadlock** occurs when a thread waits indefinitely for an event that can only be triggered by another thread that is itself blocked. Locks must always be released after use, and circular wait conditions must be avoided.

8.11. The Python GIL

The **Global Interpreter Lock (GIL)** is a mutex on the CPython interpreter itself, ensuring that only one thread at a time can hold the GIL and thus execute Python bytecode. This design:

- Prevents many deadlocks and race conditions at the interpreter level.
- **Disables true parallelism** for CPU-bound work in CPython: even on a multi-core machine, two Python threads from the same process cannot execute Python bytecode simultaneously.

The GIL is an implementation detail of CPython (the standard Python interpreter), not a requirement of the Python language. Alternative interpreters such as Jython and IronPython do not rely on it.

! Python 3.13 and the GIL

Starting with **Python 3.13** (October 2024), the GIL became optional. CPython now provides a “free-threaded” build mode in which the GIL is disabled, enabling full multi-threading for CPU-bound Python code. The standard build still includes the GIL for backwards compatibility.

! GIL Does Not Eliminate All Race Conditions

Even with the GIL, race conditions can occur during OS-level context switches. The GIL is released periodically — by default every 5 ms (`sys.getswitchinterval()`) — and briefly between the old thread releasing it and the new thread acquiring it. During that window, the OS scheduler may preempt execution mid-operation.

Compound operations that *appear* atomic in Python source code (e.g. `x += 1`, which compiles to LOAD/ADD/STORE bytecode instructions) are **not guaranteed to be atomic** under concurrent thread execution. A lock (`threading.Lock`) must still be used to protect shared state.

For a detailed analysis of the GIL’s interaction with the OS scheduler, see: <https://www.dabeaz.com/python/GIL.pdf>

8.12. Threading vs Multiprocessing in Python

	Threading (<code>threading</code>)	Multiprocessing (<code>multiprocessing</code>)
Memory	Shared across threads	Separate address space per process
GIL	Constrained by GIL (one thread executes at a time)	No GIL constraint (each process has its own interpreter)
Best for	I/O-bound tasks (disk, network)	CPU-bound tasks (numerical computation)
Communication	Shared variables (but need locks)	Queues, pipes, shared memory objects

 Tip

Other libraries extend these capabilities: **joblib** provides a unified interface to both threading and multiprocessing backends.

```
# joblib example: run square() in parallel across 4 processes
from joblib import Parallel, delayed

def square(x):
    return x ** 2

results = Parallel(n_jobs=4, backend='multiprocessing')(
    delayed(square)(i) for i in range(10)
)
# results = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

mpi4py exposes the MPI (Message Passing Interface) standard to Python, enabling distributed-memory parallelism across cluster nodes — the same model used in large-scale HPC applications.

8.13. Hands-on

The scripts in `code/parallel` walk through everything in this chapter: one thread, many threads, race conditions and locks, thread pools and queues, processes and process pools, and finally I/O-bound vs CPU-bound benchmarks. See the guide [Lab: Threads and processes in Python](#).

9. Distributed Processing and MapReduce

9.1. Scaling Processing

As discussed in the previous chapter, processing performance can scale with the number of processing units, provided that the computation is at least partially parallelisable. Speedup studies typically confirm that wall-time scales favourably with n for heavily data-parallel tasks (SPMD-style).

The natural next step is to tackle very large, possibly distributed, datasets by increasing n beyond what a single machine can offer. Two fundamental strategies exist:

Strategy	Description
Vertical scaling (Scale Up)	Increase the capacity or performance of existing hardware — buy a faster machine.
Horizontal scaling (Scale Out)	Add more machines — buy more commodity hardware.

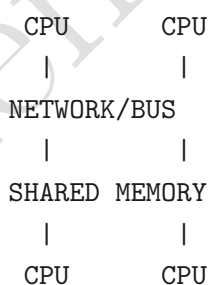
Horizontal scaling is almost always preferred for large-scale data processing: commodity hardware is cost-effective, and the system can grow incrementally.

9.2. Parallel Computing Architectures

Different memory-sharing architectures lead to different trade-offs in performance and scalability.

9.2.1. Shared Memory Multiprocessor (SMP)

In a **Shared Memory Multiprocessor**, a set of CPUs share a common memory space connected by a high-speed bus or fabric.



- All modern workstations (including smartphones) are SMP systems.
- Memory access is uniform and transparent to the programmer.

- Scaling this architecture to a very large number of processors is expensive and technically demanding.

i NUMA: Non-Uniform Memory Access

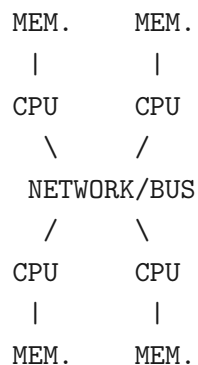
Multi-socket servers partition memory into **NUMA nodes** — one per CPU socket, each with local RAM. The `lscpu` output for a dual-socket server shows **NUMA node(s): 2**. Accessing local memory (same NUMA node as the running core) is significantly faster than accessing remote memory (on the other socket). The OS and job schedulers can pin processes to specific NUMA nodes with `numactl`:

```
numactl --cpunodebind=0 --membind=0 python3 my_script.py
```

NUMA topology is a key consideration when scaling shared-memory parallelism beyond a single socket.

9.2.2. Distributed Memory Multiprocessor (Cluster)

In a **cluster**, each processor has its own local memory — a complete unit called a **node**. All nodes are connected via a high-speed network.



- Good overall performance even from commodity hardware.
- The complexity lies in the communication across nodes.
- Scales horizontally to thousands or millions of nodes.

9.3. Distributed Computing: Clusters

A cluster distributes the processing workload across multiple networked nodes, exposing itself to users as a single virtual machine.

Typical cluster components:

- **Head node** — the access point for users; distributes workload to compute nodes.
- **Computing nodes (CN)** — where the actual processing runs; usually invisible to users.
- **Interactive nodes** — accessible to users as standard computers for testing, storage access, or launching jobs.

- **Specialised nodes** — nodes equipped with specific hardware (GPUs, large RAM, fast NVMe) for particular workloads.

9.3.1. Supercomputers as Clusters

Most of today's supercomputers are clusters:

CINECA LEONARDO (world's #10 in November 2025) - 1 536 nodes with 2× Intel Sapphire Rapids (56-core CPUs) - 3 456 nodes with 1× Intel Xeon 8358 (32-core CPU) and 4× NVIDIA Ampere GPUs

On the opposite extreme, clusters of Raspberry Pi boards or repurposed desktop PCs (**COTS** — **Commodity Off-The-Shelf** hardware) can efficiently tackle a wide range of parallel problems through **opportunistic computing** (using shared resources when they are idle).

9.4. Batch Systems

Users typically submit jobs to a cluster through a **batch system** rather than logging in interactively:

1. A job consists of an executable, optional arguments, and resource requirements (CPUs, memory, walltime, etc.).
2. The batch scheduler accepts the job and places it in a queue.
3. The scheduler selects which job runs next based on priority, previous resource usage, and availability.
4. The job's tasks are dispatched to computing nodes for execution.

Popular batch schedulers include **PBS**, **SLURM**, **LSF**, and **HTCondor**. Example HTCondor submission:

```
$ condor_submit my_job.sub
Submitting job(s).
50 job(s) submitted to cluster 5146936.

$ condor_q
OWNER      BATCH_NAME  SUBMITTED  DONE  RUN  IDLE  TOTAL  JOB_IDS
jpazzini   ID:5146936  10/4 17:29   12   25   13     50   5146936.0-49
```

An HTCondor submit file:

```
executable      = my_exec
arguments       = --input input_file_$(ProcId).txt
transfer_input_files = input_file_$(ProcId).txt
log             = my_job_$(ProcId).log
error           = my_job_$(ProcId).err
output          = my_job_$(ProcId).out
```

```
request_disk          = 8GB
request_memory        = 2GB
queue 50
```

9.5. Programming Frameworks for Parallel-Distributed Systems

Optimising complex tasks for parallel and distributed execution requires knowledge of specific programming standards:

Framework	Purpose
OpenMP	Parallelisation directives for shared-memory multi-core CPUs on a single node
MPI (Message Passing Interface)	Communication and synchronisation across nodes of a cluster
CUDA / HIP	Vector/matrix processing on NVIDIA and AMD GPUs

In this course we focus on higher-level, Python-accessible frameworks that abstract away the inner workings of distributed communication — in particular, the **MapReduce** paradigm and its implementations in Hadoop and Spark.

i GPU Computing: Going Further

This course focuses on CPU-based parallelism and distributed processing. GPU programming (CUDA, OpenCL, HIP, Numba, CuPy) and heterogeneous CPU+GPU architectures are covered in the optional companion course “**Modern Computing for Physics**”, which addresses SIMD/vectorisation, GPU kernel programming, and MPI+GPU hybrid models.

9.6. Big Data and the 3 Vs

From the early 2000s, a wave of data-processing innovations emerged, driven largely by technology companies that needed to handle datasets at an unprecedented scale. These are often summarised as **Big Data**, characterised by the three Vs:

- **Volume** — datasets at the scale of terabytes, petabytes, or beyond.
- **Velocity** — data arriving at high speed (streaming sensors, transactions, logs).
- **Variety** — structured, semi-structured, and unstructured data.

The five key ingredients of a BigData processing stack are:

1. **APIs / programming language** — the interface developers use.
2. **Software framework** for distributed processing.

3. **Cluster orchestrator / scheduler** — resource management.
4. **Distributed file system (DFS)** — storage layer.
5. **Computing nodes** — physical or cloud hardware.

9.7. Hadoop

Hadoop was born in 2006 within Yahoo! and was based on a 2003 Google paper on the Google Distributed File System. It rapidly became the de facto standard for BigData storage and processing.

Large-scale real-world deployments have included:

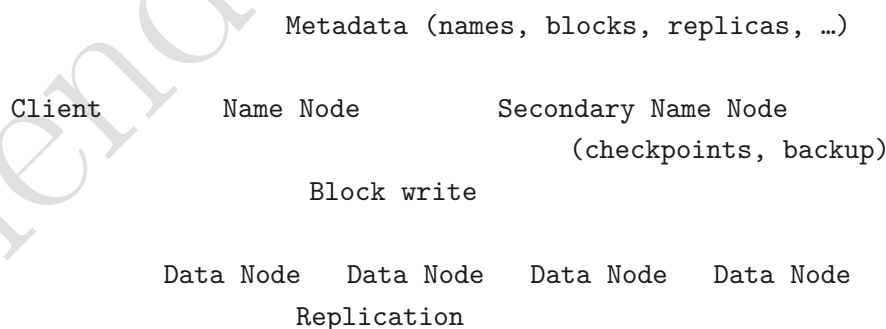
- **Facebook:** 1 100-node cluster (8 800 cores, ~12 PB storage); 300-node cluster (~3 PB)
- **Twitter:** clusters storing over 500 PB, largest with >10 000 nodes
- **Yahoo!:** >100 000 CPUs across >40 000 computers

Hadoop is open source (Apache Foundation), written in Java, and designed to scale horizontally. Its main components are:

Component	Role
Hadoop Common	Java-based shared libraries and utilities
HDFS	Hadoop Distributed File System — storage
YARN	Yet Another Resource Negotiator — cluster manager and scheduler
Hadoop MapReduce	Distributed programming model

9.8. HDFS: A Reminder

HDFS stores data as large blocks distributed across data nodes, with the name node keeping metadata:



- The **Name Node** manages the filesystem namespace and metadata.
- The **Secondary Name Node** periodically checkpoints the metadata.
- **Data Nodes** store the actual data blocks and replicate them for fault tolerance.

9.9. The MapReduce Paradigm

MapReduce is not a programming language, but a **paradigm** for developing distributed algorithms on potentially very large datasets. It decomposes computation into two phases:

9.9.1. 1. Map

Split the input data into **<key, value>** pairs, and apply a user-defined map function to each pair, producing a list of intermediate **<key, value>** pairs:

$$\text{map}(k_{\text{in}}, v_{\text{in}}) \rightarrow \text{list}[(k_1, v_1), (k_2, v_2), \dots]$$

Each key-value pair is processed **independently**, enabling full data parallelism across the cluster.

9.9.2. 2. Reduce

Collect all intermediate pairs with the same key, then apply a user-defined aggregation function to produce a final result:

$$\text{reduce}(\text{list}[(k, v_1), (k, v_2), \dots]) \rightarrow [(k, v_{\text{out}})]$$

9.9.3. Word Count Example

The canonical example: count the occurrences of each word in a text corpus.

Input: "Car Car River Deer Car Bear"

Map phase:

("Car", 1), ("Car", 1), ("River", 1), ("Deer", 1), ("Car", 1), ("Bear", 1)

Shuffle & Sort (grouping by key):

"Bear" → [1]

"Car" → [1, 1, 1]

"Deer" → [1]

"River" → [1]

Reduce phase (sum):

Bear → 1

Car → 3

Deer → 1

River → 1

9.10. MapReduce in Hadoop

9.10.1. Map Phase

Data is read from HDFS, partitioned among computing nodes, and sent to each node as a set of key-value pairs. The Map tasks process input records **independently** and write intermediate results to the **local disk** of the node. This **data locality** principle — bringing computation to the data rather than the data to the computation — is central to Hadoop’s performance.

9.10.2. Reduce Phase

Once all Map tasks complete, a **shuffle and sort** step groups and transfers the intermediate key-value pairs to the Reduce tasks. The Reduce tasks then apply the aggregation function to each group.

9.11. YARN: Yet Another Resource Negotiator

YARN acts as the “operating system” of the Hadoop cluster, managing the scheduling and execution of jobs.

Key components:

Component	Role
Resource Manager (RM)	Manages cluster-wide resource allocation; contains the Scheduler and the Application Manager
Application Master (AM)	Negotiates resources from the RM and coordinates the execution of a single application
Node Manager (NM)	Manages the user processes on each individual machine
Container	A set of allocated resources (memory, CPU) used to run a specific task

9.12. Hadoop Higher-Level Abstractions

Writing raw MapReduce jobs in Java is verbose. Higher-level tools provide more accessible interfaces:

Tool	Description
Pig	High-level scripting language translated into MapReduce programs (Yahoo!, latest release 2017)

Tool	Description
Hive	SQL-like query language (HiveQL) for data stored in HDFS (Facebook/Netflix)
Oozie	Workflow scheduler for chaining multiple MapReduce jobs

9.13. The Hadoop Ecosystem

Hadoop grew into a rich ecosystem covering the full BigData stack:

- **APIs / programming language:** Pig, Hive, custom Java/Python
- **Distributed processing framework:** Hadoop MapReduce
- **Cluster orchestrator:** YARN
- **Distributed file system:** HDFS

The broader BigData landscape includes many additional tools (Spark, Kafka, Cassandra, HBase, etc.) that either complement or replace individual components of the Hadoop stack depending on the use case.

The full BigData stack can be visualised as five layers, with Hadoop occupying all four lower layers and higher-level tools sitting on top:

APIs / Languages	Pig · Hive · HiveQL · custom Java/Python
Processing	Hadoop MapReduce · Apache Spark · Storm
Cluster Scheduler	YARN · Mesos · Kubernetes
Distributed FS	HDFS · Amazon S3 · Azure Data Lake
Computing Hardware	Commodity nodes · Cloud VMs · Bare metal

Beyond this core stack, the broader ecosystem includes: **Kafka** (streaming), **HBase** / **Cassandra** (wide-column NoSQL), **ZooKeeper** (distributed coordination), **Sqoop** (RDBMS ↔ HDFS transfer), **Oozie** (workflow scheduling), and **Mahout** (distributed machine learning).

10. Apache Spark

10.1. Motivation

As discussed in the MapReduce chapter, Hadoop provided a solid foundation for distributed processing, but it has notable limitations for certain workloads:

- **Complex/iterative algorithms** (e.g. k-means clustering) require chaining multiple MapReduce jobs, making them awkward to implement and slow due to repeated disk I/O between stages.
- **MapReduce I/O bottleneck**: every intermediate result is written to disk (HDFS), then read back for the next stage.
- **Batch-only**: Hadoop MapReduce processes data “at rest” in large batches; it does not natively support real-time or streaming processing.

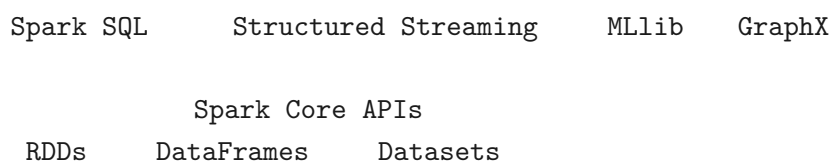
Apache Spark addresses all three limitations:

Hadoop MapReduce	Apache Spark
Strictly Map → Reduce pattern	Arbitrary DAG of transformations — not tied to MapReduce
Disk-based intermediate storage	In-memory processing — massive speedup for iterative algorithms
Batch processing only	Both batch and real-time streaming processing

Spark was originally developed at UC Berkeley AMPLab in 2009 and became an Apache project. It is open source, written in Scala, with official APIs for Python (PySpark), Java, and R.

10.2. Spark Ecosystem

Spark provides a layered set of APIs:



- **RDD (Resilient Distributed Dataset)** — the low-level, unstructured API.

- **DataFrame / Dataset** — the high-level, structured API built on top of RDDs.
- **Spark SQL** — SQL interface for structured data (DataFrames).
- **Structured Streaming** — continuous stream processing via the DataFrame API.
- **MLlib** — distributed machine learning library.
- **GraphX** — graph computation.

i PySpark vs Scala Performance

Spark is written in **Scala** and runs on the JVM. The PySpark API communicates with the JVM via the **Py4J bridge**, which introduces serialisation overhead when data is passed between Python and Java processes. For most DataFrame/SQL operations this overhead is minimal because computation runs on the JVM regardless. However, Python UDFs (User Defined Functions applied row-by-row) bypass the JVM and can be significantly slower than equivalent Scala UDFs.

In this course PySpark is used for accessibility. For production pipelines with strict performance requirements, Scala/Java provides the highest throughput. Vectorised Pandas UDFs (via PyArrow) offer a middle ground.

10.3. Spark Cluster Architecture

Spark uses a **Master/Slave** architecture.

10.3.1. Driver and Executors

When a Spark application is submitted:

1. A **Driver** process is created. The Driver is the application's brain: it converts user code into a plan of smaller execution units (tasks), and distributes and schedules those tasks across the executors.
2. A **Spark Context** is created within the Driver for each application, allowing it to access cluster resources via the Resource Manager.
3. A set of **Executors** are created on the cluster's worker nodes. Each Executor:
 - Executes the tasks assigned by the Driver.
 - Reports the computation state back to the Driver.
 - Stores data in memory or disk across the lifetime of the application.

The Driver and Executors live in Java Virtual Machines (JVMs) within the worker nodes. One Executor cannot span multiple nodes, but one node can host multiple Executors.

10.3.2. Cluster Resource Managers

Spark works with several cluster managers:

- **Spark standalone** cluster manager (built-in)
- **YARN** (Hadoop's resource manager)

- Mesos
- Kubernetes

10.4. RDD: Resilient Distributed Dataset

The **RDD** is Spark's low-level data abstraction: an immutable, partitioned collection of records that can be operated on in parallel.

Key properties:

Property	Description
Distributed	Split into partitions, each hosted on an Executor (data locality)
Resilient	Contains full lineage information to recover from partition loss
Immutable	Transformations produce new RDDs; the original is never modified
In-memory	Spark keeps RDDs in Executor memory across the application lifetime
Unstructured	Data stored as Java/Python objects (no imposed schema)

One Executor processes one partition at a time. More Executors can process more partitions simultaneously — this is the unit of parallelism.

10.5. Transformations and Actions

Operations on RDDs fall into two categories:

10.5.1. Transformations

Operations that take one or more RDDs as input and produce a **new RDD**. Transformations are **lazy** — they are not executed until an Action is triggered.

- **Narrow transformations:** each input partition contributes to exactly one output partition (e.g. `map`, `filter`, `flatMap`). Can be pipelined without data shuffling.
- **Wide transformations:** input partitions contribute to multiple output partitions (e.g. `groupByKey`, `reduceByKey`). Require a **shuffle** — data is repartitioned across the cluster.

10.5.2. Actions

Operations that trigger execution and return a value to the Driver (or write data to storage).

Examples: `count()`, `collect()`, `reduce()`, `take(n)`, `saveAsTextFile()`

10.6. Laziness and the DAG Scheduler

Spark defers execution until an Action is triggered. This **laziness** brings two main benefits:

1. **Minimises driver-executor communication:** transformations are accumulated as a recipe, not executed immediately.
2. **Enables optimisation:** Spark can optimise the entire transformation pipeline before executing it.

Spark represents the full computation plan as a **DAG (Directed Acyclic Graph)**:

- The **DAG Scheduler** groups transformations that do not require data exchange across partitions (narrow transformations) into **stages**.
- Operations within a stage are **pipelined**.
- All tasks are dispatched by the Driver to the Executors as an optimised execution plan.

10.7. Spark DataFrame

The **DataFrame** is the high-level structured API, built on top of RDDs:

- Similar interface to Pandas DataFrames.
- Offers SQL queries via the SparkSQL interface.
- Distributed and fault-tolerant (backed by RDDs).
- Suitable for structured and semi-structured datasets with a defined schema.

A DataFrame always has an underlying RDD. The schema information allows Spark to apply additional query optimisations not available to raw RDDs.

10.8. Real-Time Analytics and Structured Streaming

Processing unbounded data streams in real time is common across many domains:

- Sensor readings from physics detectors
- IoT device readings
- Financial transaction monitoring
- Web application event streams

The challenges are significant: potentially out-of-order data, high throughput, exactly-once processing guarantees, and load balancing.

Two common approaches:

Approach	Characteristics
One-Element-at-a-Time	Fixed pipeline; triggered on each new record. Low latency, limited throughput.

Approach	Characteristics
Micro-batching	Accumulate data over a fixed time window (e.g. 500 ms), then process as a batch. Higher throughput, latency limited by batch interval.

A common real-time streaming pattern is the **ETL pipeline**:

- **Extract** — read raw events from a source (e.g. Kafka topic, sensor feed)
- **Transform** — apply business logic: filtering, aggregation, enrichment, join with static data
- **Load** — write processed results to a sink (e.g. database, dashboard, another Kafka topic)

Spark Structured Streaming expresses the full ETL pipeline as a single streaming query, making the code nearly identical to a batch DataFrame transformation.

10.8.1. Spark Structured Streaming

Spark Structured Streaming is the modern streaming API (based on DataFrames, replacing the deprecated Spark Streaming/DStream API (not fully supported since Spark 3.5)).

The key idea: treat the continuous input stream as an **unbounded table** to which new rows are continually appended. Queries over this table are expressed identically to queries over static DataFrames.

Three output modes:

- **Complete**: the entire output table is written to the sink on each trigger.
- **Append**: only rows added since the last trigger are written.
- **Update**: only rows updated since the last trigger are written.

Input sources include Kafka, Amazon S3, TCP sockets, and others. Output sinks include databases, files, Kafka topics, and the console.

10.8.2. Sources and Sinks

Structured Streaming uses a unified **Source** / **Sink** abstraction:

Role	Description	Examples
Source	The origin of the unbounded data stream	Kafka topic, HDFS/S3 file directory, TCP socket, Rate (synthetic test source)
Sink	The destination where processed results are written	Kafka topic, HDFS/S3 file, JDBC database, Console (debugging), Memory (testing)

A streaming query is expressed as: *Source* $\rightarrow$ *Transformations* $\rightarrow$ *Sink*.

10.9. Hands-on

The Spark labs run on a small cluster simulated with Docker (`code/spark`). Each has its own guide:

- **Spark 1: RDDs and map-reduce:** partitions, transformations vs actions, word count, , distributed gradient descent, caching
- **Spark 2: DataFrames and the dimuon spectrum:** schemas, filters, flattening, UDFs vs native functions, a full physics analysis
- **Spark 3: Structured Streaming:** sources, sinks, output modes, a streaming fraud detector

11. Dask

11.1. Introduction

Dask is a flexible parallel computing library for Python, designed to scale Python's scientific ecosystem (NumPy, Pandas, Scikit-learn) to larger-than-memory datasets and multi-machine clusters.

Compared to Spark:

	Apache Spark	Dask
Language	Scala (Python via PySpark API)	Pure Python
Data structures	RDD, DataFrame, Dataset	Bag, Array, DataFrame
Scheduling	JVM-based DAG scheduler	Python-based task scheduler
Ecosystem	Standalone (integrates with Hadoop/YARN)	Native Python — integrates with NumPy/Pandas/Scikit-learn
Best for	Large JVM-native clusters, Java/Scala users	Python-centric workflows, data scientists

Dask's scheduler is written in Python. While this provides tight integration with the Python ecosystem, it also means that Python memory management limitations apply over the lifecycle of long-running applications.

Dask provides three high-level data structures:

- **dask.bag** — a distributed list of arbitrary Python objects, analogous to a Spark RDD.
- **dask.array** — distributed NumPy arrays.
- **dask.dataframe** — distributed Pandas DataFrames.

All three APIs support **lazy evaluation** and leverage Dask's DAG-based scheduler for optimisation and parallel execution.

11.2. Dask Fundamentals

11.2.1. Lazy Evaluation and Task Graphs

Like Spark, Dask uses **lazy evaluation**: operations are not executed immediately. Instead, each operation builds a **task graph** — a DAG (Directed Acyclic Graph) representing the full

11. Dask

computation. Execution is triggered by calling `.compute()`.

```
import dask

@dask.delayed
def add(x, y):
    return x + y

@dask.delayed
def square(x):
    return x ** 2

# Build the task graph (nothing executes yet)
a = add(1, 2)
b = square(a)

# Visualise the graph
b.visualize() # renders the DAG as an image

# Execute
result = b.compute() # triggers actual computation → 9
```

`dask.delayed` is the fundamental building block: wrap any Python function with `@dask.delayed` and it becomes a lazy node in the task graph.

11.2.2. Partitioning

All Dask collections (Bag, Array, DataFrame) split data into **partitions** that are processed in parallel. Choosing the right partition count is important:

Too few partitions	Too many partitions
Insufficient parallelism — workers sit idle	Scheduler overhead dominates — dispatching thousands of tiny tasks is expensive
Large memory footprint per worker	Excessive inter-partition communication

Rule of thumb: aim for 10–100 MB per partition, with the number of partitions roughly $2\text{--}5 \times$ the number of workers. Use `df.repartition(n)` to adjust.

⚠ Python Memory Management in Dask

Because Dask is written in pure Python, it is subject to Python’s memory management. Over the lifecycle of a long-running application, **unmanaged memory** (memory held by Python objects not yet garbage-collected) can accumulate on workers, eventually causing out-of-memory errors even when the logical data fits within the cluster’s total RAM. This is a

key difference from Spark, which runs on the JVM with a more predictable memory model. For production workloads, monitor worker memory via the Dask dashboard and consider restarting workers periodically.

11.3. Cluster Setup

A Dask cluster can be used locally (leveraging multi-core CPUs) or distributed across multiple machines. The `Client` object connects to a running Dask scheduler:

```
from dask.distributed import Client
client = Client('dask-scheduler:8786')
```

A web dashboard at `localhost:8787` provides live visualisation of workers and task graphs.

11.4. Hands-on

The Dask labs (`code/dask`) have their own guides:

- **Dask 1: delayed and futures:** lazy task graphs vs eager futures, and when parallelism does not pay
- **Dask 2: Bags:** semi-structured data, `groupby` vs `foldby`, from bag to `DataFrame`
- **Dask 3: DataFrames:** partitioned pandas, type inference, cheap vs expensive operations, the dimuon analysis again



12. Apache Kafka

12.1. Introduction to Message Streaming

Modern applications increasingly rely on the ability to ingest, route, and process continuous data streams in real time — sensor networks, financial transactions, application event logs, IoT devices, and more. A dedicated **message broker** is often placed at the centre of such architectures.

Apache Kafka is a distributed event streaming platform originally developed at LinkedIn (2011) and later open-sourced through the Apache Software Foundation. It provides:

- **High-throughput, low-latency** publish/subscribe messaging.
- **Durable, fault-tolerant** storage of streams as ordered logs.
- **Scalable** partitioned architecture that distributes load across many brokers.

12.1.1. Core Concepts

Concept	Description
Topic	A named channel to which producers publish and from which consumers read. Topics are partitioned across broker nodes.
Partition	An ordered, immutable log of records within a topic. New messages are always appended.
Offset	The position of a message within a partition. Consumers track their reading offset.
Producer	An application that publishes messages to Kafka topics.
Consumer	An application that subscribes to topics and reads messages.
Consumer Group	A set of consumers that cooperate to read from a topic; each partition is assigned to exactly one consumer in the group.
Broker	A Kafka server that stores and serves partitions. A cluster has multiple brokers for fault tolerance.

12.2. Kafka Architecture

12.2.1. Brokers and Clusters

A Kafka cluster consists of multiple **brokers** — server processes that store and serve partitions. Topics are spread across brokers; each partition lives on exactly one broker at a time (its **leader**), with additional **follower** copies on other brokers for fault tolerance.

Topic "events" - 3 partitions, replication factor 2:

Broker 1	Broker 2	Broker 3
Partition 0 (L)	Partition 0 (F)	
	Partition 1 (L)	Partition 1 (F)
Partition 2 (F)		Partition 2 (L)
L = Leader (handles all reads and writes for that partition)		
F = Follower (replicates the leader; takes over if leader fails)		

The **replication factor** (number of copies of each partition) is set per topic. A replication factor of 3 tolerates 2 broker failures without data loss.

12.2.2. Consumer Offset Management

Each consumer tracks its position in a partition using an **offset** — the sequence number of the last message processed. Offsets are committed back to Kafka so that a consumer can resume after a restart.

The `auto_offset_reset` parameter controls where a consumer starts when no committed offset exists:

Value	Behaviour
'earliest'	Start from the very beginning of the partition log
'latest'	Start from the next new message (default)
'none'	Raise an error if no committed offset is found

12.3. Delivery Semantics

Kafka provides three delivery guarantees, depending on how producers and consumers are configured:

Semantic	Description	Configuration
At-most-once	Messages may be lost; never duplicated	Producer: <code>acks=0</code> ; Consumer: auto-commit before processing
At-least-once	Messages are never lost; may be delivered more than once	Producer: <code>acks=all</code> + retries; Consumer: commit after processing
Exactly-once	Messages processed exactly once — no loss, no duplication	Transactional producer + idempotent consumer; requires Kafka 0.11

Most applications use **at-least-once** semantics and make their processing logic **idempotent** (safe to re-execute with the same message). True exactly-once requires transactional APIs and is more complex to implement.

i Note

Delivery semantics are a key concern when Kafka feeds a Spark Structured Streaming pipeline (as shown in the `KafkaSpark_StructuredStreaming` notebook). Spark's checkpointing mechanism can achieve end-to-end exactly-once semantics when combined with an idempotent sink and Kafka's transactional API.

12.3.1. Pub/Sub Decoupling

A key architectural feature of Kafka is the **complete decoupling** of producers and consumers. A producer does not need to know which consumers exist; a consumer does not need to know which producers are publishing. Messages are written to the topic log and remain available for consumption according to retention policy (time- or size-based), not according to whether a consumer has read them.

This enables flexible, scalable architectures: new consumers can read from the beginning of a topic log without affecting producers or other consumers.

12.3.2. Partitions and Keys

When a producer sends a message without a key, Kafka distributes messages across partitions using a round-robin (or load-based) strategy. When a message includes a **key**, Kafka uses a hash of the key to consistently route all messages with the same key to the same partition — guaranteeing ordering within that group.

Partitioning provides both **scalability** (different partitions can be processed by different consumers in parallel) and **ordering guarantees** (within a partition).

12.4. Kafka and Spark Structured Streaming

Kafka can act as both the **Source** and the **Sink** for a Spark Structured Streaming pipeline. This connects the Source/Sink abstraction introduced in Apache Spark with Kafka's pub/sub model:

- **Source:** Spark reads a stream of JSON messages from a Kafka topic using `.readStream.format("kafka")`
- **Transform:** Spark parses, filters, and aggregates the stream using DataFrame operations
- **Sink:** results are written back to a Kafka topic or to the console using `.writeStream`

Achieving end-to-end **exactly-once** semantics requires: (1) Kafka's transactional producer, (2) Spark checkpointing, and (3) an idempotent sink.

12.5. Hands-on

The Kafka lab (`code/kafka`) covers topics, producers and consumers from the shell and from Python, partitions and consumer groups, and the Kafka → Spark → Kafka pipeline described above. See the guide [Lab: Kafka, and Kafka + Spark](#).

Part III.

Labs

13. Running the labs

The hands-on part of the course lives in the course repository, not in these notes. These pages are a **guide** to it: they follow each notebook in order, explain what every step is doing and why, and point out what to look at while it runs. Keep the notebook open next to the guide.

13.1. Where the code is

The labs come from the professor's repository [jpazzini/MAPD-B](#); my fork [PhyAMR/MAPD-B-code](#) (branch 2026) is cloned next to these notes:

```
2sem/MAPDB/
  notes/      ← these notes
  code/       ← the labs (clone of PhyAMR/MAPD-B-code)
    mysql/    docker-compose.yml, docker/, dbs/, notebooks/
    parallel/  plain Python scripts
    spark/     docker-compose.yml, datasets/, notebooks/
    dask/       docker-compose.yml, notebooks/ (datasets inside)
    kafka/     docker-compose.yml, notebooks/
```

Every notebook exists twice: `LectureN.ipynb` (with blanks to fill in) and `LectureN_solutions.ipynb`. Work on the first one; the guides follow the solutions.

To pick up new material from the professor:

```
cd code
git fetch upstream && git merge upstream/2026
```

13.2. What a lab looks like

Every lab except `parallel/` is a small **cluster simulated on your laptop** with `docker compose`. Each box in the picture below is a separate container, i.e. a separate “machine” with its own processes and its own network address:

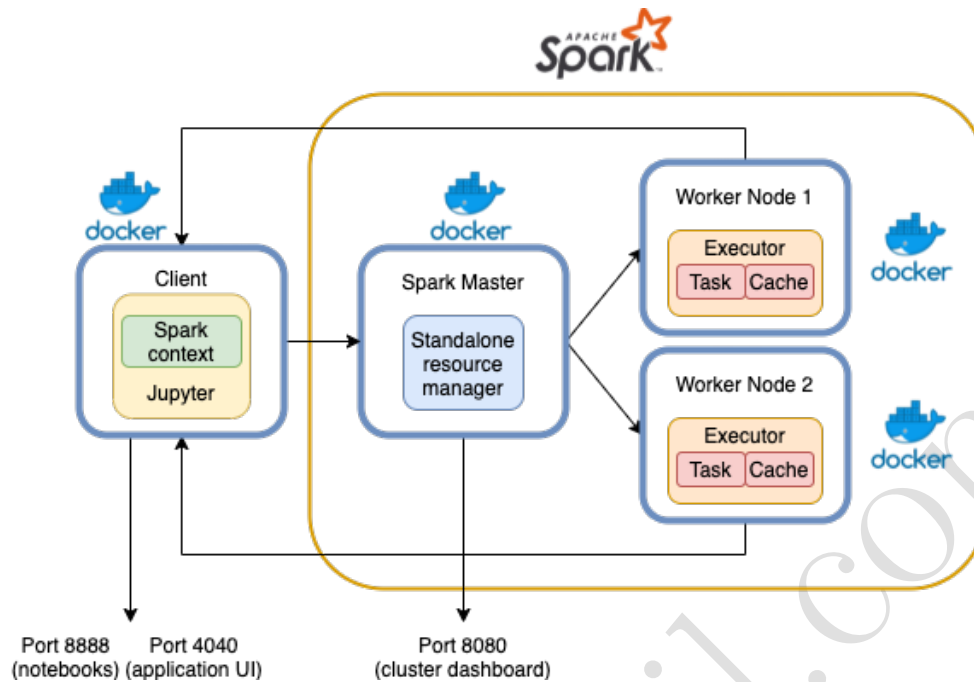


Figure 13.1.: The Spark lab as an example: a master, one or more workers, and a Jupyter client, all inside a private Docker network.

The pattern is always the same:

Role	What it does	Example
Server / head node	the service you are learning about	MySQL server, Spark master, Dask scheduler, Kafka broker
Worker nodes	do the actual processing (distributed labs only)	spark-worker, dask-worker
Client	a Jupyter server where <i>you</i> type code	jupyter-notebook / jupyter

The Jupyter container is technically redundant (you could install the client libraries locally), but it guarantees everybody has the same versions and makes the client/server split explicit: **your notebook is a client talking to a remote cluster over the network**, exactly as it would be on a real one.

13.3. The stacks at a glance

Lab	Folder	Services	Open in browser	Extra preparation
MySQL	code/mysql	db, jupyter	Jupyter localhost:1234	build the mapd_notebook image; download IMDb_pruned.sql
Parallel	code/parallel	none, plain Python	none	Python 3.13 (and optionally 3.9)
Spark	code/spark	spark-master, spark-worker, jupyter- notebook	Jupyter :1234, master UI :8080, app UI :4040	none
Dask	code/dask	dask-scheduler, dask-worker, jupyter- notebook	Jupyter :1234, dashboard :8787	none
Kafka	code/kafka	Spark stack + kafka-broker	Jupyter :1234, :8080, :4040	none

All stacks use port 1234 for Jupyter, so **run one lab at a time**.

13.4. Start, use, stop

```
cd code/spark                                # 1. ALWAYS cd into the lab folder first (see
↳ below)
docker compose up                             # 2. start the cluster (add -d to get the
↳ terminal back)
                                              # 3. open http://localhost:1234 → notebooks/
docker compose down                           # 4. stop and remove the containers when done
```

Useful variations:

```
docker compose up --scale spark-worker=2     # more workers (same for
↳ dask-worker)
docker compose logs -f jupyter-notebook      # follow one service's output
docker compose exec spark-master bash        # open a shell inside a running
↳ service
docker compose ps                             # what is running, and on which
↳ ports
```

`docker compose exec <service> bash` is the easiest way to “log into a node”. The notebooks

13. Running the labs

sometimes say `docker ps` → copy the container ID → `docker exec -it <ID> bash`; the `compose exec` form does the same thing without looking up IDs.

13.5. Three things that confuse everybody

13.5.1. 1. Run `docker compose` from inside the lab folder

Every compose file mounts the current directory into the containers:

```
volumes:
- $PWD:/mapd-workspace
```

`$PWD` is *your shell's* current directory. If you start the stack from somewhere else (e.g. `docker compose -f spark/docker-compose.yml up` from `code/`), the containers see the wrong folder and the notebooks cannot find `datasets/`.

The mount also means the folder is **shared**: what you save in Jupyter appears on your disk, and files you drop into the folder (e.g. a database dump) are visible inside the containers at `/mapd-workspace/...`

13.5.2. 2. Two kinds of addresses: service names and `localhost`

Inside the Docker network, containers find each other **by service name**: the notebook connects to `db`, `spark://spark-master:7077`, `dask-scheduler:8786`, `kafka-broker:9092`. These names mean nothing to your browser.

From your laptop you only reach what the compose file publishes with `ports`: ("`8080:8080`" = host port : container port), always as `localhost:<port>`.

You are...	...and want to reach	Use
code in a notebook	the MySQL server	<code>host="db"</code>
code in a notebook	the Spark master	<code>spark://spark-master:7077</code>
your browser	the Spark master UI	<code>http://localhost:8080</code>
your browser	the Spark application UI	<code>http://localhost:4040</code>

This is why the “Spark UI” link printed by `spark` in the notebook does not work: it points at the container’s internal hostname. Use `localhost:4040` instead.

13.5.3. 3. The notebook is not on the cluster

Variables in the notebook live in the **client** (the Jupyter container). Distributed objects (an RDD, a Spark or Dask DataFrame) live in the **workers**; the client only holds a *handle* to them. Anything that brings data back (`collect()`, `toPandas()`, `compute()`, `gather()`) moves it over the network into the client’s memory, which is the most common way to crash a lab with an out-of-memory error.

13.6. From this laptop cluster to the cloud

The Docker setup is a scale model of how distributed systems are deployed for real. Every piece has a production counterpart, and recognising them is half of “learning to use the cloud”:

In the lab	What it is	On a real cloud / cluster
Docker image (jpazzini/mapd-b:spark-worker)	a frozen, portable filesystem + program	the same images, pulled from a registry (Docker Hub, GitHub/GitLab registry, a private registry)
container	an isolated process with its own network address; much lighter than a VM, shares the host kernel	containers on virtual machines (e.g. CloudVeneto, AWS EC2) or a Kubernetes <i>pod</i>
docker-compose.yml	declares services, networks, volumes on one host	Kubernetes manifests / Helm charts declare the same across many hosts; or one VM per role
service name spark-master	DNS name resolved inside the network	service discovery: Kubernetes Services, internal DNS, fixed private IPs
ports: "8080:8080"	expose one service to the outside	load balancers, ingress, or an SSH tunnel (<code>ssh -L 8080:localhost:8080 vm</code>)
--scale spark-worker=N	horizontal scaling, by hand	autoscaling groups, Kubernetes replicas, Spark dynamic allocation, Dask adaptive scaling
\$PWD mounted as /mapd-workspace	shared local disk	a distributed file system (HDFS) or object storage (S3 / Ceph), read in parallel by every node
SPARK_WORKER_MEMORY=512m	resource limits per node	VM flavour (vCPUs, RAM) and per-executor/per-worker limits

Two ideas from the theory chapters are worth keeping in mind in every lab:

- **Horizontal scaling** (Storage): instead of a bigger machine, add more identical ones. It only pays off if the work can be split into independent pieces, which is why partitions are central in every framework.
- **Move computation to the data** (Distributed processing): networks are slow compared with local disks and memory. Frameworks send *code* to where the data is and bring back only

13. Running the labs

small results. In the labs everything is on one laptop, so you won't *feel* the network, but write the code as if you did.

The managed-service versions of what you run here are, for reference: MySQL → AWS RDS / Google Cloud SQL; Spark → AWS EMR, Google Dataproc, Databricks; Dask → Coiled, Dask on Kubernetes; Kafka → AWS MSK, Confluent Cloud. They remove the operations work, but the programming model, and everything you learn in these labs, is the same.

13.7. Troubleshooting

Symptom	Cause / fix
port is already allocated	another lab (or something else) is using 1234/8080. <code>docker compose down</code> in the other folder, or <code>docker ps</code> to find it
notebook cannot find <code>datasets/...</code>	stack started from the wrong folder (see §1); <code>down</code> , <code>cd</code> , <code>up</code> again
pull access denied for <code>mapd_notebook</code> (MySQL)	the Jupyter image for MySQL is built locally, see MySQL setup
very slow on Apple Silicon	images are <code>linux/amd64</code> and run under emulation; expected
files created from Jupyter are owned by <code>root</code> on Linux	containers run as <code>root</code> ; <code>sudo chown -R \$USER .</code> in the lab folder
a stream / consumer cell never ends	that is by design for streaming code; interrupt the kernel () or wait for the timeout
things behave strangely after many runs	<code>docker compose down</code> and <code>up</code> again for a clean cluster

14. Lab: MySQL

Lab at a glance

- **Notebook:** `code/mysql/notebooks/MySQL.ipynb` (solutions: `MySQL_solutions.ipynb`)
- **Stack:** db (MySQL 9 server) + jupyter (client), see Running the labs
- **Theory:** Databases: relational model, keys, SQL syntax, JOINS
- **You will:** talk to a database server from Python in three ways, build a small schema with keys and constraints, fill it, then answer questions on two real databases (`world`, `IMDb`)

14.1. The idea behind it

A relational DBMS is the classic **client–server** system: one process owns the data on its disk and serialises access to it, and any number of clients send it requests over the network. Two promises make it valuable, both covered in the theory:

- **Structure:** a schema with types, keys and constraints, so the server itself rejects inconsistent data.
- **Transactions (ACID):** a group of changes is applied entirely or not at all, and concurrent clients don't see half-done work. `commit()` in this lab is the visible edge of that machinery: until you commit, your changes are a pending transaction.

The price is that a single server is hard to scale out. It scales *up* (bigger machine), and beyond that you need read replicas or sharding, which give up some of the guarantees. That trade-off is why the later labs (Spark, Dask, Kafka) exist, and why NoSQL databases relax consistency (CAP / BASE).

14.2. Before you start

The Jupyter image for this lab is **built locally**, not pulled. Do it once:

```
cd code/mysql
docker build -t mapd_notebook -f docker/mapd_notebook.Dockerfile .
```

The Dockerfile is just `python:3.14-slim` + `notebook`, `matplotlib`, `pandas`, `SQLAlchemy`, `ipython-sql` and `mysql-connector-python`.

14. Lab: MySQL

For part 06 you also need `IMDb_pruned.sql` (~350 MB, link on Moodle) in `code/mysql/dbs/`. `world.sql` is already there.

Then `docker compose up` from `code/mysql` and open `localhost:1234`.

The compose file defines the credentials you will use everywhere:

User	Password	Notes
root	root_pwd	sees everything, including MySQL's internal databases
my_user	my_pwd	ordinary user, sees only what it has been granted

14.3. 01 — Three ways to talk to the server

The point of this section is that **the database is a server process on another machine** (the db container), and every tool is just a client sending it SQL over the network.

1. Interactive shell, on the server itself.

```
docker compose exec db bash          # "ssh" into the server node
mysql -u root -p                     # password: root_pwd
```

```
SHOW DATABASES;                     -- every statement ends with ;
```

2. `mysql.connector`, a Python driver. You open a *connection*, then a *cursor*; the cursor sends statements and holds their results.

```
import mysql.connector
db = mysql.connector.connect(user="root", password="root_pwd", host="db")
dbc = db.cursor()

dbc.execute("SHOW DATABASES")
dbc.fetchall()          # or fetchone() / fetchmany(n), or iterate: for row
                        ↪ in dbc
```

Note `host="db"`: the service name from the compose file, not `localhost` (see addresses).

Two cursor rules explain most errors in this lab:

- A cursor holds the result of **one** statement. Fetch everything (or call `dbc.reset()`) before executing the next one, otherwise you get *Unread result found*.
- Statements that **change** the database (`CREATE`, `INSERT`, `UPDATE`, `DELETE`, `ALTER`, `DROP`) only take effect after `db.commit()`. Read-only statements (`SELECT`, `SHOW`, `DESCRIBE`, `USE`) don't need it.

Connecting as `my_user` and running the same `SHOW DATABASES` returns a much shorter list: permissions are enforced by the server, per user.

3. Notebook magic (`ipython-sql`). The most convenient for exploration:

```
%load_ext sql
%env DATABASE_URL=mysql+mysqlconnector://root:root_pwd@db
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

Now a cell starting with `%%sql` is plain SQL, and a line `res = %sql SELECT ...` puts the result in a Python variable. That result object converts to whatever you need: `res.DataFrame()`, `res.dict()`, `res.csv()`, and it can plot itself (`res.bar()`, `res.plot()`).

Use magic for exploring, and the connector when SQL has to sit inside Python logic (loops, parameters, functions).

14.4. 02 — Databases and tables

```
CREATE DATABASE IF NOT EXISTS my_db;
USE my_db;                -- every later statement refers to my_db
SHOW TABLES;
CREATE TABLE Users (UserID varchar(30), BadgeNum int, FirstName
    ↪ varchar(255),
                        LastName varchar(255), Age int, OtherAttr float);
DESCRIBE Users;          -- the schema: columns, types, keys, defaults
```

The notebook then repeats the same with the connector (`Users_py`) to show the commit rule in practice: `CREATE TABLE` through the cursor needs `db.commit()`, while `USE/SHOW` don't.

14.5. 03 — Keys, constraints and ALTER TABLE

This is the core of the schema-design part. The goal is a tiny shop database with **products**, **suppliers**, and the relationship between them.

Products. An auto-numbered primary key and sensible constraints:

```
CREATE TABLE IF NOT EXISTS products (
    productID    INT          NOT NULL UNIQUE AUTO_INCREMENT PRIMARY KEY,
    productName  VARCHAR(30)  DEFAULT '',
    unitPrice    FLOAT        NOT NULL DEFAULT 99999.99,
    unitsInStock INT UNSIGNED NOT NULL DEFAULT 0
);
```

The schema is then evolved step by step, which is how real schemas change over time:

Change	Statement
add a column	<code>ALTER TABLE products ADD COLUMN isDiscontinued BOOL NOT NULL;</code>
give it a default	<code>ALTER TABLE products ALTER isDiscontinued SET DEFAULT 0;</code>
tighten a column	<code>ALTER TABLE products MODIFY productName VARCHAR(30) NOT NULL;</code>

MODIFY must restate the full type, because it redefines the column. ALTER ... SET DEFAULT only touches the default.

Suppliers. Same idea, with `countryCode` CHAR(3) later defaulting to "ITA".

The relationship. A supplier sells many products and a product has many suppliers: a **many-to-many** relation, which needs a *junction table*:

```
CREATE TABLE supplier_to_product (prodID INT NOT NULL, supID INT NOT NULL);

ALTER TABLE supplier_to_product ADD PRIMARY KEY (prodID, supID);      --
    ↪ composite key
ALTER TABLE supplier_to_product ADD CONSTRAINT FK_prod
    FOREIGN KEY (prodID) REFERENCES products(productID);
ALTER TABLE supplier_to_product ADD CONSTRAINT FK_sup
    FOREIGN KEY (supID) REFERENCES suppliers(supplierID);
```

Neither column alone is unique (a product appears once per supplier), but the **pair** is, so the pair is the key. The foreign keys make the database refuse rows that point to non-existent products or suppliers.

Warning

DROP TABLE deletes the table *and* its rows, and breaks every foreign key pointing at it. The notebook uses it to clean up; on a real database, think twice.

14.6. 04 — Inserting, updating, deleting

```
INSERT INTO products (productName, unitPrice, unitsInStock)
VALUES ('Founders Porter', 4.9, 12);
```

Name the columns explicitly (“explicit is better than implicit”): it keeps working if the table gains columns, and lets defaults (`productID`, `isDiscontinued`) fill themselves in. `SELECT * FROM products` afterwards shows the auto-increment and the defaults at work.

From Python, use **placeholders** instead of building the string yourself. It handles quoting and protects against SQL injection. `executemany` inserts a list in one go:

```
sql = "INSERT INTO products (productName, unitPrice, unitsInStock) VALUES
      ↪ (%s, %s, %s)"
dbc.executemany(sql, [('Punk IPA', 5.2, 45), ('HB Dunkel', 4.5, 250),
                      ('Forst Xmas', 8.0, 2), ('Damm Complot', 3.7, 120)])
db.commit()
```

UPDATE and DELETE act on **every row matching WHERE**. A wrong or missing condition is how data gets lost:

```
UPDATE products SET isDiscontinued = 1 WHERE productName = "Forst Xmas";
DELETE FROM products WHERE productName LIKE "F%s";    -- starts with F, ends
               ↪ with s
```

Here only *Forst Xmas* matches (*Founders Porter* ends in “r”). With a pattern this loose it is easy to hit more rows than intended, so run the same WHERE as a SELECT first to see what you are about to change.

The junction table is filled with id pairs: (2,1) means “product 2 is sold by supplier 1”. After the DELETE, product id 4 is gone for good (auto-increment never reuses ids), so look the ids up with SELECT before inserting.

Export. mysqldump turns a database into a .sql script that recreates it. Write it into /mapd-workspace/dbs, the folder shared with your laptop:

```
docker compose exec db bash
mysqldump -u root -p my_db > /mapd-workspace/dbs/my_db.sql
```

14.7. 04 — Basic queries

Read a SELECT in its **execution order**, not its written order (see the theory): FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

```
SELECT email FROM suppliers WHERE countryCode = "ITA";
SELECT * FROM products WHERE unitPrice < 5 ORDER BY unitsInStock DESC;
```

Aggregation. GROUP BY always goes with an aggregate function (COUNT, SUM, AVG, ...):

```
res = %sql SELECT COUNT(countryCode) AS count, countryCode FROM suppliers
      ↪ GROUP BY countryCode;
res.bar()
```

Joins. The notebook runs the same join four ways on suppliers × supplier_to_product. Compare the outputs:

Form	Keeps
FROM a, b WHERE a.id = b.id (implicit)	matching pairs, same as INNER JOIN
INNER JOIN ... ON	only matching pairs
LEFT JOIN	every supplier, NULLs where it sells nothing
RIGHT JOIN	every junction row, even if its supplier were missing

Chaining two joins goes from supplier → junction → product, which is how you answer questions across a many-to-many relation:

```
SELECT suppliers.email
FROM supplier_to_product
INNER JOIN suppliers ON supplier_to_product.supID = suppliers.supplierID
INNER JOIN products  ON supplier_to_product.prodID = products.productID
WHERE products.productName = "Punk IPA";
```

14.8. 05 — Queries on the world database

Loading a ready-made database is a server-side operation:

```
docker compose exec db bash
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS world"
mysql -u root -p world < /mapd-workspace/dbs/world.sql
```

Always start an unknown database the same way: `USE world; SHOW TABLES; DESCRIBE city; SELECT * FROM city LIMIT 5;`. It has three tables, `country` (keyed by `Code`), `city` (`CountryCode` → `country.Code`) and `countrylanguage` (`CountryCode`, `Language`, `IsOfficial`, `Percentage`).

The ten questions each practise one idea:

#	Question	Idea
1	largest population	ORDER BY ... DESC LIMIT 1
2	cities in Canada	COUNT with WHERE; COUNT(Name) vs COUNT(*) vs COUNT(1) timed with %%timeit, which are equivalent here
3	country with most cities	GROUP BY CountryCode + ORDER BY the count

#	Question	Idea
4	languages in Southeast Asia	join country countrylanguage, DISTINCT; explore first with Region LIKE "%Asia"
5	5 most spoken languages	GROUP BY Language
6	city population in the Caribbean	join + SUM; here LEFT/RIGHT/INNER give the same answer
7	countries with a “Kingston”	join filtered on the city name
8	25 least dense countries	a computed column with an alias: Population/SurfaceArea AS PopDensity
9	GNP of countries independent 1850–1910	BETWEEN a AND b
10	life expectancy vs GNP, Spanish-speaking	join + IsOfficial = 1, then plot from Python

A sample, to see the pattern:

```
-- 3. the country with the most cities
SELECT CountryCode, COUNT(1) AS nCities
FROM city
GROUP BY CountryCode
ORDER BY nCities DESC
LIMIT 1;
```

For the plots, put the query result in a variable and let it plot itself, e.g. `res = %sql SELECT Name, Population/SurfaceArea AS PopDensity FROM country ORDER BY PopDensity ASC LIMIT 25; then res.bar()`.

14.9. 06 — The IMDb database

A larger, realistic schema, loaded like `world` (`CREATE DATABASE IMDb`, then `mysql ... IMDb < /mapd-workspace/dbs/IMDb_pruned.sql`, a couple of minutes). The tables you need are `Names_` (people), `Titles`, `Had_role` (person title role), `Directors`, `Title_ratings`, `Title_genres` and `Episode_belongs_to`.

Strategy for every question: **DESCRIBE the tables involved, build the query in small steps with LIMIT, then add the aggregation.** The solutions do exactly that, and reading the intermediate queries is more useful than the final one.

Birth years. Querying everything at once hides problems; looking at MIN/MAX first reveals impossible birth years (outliers), which you then cut (`WHERE birth_year > 1850`) before histogramming. The mean and standard deviation are computed *on the server* with `AVG()` and `STD()`, so only two numbers travel back.

Joins across the schema. For “who played Spider-Man”, the solution first writes it with a subquery (`WHERE name_id IN (SELECT ...)`) and then as an explicit JOIN. Prefer the join: it is clearer, and the optimiser handles it better.

```
SELECT Names_.name_, Titles.primary_title, Titles.start_year
FROM Had_role
INNER JOIN Titles ON Had_role.title_id = Titles.title_id
INNER JOIN Names_ ON Had_role.name_id = Names_.name_id
WHERE Had_role.role_ LIKE "Spider-Man" AND Titles.title_type = "movie"
ORDER BY Titles.start_year;
```

Views. For the “TV series with more than 1000 episodes” question the solution saves intermediate results as *views*, named queries you can select from like tables:

```
CREATE VIEW MoreThan1KEpisodes AS
SELECT parent_tv_show_title_id, COUNT(episode_title_id) AS nEpisodes
FROM Episode_belongs_to
GROUP BY parent_tv_show_title_id
HAVING nEpisodes > 1000;           -- HAVING filters groups, WHERE filters
↪ rows
```

Ratings. Grouping by a computed value, here the decade `FLOOR(start_year/10)*10`, and the syntax `%%sql res << SELECT ...` that stores a multi-line query’s result in `res`. For Game of Thrones, `Titles` is joined **twice** with different aliases (T for the episode, P for the parent show), which is a common trick when one table plays two roles.

14.10. On a real system

- The server would be a dedicated VM or a managed service (RDS, Cloud SQL). You connect exactly as here, with `host=<address>` instead of “db”, usually over TLS.
- **Credentials are never written in notebooks:** read them from environment variables or a secrets manager. The `root_pwd` in the compose file is only acceptable on a throwaway local container.
- Applications get their own users with only the privileges they need (`GRANT SELECT ON world.* TO 'reader'@'%')`, as you saw with `my_user`.
- `mysqldump` is the simplest backup; real deployments add replication and point-in-time recovery (Reliability).
- Big analytical queries are typically not run on the production database: the data is exported (ETL) to a warehouse or to files that Spark/Dask can process in parallel.

💡 Takeaways

- Server and client are separate machines; every tool is a client sending SQL.
- Changes need `commit()`; a cursor holds one result at a time.
- Design: primary keys identify rows, foreign keys enforce relations, many-to-many needs a junction table.
- Querying: explore with `DESCRIBE` and `LIMIT`, aggregate on the server, bring back only small results.

14.11. SQL String Functions Reference

Function	Description	Example
<code>CONCAT(a, b, ...)</code>	Concatenate strings	<code>CONCAT(first_name, ' ', last_name)</code>
<code>SUBSTRING(str, pos, len)</code>	Extract a substring	<code>SUBSTRING(name, 1, 3)</code>
<code>UPPER(str) / LOWER(str)</code>	Change case	<code>UPPER('hello') → 'HELLO'</code>
<code>TRIM(str)</code>	Remove leading/trailing spaces	<code>TRIM(' hello ') → 'hello'</code>
<code>LEFT(str, n) / RIGHT(str, n)</code>	First/last n characters	<code>LEFT('hello', 3) → 'hel'</code>
<code>LENGTH(str)</code>	String length in bytes	<code>LENGTH('hello') → 5</code>
<code>REPLACE(str, from, to)</code>	Replace all occurrences	<code>REPLACE('aababc', 'ab', 'X') → 'aXXc'</code>
<code>LOCATE(substr, str)</code>	Find position of substring	<code>LOCATE('ll', 'hello') → 3</code>
<code>FORMAT(num, d)</code>	Format number with d decimal places	<code>FORMAT(1234567.891, 2) → '1,234,567.89'</code>

```
-- Example: build a full name and show first 10 characters
SELECT UPPER(CONCAT(first_name, ' ', last_name)) AS full_name,
       SUBSTRING(bio, 1, 100) AS bio_preview
FROM Authors;
```

14.12. SQL Date and Time Functions Reference

Function	Description	Example
<code>NOW() / CURRENT_TIMESTAMP</code>	Current date and time	<code>NOW() → 2024-05-28 14:32:00</code>
<code>CURRENT_DATE</code>	Today's date	<code>CURRENT_DATE → 2024-05-28</code>

Function	Description	Example
<code>DATE_ADD(date, INTERVAL n unit)</code>	Add an interval	<code>DATE_ADD('2024-01-01', INTERVAL 30 DAY)</code>
<code>DATE_SUB(date, INTERVAL n unit)</code>	Subtract an interval	<code>DATE_SUB(NOW(), INTERVAL 1 YEAR)</code>
<code>DATEDIFF(d1, d2)</code>	Days between two dates	<code>DATEDIFF('2024-12-31', '2024-01-01') → 365</code>
<code>TIMESTAMPDIFF(unit, t1, t2)</code>	Difference in specified unit	<code>TIMESTAMPDIFF(MONTH, '2023-01-01', '2024-05-01') → 16</code>
<code>DATE_FORMAT(date, fmt)</code>	Format a date	<code>DATE_FORMAT(NOW(), '%Y-%m-%d')</code>
<code>EXTRACT(unit FROM date)</code>	Extract a component	<code>EXTRACT(YEAR FROM '2024-05-28') → 2024</code>
<code>YEAR(d) / MONTH(d) / DAY(d)</code>	Extract components	<code>YEAR(NOW()) → 2024</code>

```
-- Example: find orders placed in the last 30 days
SELECT order_id, order_date, total
FROM Orders
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
ORDER BY order_date DESC;
```

14.13. SQL Query Exercises

Exercise 14.1

Write a single SQL query to retrieve all wines made with ‘Merlot’ or ‘Pinot Noir’ grapes since 2010, sorted by alcohol percentage in descending order.

Solution

(Assumed schema — not a real one: `Wines(name, grape, year, alcohol)`.)

```
SELECT *
FROM Wines
WHERE grape IN ('Merlot', 'Pinot Noir')
AND year >= 2010
ORDER BY alcohol DESC;
```

Exercise 14.2

Write a single SQL query to retrieve the names of professors whose average student grade across all their courses is greater than 24. Consider only sufficient grades (at least 18).

Solution

(Assumed schema — not a real one: *Professors*(*prof_id*, *name*), *Courses*(*course_id*, *prof_id*), *Exams*(*student_id*, *course_id*, *grade*).)

```
SELECT p.name
FROM Professors p
JOIN Courses c ON c.prof_id = p.prof_id
JOIN Exams e ON e.course_id = c.course_id
WHERE e.grade >= 18
GROUP BY p.prof_id, p.name
HAVING AVG(e.grade) > 24;
```

Exercise 14.3

Write a single SQL query to retrieve the maximum and average employee salary per department, limited to departments located in 'New York' and 'Chicago'. Sort the result by department name.

Solution

(Assumed schema — not a real one: *Departments*(*dept_id*, *name*, *location*), *Employees*(*emp_id*, *dept_id*, *salary*).)

```
SELECT d.name          AS department,
       MAX(e.salary)    AS max_salary,
       AVG(e.salary)    AS avg_salary
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
WHERE d.location IN ('New York', 'Chicago')
GROUP BY d.dept_id, d.name
ORDER BY d.name;
```

Exercise 14.4

Write a single SQL query to retrieve earthquakes with magnitudes between 6.5 and 7.7 that occurred after 1940 from the *quaketable*, sorted by epicentral depth (greater than 0) from highest to lowest.

Solution

(Assumed schema — not a real one: *quaketable(id, magnitude, year, depth).*)

```
SELECT *
FROM quaketable
WHERE magnitude BETWEEN 6.5 AND 7.7
      AND year > 1940
      AND depth > 0
ORDER BY depth DESC;
```

Exercise 14.5

Provide SQL queries demonstrating the following:

- One query using WHERE, AND, and OR.
- Two queries each using JOIN, ORDER BY (A–Z by name), GROUP BY, and the LIKE operator.

Solution

(All schemas below are hypothetical, not real.)

```
-- WHERE / AND / OR
SELECT *
FROM Employees
WHERE department = 'Sales'
      AND (salary > 50000 OR years_experience > 5);
```

```
-- JOIN + ORDER BY + GROUP BY + LIKE (1)
SELECT d.name AS department, COUNT(e.emp_id) AS headcount
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
WHERE e.title LIKE 'Senior%'
GROUP BY d.name
ORDER BY d.name ASC;
```

```
-- JOIN + ORDER BY + GROUP BY + LIKE (2)
SELECT a.name AS author, COUNT(b.book_id) AS books_written
FROM Authors      a
JOIN BookAuthors ba ON ba.author_id = a.author_id
JOIN Books        b ON b.book_id    = ba.book_id
WHERE b.title LIKE '%Physics%'
GROUP BY a.name
ORDER BY a.name ASC;
```

Exercise 14.6

Write a single SQL query to retrieve the average years of experience of employees who are older than 30 years and have a salary greater than \$50,000, grouped by department.

Solution

(Assumed schema — not a real one: *Employees(emp_id, department, age, salary, years_experience)*.)

```
SELECT department,
       AVG(years_experience) AS avg_years_experience
FROM Employees
WHERE age > 30
       AND salary > 50000
GROUP BY department;
```

Exercise 14.7

Using the world database, write a single SQL query to retrieve the average population of cities in each country that has a population greater than 10 million, grouped by continent.

Solution

(Standard world schema — *country(Code, Continent, Population)*, *city(CountryCode, Population)*.)

```
SELECT c.Continent,
       AVG(ci.Population) AS avg_city_population
FROM country c
JOIN city    ci ON ci.CountryCode = c.Code
WHERE c.Population > 10000000
GROUP BY c.Continent;
```

Exercise 14.8

Using the `world` database, write a single SQL query to retrieve the total number of languages spoken in countries that are located in Asia and have a population greater than 50 million, grouped by region.

Solution

(Standard `world` schema — `country(Code, Continent, Region, Population)`, `countrylanguage(CountryCode, Language)`.)

```
SELECT c.Region,
       COUNT(DISTINCT cl.Language) AS total_languages
FROM country      c
JOIN countrylanguage cl ON cl.CountryCode = c.Code
WHERE c.Continent = 'Asia'
      AND c.Population > 50000000
GROUP BY c.Region;
```

Exercise 14.9

Using the IMDb database, write a single SQL query to retrieve the average movie rating of movies in the “Action” genre that have a runtime of more than 90 minutes, grouped by year.

Solution

(Assumed IMDb-style schema — not the official one: `Movies(movie_id, year, runtime)`, `Genres(movie_id, genre)`, `Ratings(movie_id, rating)`.)

```
SELECT m.year,
       AVG(r.rating) AS avg_rating
FROM Movies m
JOIN Genres g ON g.movie_id = m.movie_id
JOIN Ratings r ON r.movie_id = m.movie_id
WHERE g.genre = 'Action'
      AND m.runtime > 90
GROUP BY m.year;
```

Exercise 14.10

Using the IMDb database, write a single SQL query to retrieve the top 5 persons who have worked in the most movies, ordered by the number of movies they have worked in (descending).

Solution

(Assumed schema — not real: *Persons*(*person_id*, *name*), *Crew*(*person_id*, *movie_id*).)

```
SELECT p.name,
       COUNT(DISTINCT c.movie_id) AS movie_count
FROM Persons p
JOIN Crew c ON c.person_id = p.person_id
GROUP BY p.person_id, p.name
ORDER BY movie_count DESC
LIMIT 5;
```

Exercise 14.11

Using the IMDb database, retrieve the average rating of movies in the “Comedy” genre that were released after 2000 and have more than 100,000 votes, grouped by director. Include only directors who have directed more than 5 such movies.

Solution

(Assumed schema — not real: *Movies*(*movie_id*, *year*, *votes*), *Genres*(*movie_id*, *genre*), *Ratings*(*movie_id*, *rating*), *Crew*(*person_id*, *movie_id*, *role*), *Persons*(*person_id*, *name*).)

```
SELECT p.name                AS director,
       AVG(r.rating)         AS avg_rating
FROM Persons p
JOIN Crew c ON c.person_id = p.person_id
              AND c.role    = 'director'
JOIN Movies m ON m.movie_id = c.movie_id
JOIN Genres g ON g.movie_id = m.movie_id
JOIN Ratings r ON r.movie_id = m.movie_id
WHERE g.genre = 'Comedy'
      AND m.year > 2000
      AND m.votes > 100000
GROUP BY p.person_id, p.name
HAVING COUNT(DISTINCT m.movie_id) > 5;
```

Exercise 14.12

Using the IMDb database, retrieve the total number of movies and the average runtime of movies in each genre, considering only movies released after 2010. Group by genre. Exclude genres with 10 or fewer movies and genres where the average runtime is less than 90 minutes.

Solution

(Assumed schema — not real: *Movies(movie_id, year, runtime)*, *Genres(movie_id, genre)*.)

```
SELECT g.genre,  
       COUNT(DISTINCT m.movie_id) AS total_movies,  
       AVG(m.runtime)             AS avg_runtime  
FROM Movies m  
JOIN Genres g ON g.movie_id = m.movie_id  
WHERE m.year > 2010  
GROUP BY g.genre  
HAVING COUNT(DISTINCT m.movie_id) > 10  
      AND AVG(m.runtime) >= 90;
```

15. Lab: Threads and processes in Python

Lab at a glance

- **Scripts:** `code/parallel/*.py` (run from a terminal, no Docker)
- **Theory:** Processing and Concurrency: processes vs threads, race conditions, the GIL, Amdahl's law
- **You will:** go from one thread to many threads to many processes, break shared state with a race condition and fix it, and measure when threading helps (I/O) and when only processes help (CPU)

15.1. The idea behind it

Before distributing work over many machines, use all the cores of one machine. This lab is the single-node foundation for everything after it:

- **Threads vs processes.** Threads share one memory space (cheap communication, but races); processes have separate memories (safe, but data must be copied between them). Distributed frameworks are “processes on different machines”: the same trade-offs, with a network in between.
- **Amdahl's law** (theory): the serial part of a program bounds the speed-up. Every lock, every join, every final sum is serial.
- **CPU-bound vs I/O-bound:** waiting can be overlapped (threads are enough), while computing needs more cores (processes, or more machines).

15.2. How to run it

```
cd code/parallel
pip install colorama # used by 03_multi_threaded_nolock.py
python3 00_single_threaded.py
```

Watch every run in a second terminal with `htop` (one bar per core). The whole lab is about *what those bars do*: one bar at 100% means one core is working; several bars at 100% means real parallelism.

15. Lab: Threads and processes in Python

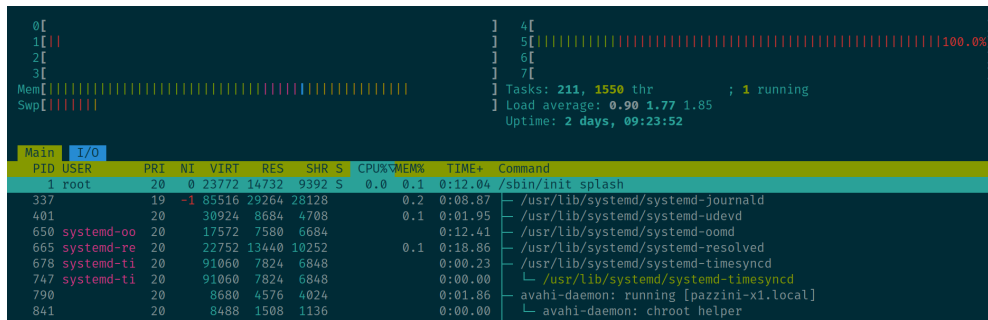


Figure 15.1.: `htop` while a multi-process script runs: several cores busy at once.

Two Python versions make the GIL visible. With a standard build (3.12 or the default 3.13+ build), threads never use more than one core for Python code; with a free-threaded build (`python3.13t`, GIL disabled) they can. With `uv`: `uv run --python 3.13t script.py`.

To pin a run to specific cores, which is useful to simulate a machine with fewer cores:

```
taskset -c 0 python3 07_multi_processing.py # only core 0: processes
↳ can no longer run in parallel
taskset -c 0,1 python3 07_multi_processing.py # cores 0 and 1
```

💡 What to look for in each script

1. **Wall time** printed at the end, compared with `00_single_threaded.py`.
2. **htop**: how many cores are busy.
3. **Correctness**: do the counters and lists at the end have the values they should?

15.3. The scripts, in order

15.3.1. 00 — Single-Threaded Baseline

A simple CPU-bound task: sum all integers in a range. This serves as the performance baseline before introducing concurrency. The wall-time here is the cost of pure sequential execution on a single core.

15.3.2. 01 — Two Concurrent Threads

The same summation split across two `threading.Thread` instances. `start()` is non-blocking (launches the thread), while `join()` blocks until that thread finishes. Due to the GIL, wall-time will not improve for CPU-bound work — both threads still share a single interpreter lock.

15.3.3. 02 — Thread-Local Data

Uses `threading.local()` to create storage that is private to each thread. Even though the variable name is the same across threads, each thread sees its own independent copy — no sharing, no synchronisation needed.

```

import threading

local_data = threading.local()

def worker():
    local_data.counter = 0
    while True:
        local_data.counter += 1
        print(local_data.counter)
        if local_data.counter == 1000000:
            return
    
```

15.3.4. 03extra — Minimal Race Condition

Ten threads each increment a shared `counter` 1 000 000 times without any lock. The read–modify–write sequence (`counter += 1`) is not atomic, so threads interleave and overwrite each other’s updates. The final value is almost certainly **less than** 10 000 000.

```

import threading

counter = 0

def worker():
    global counter
    for _ in range(1000000):
        counter += 1
    
```

15.3.5. 03 — Multi-Threaded Without Lock (Race Condition)

Two threads concurrently write to `COMMON_COUNTER` and `COMMON_LIST` with no synchronisation. The mismatch between the counter and the list length at the end reveals the race condition in action.

```

import threading

COMMON_COUNTER = 0
COMMON_LIST = []

def worker():
    global COMMON_COUNTER, COMMON_LIST
    for _ in range(1000000):
        COMMON_COUNTER += 1
        COMMON_LIST.append(1)
    
```

15.3.6. 04extra — Minimal Race Condition Fixed with Lock

The counter example from above, now protected by `threading.Lock()`. Every `counter += 1` is bracketed by `acquire()` / `release()`, so only one thread modifies the counter at a time. The final value is always exactly 10 000 000.

```

import threading

counter = 0
lock = threading.Lock()

def worker():
    global counter
    for _ in range(1000000):
        lock.acquire()
        counter += 1
        lock.release()
    
```

15.3.7. 04 — Multi-Threaded With Lock

The full race-condition scenario corrected: each access to `COMMON_COUNTER` and `COMMON_LIST` is guarded by `the_lock.acquire()` / `the_lock.release()`. The lock is passed explicitly as an argument so each thread uses the same shared lock object.

```

import threading

COMMON_COUNTER = 0
COMMON_LIST = []
the_lock = threading.Lock()

def worker(lock):
    global COMMON_COUNTER, COMMON_LIST
    for _ in range(1000000):
        lock.acquire()
        COMMON_COUNTER += 1
        COMMON_LIST.append(1)
        lock.release()
    
```

15.3.8. 05 — Thread Pool with Lock (`concurrent.futures`)

`ThreadPoolExecutor` manages a pool of reusable worker threads. `submit()` dispatches a task and returns a `Future`; `wait()` blocks until all futures complete. The return value from each thread is retrieved with `f.result()` — a cleaner pattern than writing to global state.

15.3.9. 06 — Thread Pool with Queue

Replaces the explicit lock with a `queue.Queue`, which is inherently thread-safe (its internal lock is managed automatically). Threads call `put()` to deposit results; the main thread reads them after all workers finish. Queues are the idiomatic Python pattern for producer–consumer communication.

15.3.10. 07 — Multi-Processing

`multiprocessing.Process` spawns a completely separate OS process with its own Python interpreter and memory space, bypassing the GIL entirely. True CPU parallelism is possible: both processes can execute Python bytecode simultaneously on different cores.

15.3.11. Synchronous vs Asynchronous Execution

Before the next two scripts, it is useful to define these terms precisely:

- **Synchronous (blocking)**: the calling code waits for the operation to complete before continuing. Execution is sequential.
- **Asynchronous (non-blocking)**: the calling code submits the operation and continues immediately. Results are collected later via a handle or callback.

In the context of `multiprocessing.Pool`: - `starmap()` → **synchronous**: blocks until all worker processes finish. - `starmap_async()` → **asynchronous**: returns an `AsyncResult` immediately; call `.get()` when results are needed.

15.3.12. 08 — Multiprocessing Pool (`starmap`, blocking)

`Pool.starmap()` distributes a list of argument tuples across a pool of worker processes. The call **blocks** until all workers finish, then returns the collected results as a list. Each worker independently sums its sub-range and returns the partial sum.

15.3.13. 09 — Multiprocessing Pool (`starmap_async`, non-blocking)

`starmap_async()` returns immediately with an `AsyncResult` object, allowing the main process to continue (or poll) while workers run. `.get()` blocks until all results are ready. This pattern is useful when the main process has other work to do while waiting.

15.4. Benchmarking: I/O-bound vs CPU-bound Tasks

The following pairs of scripts directly compare single-threaded, multi-threaded, and multi-process execution on two contrasting workloads.

15.4.1. I/O-Bound Tasks

I/O-bound tasks spend most of their time waiting for external operations (disk, network). The GIL is released while waiting for I/O, so **threading is effective** here — multiple threads can overlap their wait times even though only one runs Python bytecode at a time.

Single-threaded — downloads five images sequentially; each download blocks until complete:

Multi-threaded — the same downloads dispatched concurrently via `ThreadPoolExecutor`; all five network waits overlap. Significant speedup is expected:

Multi-process — same task using `multiprocessing.Pool`. For pure I/O work the process-spawn overhead typically outweighs any benefit; performance is similar to or worse than threading:

15.4.2. CPU-Bound Tasks

CPU-bound tasks are limited by raw computation speed. Because the GIL serialises Python bytecode execution, **threading provides no speedup** for CPU-bound code — threads take turns rather than running in parallel. Multiprocessing spawns separate interpreter instances that truly run in parallel on different cores.

15. Lab: Threads and processes in Python

The example performs a Monte Carlo numerical integration of $f(x) = \sin^2\left(\frac{1}{x(2-x)}\right)$ over $[0, 2]$ across multiple independent chunks.

Single-threaded — 10 chunks processed one after another:

Multi-threaded — 10 chunks dispatched to a `ThreadPoolExecutor`; the GIL prevents true parallelism, so the result is similar to (or slower than) single-threaded due to context-switch overhead:

Multi-process — 10 chunks dispatched to a `ProcessPoolExecutor`; each chunk runs in its own interpreter with no GIL contention. True speedup proportional to the number of cores:

15.5. On a real system

- A cloud VM's **vCPU** is usually one *hyper-thread*, so 8 vCPUs may be 4 physical cores (Hyper-Threading). Benchmark CPU-bound code before trusting the number.
- On shared clusters, your job gets an **allocation** from a batch system (e.g. Slurm, see Batch systems). `taskset` in this lab mimics that: you only get the cores you are given.
- When one machine is not enough, the same “split the work, run independent pieces, combine results” structure moves to a cluster framework. `multiprocessing.Pool.starmap` is conceptually a tiny `map + reduce`, which is exactly what the next labs scale out.

💡 Takeaways

- Threads share memory: cheap to start and to communicate, but every shared write needs a lock (or a thread-safe `queue.Queue`).
- With the GIL, threads speed up **waiting** (I/O), not **computing**. For CPU-bound Python code, use processes.
- Processes have separate memory: no race conditions on Python objects, but arguments and results are copied between them, and starting them costs time.
- Pools (`ThreadPoolExecutor`, `ProcessPoolExecutor`, `multiprocessing.Pool`) are the practical interface; blocking (`starmap`) vs non-blocking (`starmap_async`, futures) decides whether the main program waits.

16. Lab: Spark 1, RDDs and map-reduce

Lab at a glance

- **Notebook:** `code/spark/notebooks/Lecture1.ipynb` (solutions: `Lecture1_solutions.ipynb`)
- **Stack:** `spark-master` + `spark-worker` ($\times N$) + `jupyter-notebook`, see Running the labs
- **Theory:** Apache Spark: driver/executors, RDDs, transformations vs actions, the DAG. Also MapReduce
- **You will:** connect to a cluster, distribute a list, write map/reduce pipelines, count words, estimate π , and fit a line with a distributed gradient descent

16.1. The idea behind it

Spark generalises MapReduce (theory) with three ideas that this lab makes visible:

- **Data is partitioned across workers, and code is shipped to it.** Your lambdas are serialised and sent to the executors; only results come back.
- **Lazy evaluation and the DAG.** Transformations only build a graph; an action makes the scheduler cut it into **stages** at every shuffle and run one **task per partition**. *Narrow* transformations (`map`, `filter`) need no data movement and are pipelined in one stage; *wide* ones (`reduceByKey`) need a shuffle and start a new stage.
- **Resilience through lineage.** An RDD remembers *how* it was computed. If an executor dies, Spark recomputes the lost partitions from that recipe instead of keeping replicas. This is the “R” in RDD, and the reason recomputation (and therefore caching) matters.

16.2. Before you start

```
cd code/spark
docker compose up --scale spark-worker=2      # two workers make the
↪ parallelism visible
```

Keep three browser tabs open:

Tab	What it shows
localhost:1234	Jupyter, where you open notebooks/Lecture1.ipynb
localhost:8080	master UI : registered workers, their cores and memory, running applications
localhost:4040	application UI (exists only while your <code>SparkSession</code> is alive): jobs, stages, tasks, storage

The habit to build in this lab is: **run a cell, then look at 4040** to see what Spark actually did.

16.3. Connecting: the SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("My first spark application") \
    .config("spark.executor.memory", "512m") \
    .getOrCreate()

sc = spark.sparkContext          # the entry point for RDDs
```

What happens: the notebook becomes the **driver** of a new *application*. The master allocates it **executors** on the workers (one per worker here, with the memory you asked for). Refresh :8080 and the application appears; open :4040 to follow it.

`getOrCreate()` reuses an existing session, so re-running the cell is harmless. Changing configuration requires `spark.stop()` first.

16.4. Parallelize, partitions and laziness

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
dist_data = sc.parallelize(data)    # nothing happens yet
dist_data.count()                  # now a job runs
```

After `parallelize`, the application UI shows **no job**. `parallelize` is a *transformation*: it only records what to do. `count()` is an *action*: it needs a result, so Spark finally builds the plan and runs it. This laziness is the most important idea of the lab: you write a whole pipeline of transformations, and nothing runs until an action asks for a value.

In the job's stage view you will see **one task per partition**. An RDD is split into partitions, and a partition is the unit of parallelism: one core processes one partition at a time.



Figure 16.1.: Partitions are the unit of parallelism: each core takes one partition at a time.

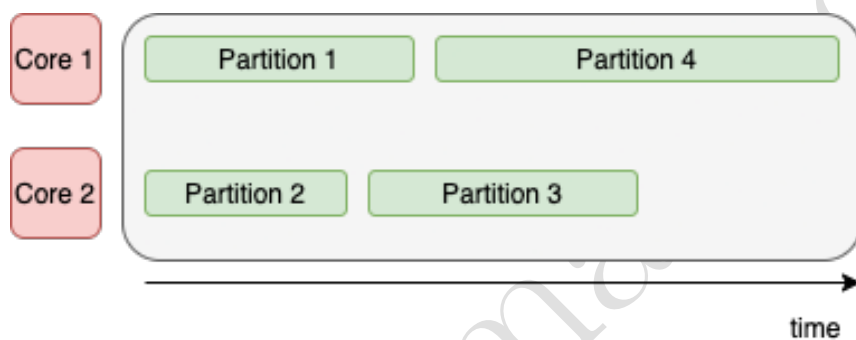


Figure 16.2.: With 2 cores and 4 partitions, two partitions wait for a free core.

```
sc.parallelize(data, numSlices=8).getNumPartitions()    # 8
dist_data.getNumPartitions()                           # default number of
↳ cores
```

Fewer partitions than cores leaves cores idle; many more partitions than cores adds scheduling overhead. Always check what you got.

Where the data lives. `print(dist_data)` prints a description, not numbers: the data is in the executors. Bringing it back is explicit:

Action	Brings back	Use
<code>collect()</code>	everything , to the driver	only for small, final results
<code>take(n)</code>	the first <code>n</code> elements (reads as few partitions as needed)	peeking at data
<code>count()</code> , <code>reduce(f)</code> , <code>min()</code> ...	one value	aggregates

`collect()` on a large RDD is the classic way to crash the driver (out of memory).

16.5. Map and reduce

`map(f)` applies `f` to every element (a transformation); `reduce(f)` combines elements pairwise until one value is left (an action):

```
dist_data.map(lambda x: x + 1).collect()           # [2, 3, ..., 9]
dist_data.map(lambda x: x + 1).map(lambda x: x / 2).collect()
dist_data.map(lambda x: x + 1).reduce(lambda x, y: x + y) # 44
```

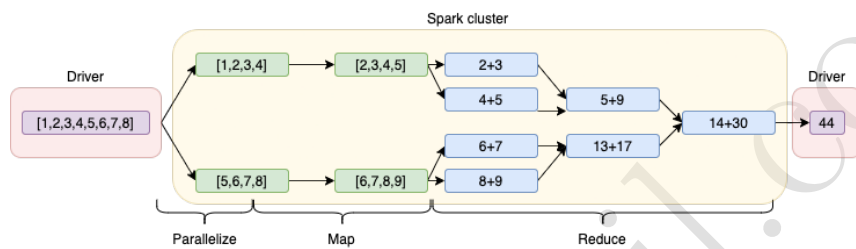


Figure 16.3.: `map` works on each element independently; `reduce` combines pairs until a single value remains.

Unlike Hadoop, Spark is not limited to one map followed by one reduce. You can chain any number of transformations. In the UI, the two chained maps appear as **one stage**: the DAG scheduler *pipelines* them, so each element goes through both functions in one pass without intermediate storage.

The reduce function must be **associative and commutative** (sum, min, max...), because partitions are reduced independently and then combined in arbitrary order.

16.5.1. Exercises 1–2

Exercise 16.1

Find the minimum of `dist_data` with `reduce`.

Solution

`dist_data.reduce(lambda x, y: min(x, y))`. Any two-argument function works, so `dist_data.reduce(min)` also does, and for this common case there is the built-in action `dist_data.min()`.

Exercise 16.2

Create 100 random 3D points on the client, distribute them, and find the largest distance from the origin.

Solution

```
import random
points = [(random.random(), random.random(), random.random()) for _ in
           ↪ range(100)]
sc.parallelize(points) \
    .map(lambda p: (p[0]**2 + p[1]**2 + p[2]**2) ** 0.5) \
    .reduce(max)
```

The lesson is the cost model: generating data on the client and **parallelize**-ing it moves it through the driver to the executors. Fine for 100 points; for real data, **read it directly on the workers** (exercise 4).

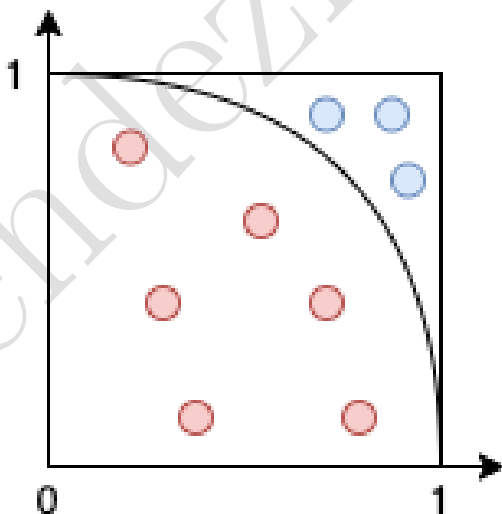
16.5.2. Intermezzo: broadcast variables

If every task needs the same small object (a lookup table, parameters), referencing it inside a lambda ships a copy **with every task**. `sc.broadcast(obj)` sends it once per executor and keeps it there:

```
lookup = sc.broadcast({"a": 1, "b": 2, "c": 3})
sc.parallelize(["a", "b", "c", "a", "d"]).map(lambda k: lookup.value.get(k,
↪ 0)).collect()
```

16.5.3. Exercise 3: estimate

Throw random points in the unit square and count those inside the quarter circle: $\pi \approx 4 N_{\text{in}}/N$.



The trick is that the **points are generated on the workers**. You only distribute placeholders:

```

num_points = 100_000
points_rdd = sc.parallelize([0] * num_points, numSlices=8)    # 8 tasks

inside = points_rdd.map(lambda _: random.random()**2 + random.random()**2) \
    .filter(lambda r2: r2 < 1) \
    .count()
print("pi =", 4 * inside / num_points)

```

`filter(f)` keeps the elements where `f` is true. The first solution uses `map` to 0/1 and `reduce(+)` instead; both give the same result.

16.6. Key-value pairs, `reduceByKey`, `flatMap`

Many problems are “aggregate per key”. Store elements as (key, value) tuples:

```

class_rdd = sc.parallelize([('group1', 10), ('group2', 4), ('group3', 1),
                           ('group2', 7), ('group1', 8)])

class_rdd.mapValues(lambda v: v + 1)                # touch values, keep
    ↪ keys
class_rdd.reduceByKey(min).collect()                # one result per key
class_rdd.reduceByKey(max).takeOrdered(2, key=lambda kv: -kv[1]) # top-2
    ↪ by value

```

`reduceByKey` is a **transformation**, despite its name, because its result can still be huge (one entry per key). It is also the first operation here that needs a **shuffle**: values with the same key must be brought to the same partition, which moves data between workers. In the UI this shows up as a **new stage**. Shuffles are where distributed jobs spend their time.

`flatMap(f)` is `map` where `f` returns a list, then flattens all lists into one RDD: one element in, *any number* out.

16.6.1. Exercise 3b: word count

The MapReduce “hello world”, now in three lines:

```

words = sc.parallelize(message).flatMap(clean_row)    # line → list of
    ↪ clean, lowercase words
counts = words.map(lambda w: (w, 1)).reduceByKey(lambda a, b: a + b)
counts.takeOrdered(10, key=lambda kv: -kv[1])        # the 10 most
    ↪ frequent words

```

`clean_row` strips punctuation (`string.punctuation`) and lowercases, so “Ring,” and “ring” count as the same word.

16.7. Exercise 4: distributed linear regression

The most complete example of the lab. Files `datasets/lecture1/file_*.txt` hold lines like `Measure 0: (.05,0.70)`, some with `None` values.

Step 1: read on the workers. Instead of reading files in the notebook, distribute the **list of file names** and let each executor open its own files:

```
files = [f'/mapd-workspace/datasets/lecture1/file_{i}.txt' for i in range(1,
↪ 5)]
data_rdd = sc.parallelize(files).flatMap(parse_file)      # parse_file ↪
↪ list of (t, val)
```

The path is the one **inside the containers**. `parse_file` uses a regular expression, `-?[0-9]*\.[0-9]*` twice separated by a comma, to extract the pair, and skips lines without numbers (the `None` measurements). For plain text, `sc.textFile("file:///mapd-workspace/...")` does the reading for you. The custom loader is shown because real formats usually need one.

`data_rdd.sample(False, 0.2).collect()` brings back a random 20% for a quick scatter plot without collecting everything.

Step 2: gradient descent as map-reduce. For the model $y = w_0 + w_1x$ and squared loss, the gradient is a **sum over points**:

$$\nabla J = \frac{1}{n} \sum_i (y(w, x_i) - y_i) (1, x_i)$$

A sum over points is exactly a map (per-point gradient) followed by a reduce (add them up):

```
W, lr, n = np.array([10, 0.5]), 0.01, data_rdd.count()
for _ in range(20):
    grad = data_rdd.map(lambda P: gradient(P, W)).reduce(lambda a, b: a + b)
    W = W - lr / n * grad
```

Each iteration is one Spark job: the current `W` is shipped to the executors inside the `lambda`, every executor computes partial gradients on its partitions, and only a 2-vector comes back.

Step 3: residuals without collecting. `residuals_rdd.collect() + plt.hist` works here but would not scale. `rdd.histogram(bins)` computes the counts on the cluster and returns only the bins:

```
counts = residuals_rdd.histogram(np.linspace(-1, 1, 50).tolist())
plt.stairs(counts[1], counts[0])
```

16.8. Caching

Open the application UI after the gradient descent: **every iteration re-ran everything from the start**, re-reading and re-parsing the files 20 times. RDDs are recomputed from their lineage on every action unless you keep them:

```
data_rdd = data_rdd.persist()    # mark for caching (lazy, like any
    ↪ transformation)
data_rdd.count()                 # first action fills the cache
# ... gradient descent loop: now fast; see the "Storage" tab in :4040
data_rdd = data_rdd.unpersist() # free executor memory when done
```

Cache what you reuse in a loop; don't cache everything, because memory is limited and caching itself costs time.

16.9. On a real cluster

- **Submitting.** Instead of a notebook, production jobs are scripts run with `spark-submit --master <yarn|k8s://...|spark://...> job.py`, often from a scheduler (cron, Airflow).
- **Reading data.** Files sit in HDFS or object storage and are read *by the executors*: `sc.textFile("hdfs://namenode:8020/path")` or `"s3a://bucket/path"`. Each block becomes a partition, scheduled where possible on the node that stores it (**data locality**).
- **Sizing.** You choose the number of executors, cores per executor and memory (`--num-executors`, `--executor-cores`, `--executor-memory`), or let **dynamic allocation** add and remove executors with the load. A rule of thumb is 2–4 partitions per core.
- **Failures are normal.** With hundreds of nodes, some fail during long jobs; lineage-based recomputation is what makes that a non-event. Very long lineages (e.g. iterative algorithms) are cut with `checkpoint()` to reliable storage.
- **Monitoring** uses the same UI you saw on `:4040` (and the history server after the job ends). Long stages, skewed tasks and large shuffles are what you look for.

16.10. Wrap up

```
sc.stop(); spark.stop()    # releases the executors: the app disappears from
    ↪ :8080
```

Then `docker compose down`.

💡 Takeaways

- Transformations are lazy and build a DAG; actions trigger jobs. Learn which is which.
- Partitions are the unit of parallelism: check `getNumPartitions()` against your cores.
- Move code to data: read files on the workers, and bring back only small results (**take**,

`reduce, histogram, not collect`).

- `reduceByKey` and other per-key operations shuffle data, which costs a new stage.
- Persist RDDs you reuse in loops, and unpersist them afterwards.

17. Lab: Spark 2, DataFrames and the dimuon spectrum

Lab at a glance

- **Notebook:** `code/spark/notebooks/Lecture2.ipynb` (solutions: `Lecture2_solutions.ipynb`)
- **Stack:** same as Spark 1
- **Theory:** Apache Spark $\rightarrow$ DataFrame
- **You will:** build DataFrames in several ways, then run a full physics analysis: load CMS open data, filter, flatten nested records, compute the dimuon invariant mass, and compare data with simulation

17.1. The idea behind it

- **Schema-aware processing.** Once Spark knows the columns and types, it can optimise your query (the *Catalyst* optimiser): push filters before joins, read only the needed columns, generate efficient JVM code. The same idea underlies SQL databases (MySQL lab), now applied to distributed files.
- **Declarative over imperative.** With RDDs you say *how* (call this lambda on every element); with DataFrames you say *what* (filter, group, aggregate) and Spark decides how. Everything you write in Python is a plan; execution stays in the JVM unless you force Python code in with UDFs.
- **Physics analysis is an ETL pipeline:** Extract (read raw events), Transform (select, flatten, compute features), Load/reduce (histograms). The same shape appears in industry pipelines.

17.2. Why DataFrames

An RDD is a distributed list of arbitrary Python objects: Spark cannot see inside them, so it cannot optimise anything. A **DataFrame** is a distributed *table*: named columns with types (a **schema**). Knowing the structure lets Spark plan the query (the Catalyst optimiser), store data column-wise, and run most operations inside the JVM without touching Python. Underneath, it is still partitions of an RDD (`df.rdd`), so everything from Spark 1 about laziness, partitions, `collect` and `persist` still applies.

The session is the same as before, with two extra settings that switch on **Apache Arrow** for moving data between the JVM and Python (used by `toPandas()` and pandas UDFs):

```
.config("spark.sql.execution.arrow.pyspark.enabled", "true")
.config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false")
```

17.3. Creating DataFrames

From	How	Schema
list of Row(a=1, b=2., ...)	<code>spark.createDataFrame(...)</code>	inferred from the Rows
list of tuples	<code>spark.createDataFrame(...)</code>	given explicitly <code>schema='a long, b double, ...'</code>
a pandas DataFrame	<code>spark.createDataFrame(pandas_df)</code>	from pandas dtypes
files	<code>spark.read.format('json').load(path)</code>	inferred or given
an RDD of Rows	<code>rdd.toDF()</code>	inferred

`df.show()` prints rows (an action); `df.printSchema()` prints the structure (no job).

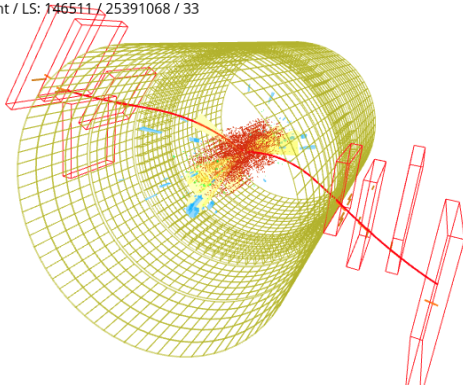
The `toDF()` route is the general escape hatch: when the file format is not supported natively, parse it with RDD code (as in Spark 1), produce `Row` objects, and convert.

pyspark.pandas offers a near-identical pandas API on top of Spark (`ps.DataFrame(...)`). It is handy for porting pandas code, but not everything is implemented, and some pandas habits are expensive when distributed. For example, there is **no natural row order** across partitions, and enforcing one means a full sort. The lab uses the regular PySpark API.

17.4. The analysis: dimuon invariant mass

The dataset is a small sample of CMS 2010 events (CERN Open Data), already converted to JSON, plus matching Monte Carlo simulation. Each record has `nMuon`, `sample` ("data" or "mc") and a nested array `Muons`, where each muon has `E`, `px`, `py`, `pz`, `charge`.

 CMS Experiment at the LHC, CERN
Data recorded: 2010-Sep-23 20:01:32.604966 GMT
Run / Event / LS: 146511 / 25391068 / 33



The physics: when a particle decays into two muons of opposite charge, the **invariant mass** of the pair, $M = \sqrt{E^2 - |\vec{p}|^2}$ with $E, \vec{p}$ summed over both muons, peaks at the parent's mass. In this

range that means the Z boson at ~91 GeV.

17.4.1. Step 0: load and inspect

```
df = spark.read.option("inferTimestamp", "false").option("prefersDecimal",
    ↪ "false") \

    ↪ .format('json').load('/mapd-workspace/datasets/lecture2/dimuon/*.json')
df.printSchema()
df.groupBy('sample').count().show()
```

Reading is lazy too: Spark only samples the files to infer the schema. Look at the schema closely: Muons is `array<struct<...>>`, so the data is **not flat**, and dealing with that is half the exercise.

Every later action would re-read the JSON from disk, so the solution caches it:

```
df = df.persist()    # NOT in place: without the assignment df still reads
    ↪ from files
df.count()           # trigger it
```

17.4.2. Step 1: aggregate on the cluster, plot locally

Histogram of muons per event, data vs MC:

```
num_muons = df.groupBy('sample', 'nMuon').count().sort('nMuon').toPandas()
```

This is the pattern for the whole lab: **the cluster does the heavy aggregation, and only a small table comes back** (`toPandas()`, which is fast thanks to Arrow) for matplotlib. `toPandas()` has the same danger as `collect()`: never call it on the raw dataset.

17.4.3. Step 2: filter and flatten

Keep events with exactly two muons. These are equivalent, so use whichever reads best:

```
from pyspark.sql.functions import col
df[df['nMuon'] == 2]
df.where(col('nMuon') == 2)      # where == filter
df.filter("nMuon == 2")         # SQL expression string

df.createOrReplaceTempView("dimuon_df_table")
spark.sql("SELECT * FROM dimuon_df_table WHERE nMuon = 2")  # plain SQL
```

All four produce the **same plan**, because the optimiser sees through the syntax.

Flattening turns the nested array into ordinary columns:

```

dimuon_flat = (
    dimuon_df.select(col('sample'), col('nMuon'), col('Muons')[0],
        ↪ col('Muons')[1])
        .withColumn('E1', col('Muons[0].E')) .withColumn('px1',
        ↪ col('Muons[0].px'))
        .withColumn('py1', col('Muons[0].py')) .withColumn('pz1',
        ↪ col('Muons[0].pz'))
        .withColumn('c1', col('Muons[0].charge'))
        # ... same for muon 2 ...
        .drop('Muons[0]', 'Muons[1]')
)

```

`select` chooses columns, `withColumn(name, expr)` adds or replaces one, and `drop` removes them. Each returns a **new** DataFrame, since DataFrames are immutable.

17.4.4. Step 3: new features (native functions vs UDFs)

Opposite charge, then the four-momentum of the pair and its mass, **using column expressions**:

```

from pyspark.sql.functions import sqrt
dimuon_flat = (dimuon_flat.where(col('c1') != col('c2'))
    .withColumn('E', col('E1') + col('E2'))
    .withColumn('px', col('px1') + col('px2')) # ... py, pz
    .withColumn('M', sqrt(col('E')**2 - (col('px')**2 + col('py')**2 +
        ↪ col('pz')**2))))

```

The notebook then computes the same mass in two other ways, to compare the cost:

Way	Code	What runs where
native column functions	<code>sqrt(col(...)) ...</code>	entirely in the JVM, optimised, so fastest
Python UDF	<code>@udf('float') def m(E, px, py, pz): return math.sqrt(...)</code>	rows are shipped one by one to a Python process and back, so slowest
pandas UDF	<code>@pandas_udf('float') def m(E: pd.Series, ...) -> pd.Series: return np.sqrt(...)</code>	whole batches go to Python via Arrow and are processed vectorised with numpy, a good middle ground

The rule: **use a native function if one exists**, a pandas UDF if you need custom Python, and a plain UDF only as a last resort. The same comparison is repeated for an aggregation (mean energy per sample): `native groupBy('sample').mean()` vs `groupBy('sample').agg(mean_udf(col('E')))` with a pandas UDF that returns a scalar.

17.4.5. Step 4: rediscover the Z boson

The histogram is computed **on the cluster** with the RDD histogram from Spark 1, so only bin counts travel:

```
bins, counts = dimuon_flat.where(col('sample') == 'data').select('M') \
    .rdd.map(lambda r: r.M) \
    .histogram(list(np.arange(0, 150, 4)))
```

Repeat for mc, then plot data as points with $\sqrt{N}$ error bars over the simulation as a filled histogram. The full range (log scale) shows several resonances; zooming into 70–115 GeV with 0.5 GeV bins shows the Z peak and where the simulation disagrees with data.

A selection to improve agreement. The transverse momentum of the pair, $p_T = \sqrt{(p_{x1} + p_{x2})^2 + (p_{y1} + p_{y2})^2}$, computed with a pandas UDF (`np.hypot`), then a boolean column with `when/otherwise`, the DataFrame version of an if/else:

```
from pyspark.sql.functions import when
dimuon_flat = dimuon_flat.withColumn("pT", candidate_pt(col('px1'),
    ↪ col('py1'), col('px2'), col('py2')))) \
    .withColumn("highpt", when(col('pT') > 30,
    ↪ True).otherwise(False))
```

Plotting low- p_T and high- p_T candidates side by side shows how the selection changes the background and the data–MC agreement.

Warning

In the solutions, the `pandas_udf` `candidate_pt` declares its parameters as `(px1, px2, py1, py2)` but is called with `(px1, py1, px2, py2)`. Arguments are matched **by position**, so inside the function `px2` is actually `py1`. Write the signature in the same order you call it.

17.5. On a real cluster

- **File formats matter more than code.** JSON is convenient but slow (text, parsed row by row, schema inferred). Real pipelines convert once to **Parquet**: columnar, compressed, with the schema stored inside, so a query reading 2 of 50 columns reads ~4% of the data. `df.write.parquet("s3a://bucket/dimuon/", partitionBy=["sample"])` then `spark.read.parquet(...)`.
- **Partitioned storage** (`partitionBy`) lets Spark skip whole directories when you filter on that column (*partition pruning*).
- `spark.sql(...)` and the DataFrame API are interchangeable, so data teams often expose tables to SQL users through a metastore (Hive, Unity Catalog) on top of the same files.
- UDF cost scales with data: a Python UDF that is “a bit slower” on this sample can dominate the bill on terabytes. Check the physical plan with `df.explain()`.

💡 Takeaways

- DataFrames = RDD + schema; the schema is what allows optimisation.
- Filters can be written as pandas-style indexing, **where/filter** or SQL: same plan, so pick the most readable.
- Nested data → **select** + **withColumn** + **drop** to flatten.
- Prefer native column functions > pandas UDFs > Python UDFs.
- Aggregate on the cluster, then **toPandas()** or **histogram** only the small result.

18. Lab: Spark 3, Structured Streaming

Lab at a glance

- **Notebook:** `code/spark/notebooks/Lecture3.ipynb` (solutions: `Lecture3_solutions.ipynb`; data generator: `code/spark/notebooks/utils/producer.py`)
- **Stack:** same as Spark 1
- **Theory:** Apache Spark → Real-time analytics and Structured Streaming
- **You will:** read a live stream of fake credit-card transactions from a TCP socket, parse it, and flag suspicious users batch by batch

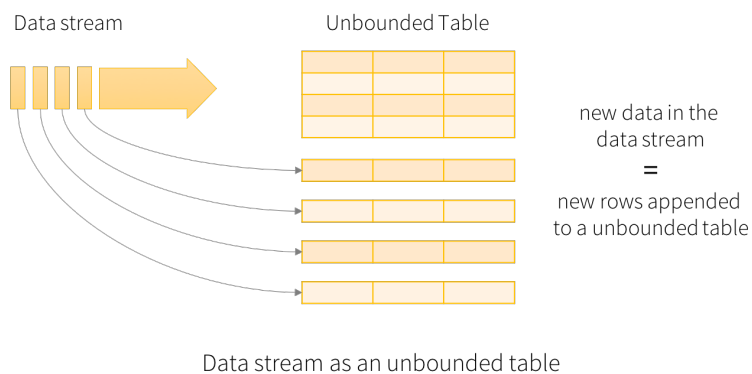
18.1. The idea behind it

- **Batch vs stream.** Batch processing has all the data at the start; streaming processes **unbounded** data as it arrives, with low latency. Many real systems need both: a stream for fresh results, plus batch jobs that reprocess history (Real-time analytics).
- **Micro-batching.** Spark turns a stream into a sequence of small batch jobs (one per trigger). You reuse the whole batch engine, at the price of latency of the order of the trigger interval, which is fine for most monitoring, not for microsecond trading.
- **State and fault tolerance.** Aggregations over a stream must remember past batches (running counts). That state, plus the position in the input, has to survive crashes, which is what **checkpoints** and *replayable* sources are for. The TCP socket used here is not replayable, which is why Spark calls it test-only.

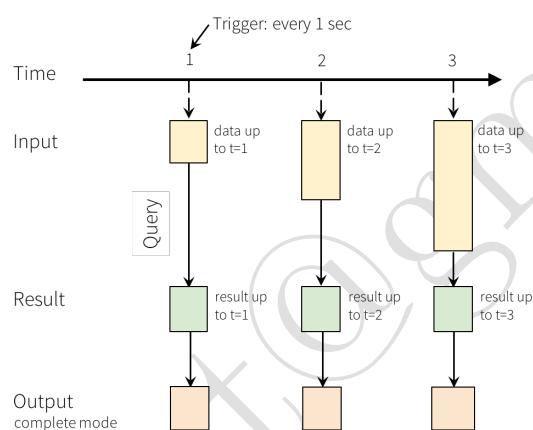
18.2. The model: a stream is a table that keeps growing

Structured Streaming lets you write **the same DataFrame code as in Spark 2**, and Spark runs it repeatedly on new data.

18. Lab: Spark 3, Structured Streaming



- New records are appended as new rows of an (unbounded) **input table**.
- Every **trigger** (e.g. every 2 s) processes the new rows as a *micro-batch* and updates a **result table**.
- An **output mode** decides what is written to the **sink** each time.



Spark never stores the whole input table: each micro-batch is processed and dropped, and only the state needed for the result (e.g. running counts) is kept.

Output mode	Writes each trigger	Allowed for
append	only new result rows	queries without aggregation
update	only result rows that changed	most queries; used in this lab
complete	the whole result table	aggregations only

The overall pattern is **source** → **transformations** → **sink**. This lab uses a TCP socket as source and the console as sink; the **Kafka lab** replaces both with Kafka. Reading, cleaning and writing a stream continuously is called a *real-time ETL* pipeline (Extract, Transform, Load).

(The older RDD-based *Spark Streaming*, with DStreams, is deprecated; ignore it.)

18.3. Session

Same as before, plus settings that keep streaming demos predictable on a tiny cluster:

```
.config("spark.sql.streaming.forceDeleteTempCheckpointLocation", "true")
.config("spark.sql.adaptive.enabled", "false")
...
spark.sparkContext.setLogLevel("ERROR")           # otherwise the console sink
↳ is buried in logs
```

18.4. The source: a socket and a producer

`producer.py` opens a server socket on `spark-master:5555`, waits for one client to connect, then sends a JSON line every 0.2 s:

```
{"name": "Jill", "surname": "Millers", "amount": 736.56, "delta_t": 7.78,
  ↳ "flag": 0}
```

`flag` is 1 with probability 0.2 (a “suspicious” transaction). Read the script before running it, since it is short.

It must run **inside the cluster network**, so start it in the master container:

```
cd code/spark
docker compose exec spark-master bash
python /mapd-workspace/notebooks/utils/producer.py --hostname spark-master
```

It exits when the streaming query disconnects, so **restart the producer before every new query**.

! Where is the output?

The console sink prints on the **driver’s stdout**, i.e. in the Jupyter container’s log, not in the notebook cell. Follow it with `docker compose logs -f jupyter-notebook`. The cell that calls `.start()` returns immediately: the query keeps running in the background until `query.stop()`.

18.5. First query: raw lines to the console

```
raw = (spark.readStream.format("socket")
      .option("host", "spark-master").option("port", 5555).load())

query = (raw.writeStream.outputMode("update").format("console"))
```

```

        .trigger(processingTime='2 seconds').option("truncate",
        ↪ False).start())
# ... watch the log ...
query.stop()

```

Note the symmetry with batch code: `read` $\rightarrow$ `readStream`, `write` $\rightarrow$ `writeStream`. Spark warns that the socket source is for testing only, because it cannot replay data after a failure.

`raw` has a single string column value: Spark knows nothing about the JSON inside it yet.

18.6. Develop on static data, then go live

Debugging a stream is painful, so the notebook first builds the parsing on a **static** `DataFrame` with the same shape (one string column of JSON lines):

```

from pyspark.sql.types import StringType, StructType, StructField,
    ↪ DoubleType, IntegerType
import pyspark.sql.functions as f

testDf = spark.createDataFrame(testData, StringType())

schema = StructType([StructField("name", StringType()),
    ↪ StructField("surname", StringType()),
                        StructField("amount", DoubleType()),
    ↪ StructField("delta_t", DoubleType()),
                        StructField("flag", IntegerType())])

parsed = testDf.select(f.from_json('value',
    ↪ schema=schema).alias('data')).select('data.*')
parsed.show()

```

`from_json` turns the string into one nested struct column; `.alias('data')` names it, and `select('data.*')` flattens it into five columns (the same flattening idea as in Spark 2).

Once it works, **the identical code runs on the stream**: swap `testDf` for `raw`. This is the main selling point of Structured Streaming.

18.7. The fraud detector

Per micro-batch: count flagged transactions per user, and report users with more than one.

```

numTransactions = (parsed.where(f.col('flag') == 1)
                    .withColumn('id', f.concat(f.col('name'),
    ↪ f.col('surname'))))

```

```

        .groupBy('id').count())

susTransactions = (numTransactions.where(f.col('count') > 1)
                        .withColumn('fraud', f.lit(1))          #
    ↪ constant column
                        .select('id', 'fraud',
    ↪ f.col('count').alias('nTransactions'))))

spark.conf.set("spark.sql.shuffle.partitions", 4)  # default is 200: far
    ↪ too many for this cluster

query = (susTransactions.writeStream.outputMode("update").format("console")
        .trigger(processingTime='5
    ↪ seconds').option("truncate", False).start())

```

Two things to notice:

- `groupBy` in a stream is **stateful**: counts accumulate across batches (Spark keeps the running state). In `update` mode you see only the users whose count changed in the last trigger.
- Re-run with `outputMode("complete")`: now every trigger prints the whole table. `complete` is only allowed because the query aggregates.

The `shuffle.partitions` line matters: the `groupBy` shuffles into 200 partitions by default, which means 200 tiny tasks per micro-batch on a 1–2 core cluster, and each trigger becomes very slow.

18.8. On a real system

- The source would be a **durable, replayable log**, almost always Kafka (see the [Kafka lab](#)), so that after a crash Spark can re-read from the last checkpointed offset.
- The sink would be a database, files in object storage, or another Kafka topic, not the console. Always set `.option("checkpointLocation", "<reliable storage path>")`.
- Unbounded state (like the per-user counts here, which grow forever) is controlled with **event-time windows and watermarks**: `withWatermark("ts", "10 minutes").groupBy(window("ts", "5 minutes"), "id")`, which lets Spark drop state for windows that can no longer change.
- Streaming jobs run for days or months, so they are deployed with `spark-submit` and monitored (input rate vs processing rate: if processing is slower, the lag grows without bound).

💡 Takeaways

- A stream is a growing table; the query is ordinary DataFrame code, executed per micro-batch.
- Develop on a static sample with the same schema, then switch the source.
- Output modes: **append** (new rows), **update** (changed rows), **complete** (everything; aggregations only).

18. Lab: Spark 3, Structured Streaming

- Aggregations are stateful across batches.
- Console output goes to the driver log; queries run until `stop()`.

19. Lab: Dask 1, delayed and futures

Lab at a glance

- **Notebook:** `code/dask/notebooks/Lecture1.ipynb` (solutions: `Lecture1_solutions.ipynb`)
- **Stack:** `dask-scheduler` + `dask-worker` ($\times N$) + `jupyter-notebook`, see Running the labs
- **Theory:** Dask: task graphs, lazy evaluation, scheduler
- **You will:** turn ordinary Python functions into a task graph (lazy, **delayed**) or submit them straight to the cluster (eager, futures), and learn when parallelising does *not* pay

19.1. The idea behind it

- **Task graphs are the common language of parallel computing.** Spark's DAG, Dask's graph and a Makefile are the same idea: nodes are pieces of work, edges are dependencies, and anything without a path between them can run in parallel. The scheduler's job is to find that parallelism and place tasks on workers.
- **Granularity.** Every task has a fixed cost (scheduling, serialisation, network: $\sim 100 \mu\text{s}$ – 1 ms in Dask). Parallelism helps only when each task's work is much larger than that. Most of this lab's exercises are deliberately too fine-grained, so you can *see* the overhead win.
- **Lazy vs eager** are two styles of the same machinery: build the whole plan and let the scheduler optimise it (**delayed**), or start work immediately and keep results remote (futures), the way a batch system or a job queue behaves.

19.2. Before you start

```
cd code/dask
docker compose up --scale dask-worker=2
```

Tab	What it shows
localhost:1234	Jupyter
localhost:8787	Dask dashboard: task stream (one row per worker thread, one bar per task), progress, worker memory

Keep the **Task Stream** panel of the dashboard visible: it is the Dask equivalent of the Spark UI, and the clearest way to see whether tasks really run in parallel.

19.3. Dask in one paragraph

Dask is a **task scheduler written in Python**. You describe work as a graph of Python function calls, and the scheduler sends the calls to workers, respecting dependencies. On top of that it offers collections (**bags**, **arrays**, **DataFrames**) that generate those graphs for you. The ideas are the ones from Spark: partition the data, plan before executing, run close to the data. The difference is that everything, including the scheduler, is Python.

```
from dask.distributed import Client
client = Client('dask-scheduler:8786')      # service name + scheduler port
client                                     # shows workers, cores, memory,
    ↪ dashboard link
```

19.4. Lazy: delayed

Three toy functions that each `sleep(1)` stand in for “real work”:

```
x = increment(1); y = decrement(2); z = add(x, y)      # plain Python: ~3 s
```

Wrapping calls in `delayed` records them instead of running them:

```
from dask import delayed
x = delayed(increment)(1)
y = delayed(decrement)(2)
z = delayed(add)(x, y)      # instant: z is a *plan*, not a number
z.visualize()               # draws the graph (needs graphviz)
z.compute()                 # runs it: ~2 s
```

Why 2 s and not 3: the graph shows `increment` and `decrement` are independent, so they run **in parallel**, and `add` waits for both. Dask found the parallelism from the dependencies; you did not write any threading code.

- `delayed(f)(args)` = “call `f` later”. You can also decorate a function with `@dask.delayed`.
- `.visualize(optimize_graph=True)` shows the graph after Dask’s optimisations; for graphs this simple there is nothing to optimise.
- `.compute()` is **blocking**: it returns when the result is back in the notebook.

19.5. Eager: submit, map and futures

```
future = client.submit(increment, 1)    # starts running NOW on the cluster
future                                # status: pending → finished
client.gather(future)                  # bring the result to the notebook
```

A **future** is a handle to a result that is being computed, or already exists, **on a worker**. Data stays there until you **gather** it. You can pass futures as arguments to further **submits**, and Dask wires up the dependency without moving data through the notebook:

```
x = client.submit(increment, 1)
y = client.submit(decrement, 2)
total = client.submit(add, x, y)      # depends on x and y, which may not
    ↪ be ready yet
client.gather(total)
```

`client.map(f, list)` submits `f` once per element, which is the natural way to run the same function over many inputs (a parameter scan, say).

	delayed + compute	submit/map + gather
when it runs	when you call <code>compute()</code>	immediately
style	build a whole graph, then run it	fire tasks as you go
result	returned by <code>compute()</code>	stays on workers until <code>gather()</code>
analogy	Spark transformations + action	a batch system

`wait(futures)` blocks until futures are finished **without** moving results to the notebook; `gather` waits *and* transfers.

19.6. Exercises

Exercise 19.1

1. Parallel for loop. For each element of `[1..8]`: add a random number, multiply by 10; then sum everything. Do it with `delayed`, with `submit`, and with `map`.

Solution

```

results = [delayed(mult_ten)(delayed(add_rand)(x)) for x in data]
total = delayed(sum)(results)
total.compute()

# eager, element by element
total = client.submit(sum, [client.submit(mult_ten,
    ↪ client.submit(add_rand, x)) for x in data])
# eager, with map
total = client.submit(sum, client.map(mult_ten, client.map(add_rand,
    ↪ data)))
client.gather(total)

```

The graph is 8 independent chains joined by one `sum`: a pure “map then reduce”.

Exercise 19.2

2. Tree reduction. Sum a list pairwise (add sleeps 1 s) instead of left to right.

Solution

Keep the plain-Python pairwise loop and wrap only the call: `pair_sum = delayed(add)(L[i], L[i+1])`. The loop now only **builds** a binary tree of additions; `L[0].compute()` runs each level in parallel. 9 elements take $\lceil \log_2 9 \rceil = 4$ levels, so about 4 s, against ~11 s for the sequential loop (it makes $5 + 3 + 2 + 1$ calls, padding odd levels with `add(x, 0)`). This is exactly how distributed **reduce** works.

Exercise 19.3

3. Word count over ~8000 newsgroup documents.

Solution

```

total = delayed(sum)([delayed(count_words)(doc) for doc in dataset])
    ↪ # lazy
total = client.submit(sum, client.map(count_words, dataset))
    ↪ # eager

```

Compare the timings with the plain loop: the parallel versions are **not** faster. Counting words in one document takes microseconds, while scheduling a task takes hundreds of microseconds. Too many tiny tasks are the problem, and Dask bags (next lab) fix it by batching elements into partitions.

Exercise 19.4**4. Fibonacci with delayed.****Solution**

Each new element depends on the two previous ones, so the graph is a **chain**: `result.visualize(rankdir="LR")` shows no branch that could run in parallel. A delayed object can be indexed (`fibonacci[i]`) and that indexing becomes a task too. Dask gives no speed-up for inherently sequential algorithms.

Exercise 19.5**5–6. Monte Carlo integral and . One delayed (or submit) call per random point.****Solution**

These run **slower than plain Python**, which is the point of the exercise. 1,000–10,000 tasks of a few microseconds each are dominated by scheduling overhead (each task costs ~100s of μ s). The fix is coarser tasks: each task generates a *chunk* of points and returns a partial count.

With futures, random functions need `pure=False`:

```
futures = [client.submit(in_circle, pure=False) for _ in
            range(num_points)]
```

By default Dask assumes functions are *pure* (same inputs give the same output) and deduplicates identical calls by hashing them, so without `pure=False` all 10,000 calls collapse into **one** task and you get the same point 10,000 times.

19.7. On a real cluster

- The scheduler/worker split is the same; only the deployment changes: `dask-kubernetes` / Dask Gateway on a Kubernetes cluster, `dask-jobqueue` on a Slurm/PBS HPC cluster (each worker is a batch job), or a managed service such as Coiled.
- **Adaptive scaling** (`cluster.adapt(minimum=2, maximum=50)`) adds workers when tasks queue up and releases them when idle, which matters when you pay per VM-hour.
- The dashboard you used on `:8787` is the main debugging tool in production too: idle workers mean too little parallelism, many tiny tasks mean too fine granularity, and red transfer bars mean too much data movement.
- Futures are a good fit for **parameter scans and simulations** on HPC (thousands of independent runs), which is a very common scientific use of Dask.

19.8. Wrap up

`client.close()`, then `docker compose down`.

💡 Takeaways

- **delayed** builds a task graph from ordinary Python; the parallelism comes from the dependencies.
- Futures run immediately and keep results on the workers until **gather**.
- Task overhead is ~100 μ s: parallelise work that is much bigger than that, in chunks.
- Sequential algorithms (Fibonacci) stay sequential.
- Mark random functions **pure=False** when submitting them.

20. Lab: Dask 2, Bags

Lab at a glance

- **Notebook:** `code/dask/notebooks/Lecture2.ipynb` (solutions: `Lecture2_solutions.ipynb`)
- **Data:** `code/dask/notebooks/datasets/accounts_json/accounts.*.json.gz`
- **Stack:** same as Dask 1
- **Theory:** Dask; compare with RDDs in Spark 1
- **You will:** process semi-structured JSON with a bag, aggregate per key with `groupby` vs `foldby`, and turn the cleaned bag into a DataFrame

20.1. The idea behind it

- **Partition-level parallelism** fixes the overhead problem of Dask 1: one task processes a whole partition (thousands of records), so the per-task cost is amortised.
- **The shuffle is the expensive part of distributed computing.** Any “group by key” needs records with the same key on the same worker, so data crosses the network. `groupby` moves *all* records; `foldby` (like Spark’s `reduceByKey`, Hadoop’s *combiner*) first reduces inside each partition and moves only small partial results. Associativity of the reduction is what makes that legal.
- **Schema-on-read.** Semi-structured data (JSON, logs) has no schema until you impose one by parsing and flattening, which is the “T” of ETL.

20.2. What a bag is

A `dask.bag` is a **partitioned, lazy list of arbitrary Python objects**: the Dask counterpart of a Spark RDD. It is the right tool for data that is not a table yet: text lines, logs, nested JSON. It solves the overhead problem from Dask 1: a function is applied to a whole **partition** per task, not to one element per task.

```
import dask.bag as db
b = db.from_sequence(range(1, 11), npartitions=4)
b.take(3)                                # peek: computes only what is
    ↪ needed
```

From files, with one element per line and compression detected from the extension:

```
lines = db.read_text('datasets/accounts_json/accounts.*.json.gz',
    ↪ files_per_partition=4)
```

Paths are relative to the notebook (/mapd-workspace/notebooks in the container).

20.3. Map, filter, fold

```
c = b.filter(lambda n: n % 2 == 0).map(lambda x: x ** 2)    # lazy
c.visualize()                                              # one chain per
    ↪ partition
c.compute()                                              # [4, 16, 36, 64,
    ↪ 100]
```

The reduction is called **fold**, and it shows the two levels of a distributed reduce explicitly:

```
c.fold(binop=lambda acc, x: acc + x,          # reduce inside each partition
    combine=lambda a, b: a + b)              # combine the partial results
```

Partitions matter. Re-run with `npartitions=3` and compare the graphs. One partition means no parallelism; thousands of tiny partitions bring the scheduling overhead back. Aim for “a few partitions per core, each comfortably fitting in memory”.

20.3.1. Broadcast: scatter

The Dask version of Spark’s broadcast variables: send a read-only object (a lookup table, model parameters) to every worker **once**, then pass the future to tasks:

```
lut = client.scatter({0: "a", 1: "b", 2: "c"}, broadcast=True)
client.gather(client.map(lambda x, lut: f"{x} {lut[x % 3]}", range(20),
    ↪ lut=lut))
```

Without `broadcast=True` a single object goes to one worker (a list is spread round-robin), and the other workers fetch it when a task needs it.

20.4. Exploring the accounts data

Each line is a JSON document: `{"id": ..., "name": ..., "transactions": [{"transaction-id": ..., "amount": ...}, ...]}`.

```
import json
records = lines.map(json.loads)          # text → dicts
records.take(1)
```

The bag toolkit for nested data:

Method	Does
<code>map(f)</code>	apply <code>f</code> to each element
<code>filter(f)</code>	keep elements where <code>f</code> is true
<code>pluck(key)</code>	<code>element[key]</code> for each element (go one level deeper)
<code>flatten()</code>	a bag of lists → a bag of their items

Questions are answered as pipelines, building them up one step at a time with `.take(...)` to check each stage:

```
alice = records.filter(lambda r: r['name'] == 'Alice')

# average number of transactions per Alice account
alice.map(lambda r: len(r['transactions'])).mean().compute()

# average amount over all of Alice's transactions
alice.pluck('transactions').flatten().pluck('amount').mean().compute()
```

20.5. Per-key aggregation: `groupby` vs `foldby`

“How many accounts per name?” can be done in two ways.

`groupby` puts every element with the same key together, then you reduce each group:

```
records.groupby(lambda r: r['name']).starmap(lambda name, group: (name,
    ↪ len(group))).compute()
```

`starmap` unpacks the (key, group) tuples into two arguments. The cost is that **all records move across the network** so that each key’s records end up together (a full shuffle), only to be counted.

`foldby` reduces *while* grouping: each partition keeps a running value per key, and only those small partial results are combined:

```
from operator import add
records.foldby(key='name',
               binop=lambda count, _: count + 1, initial=0, # inside each
               ↪ partition
               combine=add, combine_initial=0).compute()    # across
               ↪ partitions
```

It is the same idea as Spark’s `reduceByKey/combineByKey`. Time both versions in the notebook: `foldby` wins, and the gap grows with the data. `binop(acc, element)` folds an element into the

accumulator; `combine(acc1, acc2)` merges two accumulators. For a count, the two are different functions, which is why `foldby` needs both.

Exercise 20.1

Total transferred amount per name, with `foldby`.

Solution

Pre-compute each account's total with `map`, then fold the totals:

```
def sum_amount(r):
    return {'name': r['name'], 'total': sum(t['amount'] for t in
        ↪ r['transactions'])}

records.map(sum_amount).foldby(key='name',
                               binop=lambda acc, r: acc + r['total'],
        ↪ initial=0,
                               combine=add,
        ↪ combine_initial=0).compute()
```

20.6. From bag to DataFrame

Bags are typically the **E** and **T** of an **ETL**: read messy input, clean and flatten it, then hand a table to something more efficient. `to_dataframe()` needs flat records with the same keys:

```
def flatten_account(r):
    return {'id': r['id'], 'name': r['name'],
        'first_transaction_id':
            ↪ r['transactions'][0]['transaction-id'],
        'first_transaction_amount': r['transactions'][0]['amount']}

ddf = records.map(flatten_account).to_dataframe()
ddf          # prints structure and npartitions only: no data is computed
ddf.head(10) # computes just enough to show 10 rows
```

Other exits from a bag: `to_textfiles`, `to_avro`, `to_delayed`.

20.7. On a real cluster

- Bags read the same storage as everything else: `db.read_text("s3://bucket/logs/*.json.gz")` (with `s3fs` installed), with one or a few files per partition. Large single files can be split with `blocksize=`, but **gzip files cannot be split** (the whole file is one partition), which is a common real-world bottleneck; prefer splittable formats or many medium-sized files.

- Typical production use is the **ingestion stage**: parse and clean raw logs/JSON with a bag, then `to_dataframe()` and write Parquet for the analysis stage.
- Broadcasting (`scatter(..., broadcast=True)`) matters more at scale: sending a 100 MB lookup table with every one of 10,000 tasks is 1 TB of network traffic.

💡 Takeaways

- Bags = RDDs for Python: lazy, partitioned, for semi-structured data.
- `map` / `filter` / `pluck` / `flatten` navigate nested records; build pipelines step by step with `take`.
- Prefer `foldby` (reduce while grouping) over `groupby` (shuffle everything first).
- Choose partitions deliberately: not 1, not thousands.
- Clean with a bag, then `to_dataframe()` for tabular work.

21. Lab: Dask 3, DataFrames

Lab at a glance

- **Notebook:** `code/dask/notebooks/Lecture3.ipynb` (solutions: `Lecture3_solutions.ipynb`)
- **Data:** `datasets/accounts_csv/`, `datasets/nycflights/`, `datasets/dimuon/` (all under `code/dask/notebooks/`)
- **Stack:** same as Dask 1
- **Theory:** Dask; compare with Spark 2
- **You will:** load many CSVs as one DataFrame, fix wrongly inferred types, see which pandas operations are cheap and which are expensive when distributed, share work between computations, and redo the dimuon analysis in Dask

21.1. The idea behind it

- **Horizontal partitioning of tables.** A large table is split by rows into chunks that each fit in one worker's memory; operations that work row-by-row or per-chunk parallelise perfectly, and operations that need a global view (sorting, global positions) don't.
- **The index as the partitioning key.** Dask can track the min/max index of each partition (its *divisions*). With a sorted index, lookups and joins on it touch only the relevant partitions, while on any other column they need a shuffle. This is the same idea as partition pruning in Spark and sharding by key in NoSQL databases.
- **Common sub-expression sharing:** merging graphs (`dask.compute(a, b)`) avoids recomputing shared steps, the same optimisation a SQL planner does.

21.2. What a Dask DataFrame is

A Dask DataFrame is **many pandas DataFrames**, the *partitions*, split along the index (row-wise, i.e. *horizontal* partitioning) and spread over the workers. Every operation you call is translated into the same pandas operation on each partition, plus whatever is needed to combine the results. The API is almost the pandas one, but the costs are different, and the lab is mostly about learning those costs.

21.3. Loading

In pandas, reading many files means a loop and `pd.concat` into one in-memory table. In Dask, a glob is enough:

```
import dask, dask.dataframe as dd
dask.config.set({'dataframe.query-planning': True})  # enable the query
                                                    ↪ optimiser (default in recent Dask)

df = dd.read_csv('datasets/accounts_csv/accounts/*.csv')
len(df)      # triggers work: len() of every partition, then a sum
df           # structure only: columns, dtypes, npartitions
df = df.repartition(npartitions=8)
```

By default you get roughly one partition per file, which is often not what you want. The number of partitions is **your** responsibility, as in Dask 2.

Unlike Spark, Dask DataFrames historically did **no query optimisation**: you got exactly the operations you wrote, in that order. The newer query planner adds some (e.g. pushing column selections and filters down), which is why the notebook makes sure it is on.

21.3.1. Type inference pitfall

Dask infers dtypes from a **sample at the start** of the data, then enforces them on all partitions. In `nycflights`, `CRSElapsedTime` and `TailNum` look like integers in the first rows but are not later, so `head()` works and `tail()` (a different partition) fails.

Fix it by stating the types:

```
df = dd.read_csv('datasets/nycflights/*.csv',
                 dtype={'TailNum': str, 'CRSElapsedTime': float})
```

Alternatives are a larger `sample=` or `assume_missing=True` (treat integer columns as floats, so they may contain NaN), but explicit dtypes are the robust fix.

Building a date column from three columns also needs `meta`, which tells Dask the output type without computing anything:

```
df['Date'] = dd.to_datetime(df.rename(columns={'Year': 'year', 'Month':
                                                    ↪ 'month', 'DayofMonth': 'day'}))
                                     [['year', 'month', 'day']],
                                     meta=pd.Series(dtype="datetime64[ns]"))
```

21.4. Cheap and expensive operations

Cheap (each partition works alone, then small results are combined):

```

df.DepDelay.max().compute()
df[df.UniqueCarrier == 'UA'].ArrDelay.mean().compute()
df.groupby(['Origin', 'Dest']).AirTime.max().compute()
df.groupby(['Origin', 'Dest'])['AirTime'].aggregate(['mean',
    ↪ 'std']).compute()
df.loc[df['Dest'] == 'DEN', ['UniqueCarrier']].compute()

```

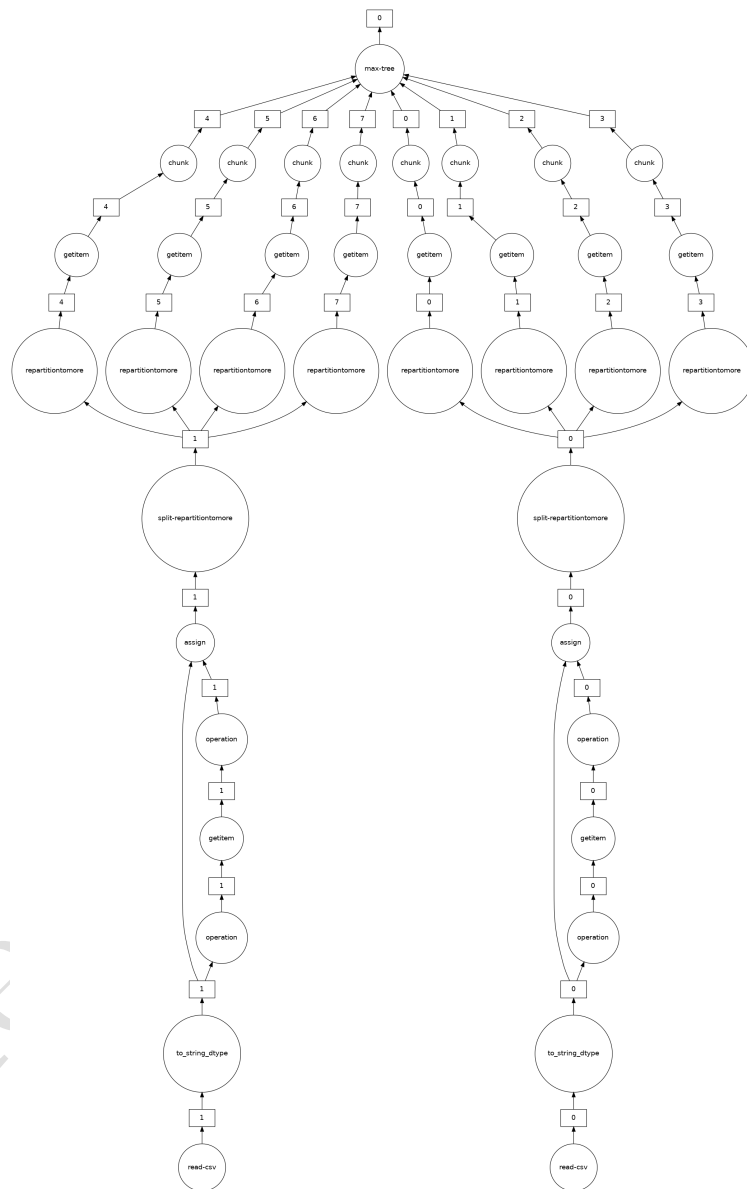


Figure 21.1.: Task graph of `max()`: a max per partition, then a max of maxes.

Group-by aggregations are still reasonably fast: partial aggregates per partition, then a combine. The result lands in **one** partition; if it is large, `split_out=` spreads it.

Not available or expensive:

Operation	Why
<code>df.iloc[0:10]</code> (row positions)	raises: partitions don't know their global row numbers. <code>iloc[:, cols]</code> (columns only) works
<code>set_index(col)</code>	needs a full sort of the data across workers
<code>sort_values</code> , joins on a non-index column	full shuffle
<code>df.compute()</code> on everything	brings all data to the client, so only after reducing it

The typical workflow is therefore: **bag** to clean the raw input, **Dask DataFrame** to filter and reduce the bulk, then **pandas** (`.compute()`) on the small result for anything order-dependent or fiddly.

After an unavoidable expensive step, persist the result so later operations don't repeat it:

```
ua_den = df[(df.UniqueCarrier == 'UA') & (df.Dest ==
    ↪ 'DEN')].set_index('Date')    # sort happens here
ua_den.ArrDelay.rolling('5D').mean().compute()    # time-based rolling
    ↪ needs the datetime index
```

21.5. Share work: one `dask.compute` for several results

```
df = df.persist()    # keep the parsed data in worker
    ↪ memory (not in place!)
non_cancelled = df[df.Cancelled == 0]
mean_delay = non_cancelled.DepDelay.mean()
std_delay = non_cancelled.DepDelay.std()

mean_delay.compute(); std_delay.compute()    # two graphs, common
    ↪ steps done twice
dask.compute(mean_delay, std_delay)    # one merged graph:
    ↪ ~half the time
```

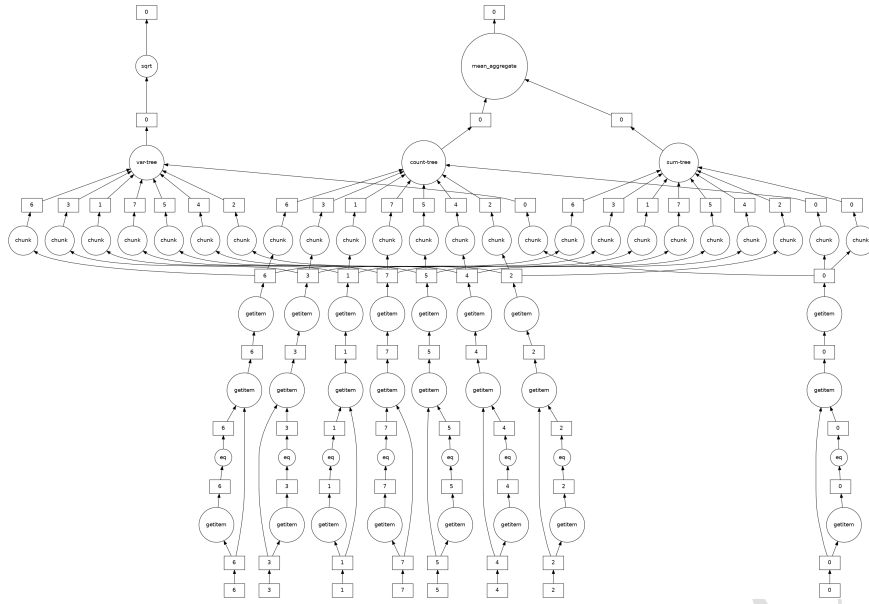


Figure 21.2.: Merged graph: reading, filtering and the shared sums/counts are computed once and feed both results.

21.6. Exercise: the dimuon spectrum in Dask

The same analysis as Spark 2, built from the Dask pieces. The JSON files are already in `code/dask/notebooks/datasets/dimuon` in the fork; the notebook's instruction to copy them over from `spark/` is only needed on a fresh upstream checkout.

1. **Read and parse with a bag:** each file is one JSON *array* of events, so parse and flatten:

```
events = db.read_text('datasets/dimuon/*.json',
    ↪ files_per_partition=4).map(json.loads).flatten()
```

2. **Events per sample** with `foldby` (count per key, as in Dask 2):

```
events.foldby(key='sample', binop=lambda n, _: n + 1, initial=0,
    ↪ combine=add, combine_initial=0).compute()
```

3. **Muon multiplicity** per sample: filter on `sample`, `pluck('nMuon')`, `compute()`, histogram in `matplotlib`.
4. **Select and flatten to a DataFrame:** two muons, opposite charge, then one flat dict per event:

```
df = (events.filter(lambda e: e['nMuon'] == 2)
    .filter(lambda e: e['Muons'][0]['charge'] ==
    ↪ -e['Muons'][1]['charge']))
```

```

        .map(flatten_records)           # {'sample', 'E1', 'px1',
    ↪ ..., 'pz2'}
        .to_dataframe()

```

5. Features are plain pandas-style column arithmetic (lazy):

```

df['pT1'] = np.sqrt(df.px1**2 + df.py1**2); df['pT2'] =
    ↪ np.sqrt(df.px2**2 + df.py2**2)
df = df[(df.pT1 > 15) & (df.pT2 > 15)]
df['E'] = df.E1 + df.E2; df['Px'] = df.px1 + df.px2 # ... Py, Pz
df['M'] = np.sqrt(df.E**2 - (df.Px**2 + df.Py**2 + df.Pz**2))

```

6. Reduce, then go to pandas for plotting: `df[['sample', 'M', 'pT1', 'pT2']].compute()`.

Compared with Spark: no UDF machinery, since numpy works directly on the columns because every partition *is* a pandas DataFrame.

21.7. On a real cluster

- Use **Parquet** on object storage: `dd.read_parquet("s3://bucket/flights/", columns=[...], filters=[("Year", ">=", 2000)])` reads only the needed columns and row groups, and the stored schema removes the dtype-inference problem entirely.
- Set the index **once** (expensive) and save the result partitioned by it, so later jobs get the divisions for free.
- Size partitions to roughly 100 MB–1 GB in memory; check with `df.memory_usage_per_partition().compute()`. Workers that run out of memory spill to disk and slow down dramatically.
- The “bag → Dask DataFrame → pandas” pattern is how most Python data platforms are built: distributed tools for the heavy reduction, single-machine tools for the final, small result.

21.8. Wrap up

`client.close()`, then `docker compose down`.

💡 Takeaways

- Dask DataFrame = row-wise partitions that are each a pandas DataFrame.
- State dtypes explicitly; inference only looks at the beginning of the data.
- Per-partition operations and group-by aggregations are cheap; `iloc`, `set_index`, sorting and non-index joins are not.
- `persist` after expensive steps; compute related results together with `dask.compute(...)`.
- Bag (clean) → Dask DataFrame (reduce) → pandas (finish).

22. Lab: Kafka, and Kafka + Spark

Lab at a glance

- **Notebooks** (in `code/kafka/notebooks/`, used *together*, in this order):
 1. `Producer.ipynb`: topics and producing messages
 2. `Consumer.ipynb`: reading them back, partitions, consumer groups
 3. `ConsumerGroup.ipynb`: a second consumer, run alongside notebook 2
 4. `KafkaSpark_StructuredStreaming.ipynb`: Spark reads from and writes to Kafka
- **Stack**: the Spark stack plus `kafka-broker` (one broker, no replication), see Running the labs
- **Theory**: Apache Kafka: topics, partitions, offsets, consumer groups; Spark Structured Streaming
- **You will**: publish and consume messages from the shell and from Python, see how partitions and consumer groups split the work, and rebuild the fraud detector with Kafka as source and sink

22.1. The idea behind it

- **Decoupling with a log**. Producers and consumers never know about each other; the broker stores an ordered, durable log in between. New consumers can be added later and replay history, and a slow consumer does not slow down producers. This is the backbone of *event-driven* architectures.
- **Partitions are the unit of parallelism and ordering**, as in Spark: more partitions allow more consumers in parallel, and order is guaranteed only inside one partition. The key decides the partition, so you choose the key for the ordering you need (e.g. per user).
- **Replication for fault tolerance**. Each partition has a leader and followers on other brokers (theory). This lab uses one broker, so `replication_factor=1`, i.e. no fault tolerance at all.
- **Delivery semantics**. At-most-once, at-least-once or exactly-once depend on *when* offsets are committed relative to processing, and on whether writes are idempotent.

22.2. Before you start

```
cd code/kafka
docker compose up --scale spark-worker=2      # the README advises at most 2
↪ workers
```

The broker is reachable **inside** the Docker network as `kafka-broker:9092`; it is not published to your laptop, so everything (shell tools, Python clients, Spark) runs in a container.

Notebook 1 installs the client library with `!pip install kafka-python`. It lives in the container, so you need to repeat it after every `docker compose down`.

22.3. Kafka in one picture

Kafka is a **log**, not a queue. A *topic* is split into *partitions*; each partition is an append-only, ordered list of records, and each record gets a position, the *offset*. Producers append to the end. Consumers read from wherever they like and remember their offset. **Reading does not delete anything**, so many independent consumers can read the same data, at different speeds, now or later. Producers and consumers never talk to each other, only to the broker. That decoupling is the whole point.

22.4. Part 1: from the shell (on the broker)

```
docker compose exec kafka-broker bash
cd $KAFKA_HOME/bin
./kafka-topics.sh --list      --bootstrap-server kafka-broker:9092
./kafka-topics.sh --create    --topic my_awesome_topic --bootstrap-server
↪ kafka-broker:9092
./kafka-topics.sh --describe --topic my_awesome_topic --bootstrap-server
↪ kafka-broker:9092
```

Then open **two** shells on the broker, one for each side:

```
./kafka-console-producer.sh --topic my_awesome_topic --bootstrap-server
↪ kafka-broker:9092
./kafka-console-consumer.sh --topic my_awesome_topic --bootstrap-server
↪ kafka-broker:9092 --from-beginning
```

Type in the producer and lines appear in the consumer. Start the consumer *without* `--from-beginning` and it only shows new messages; *with* it, it replays everything. The data was kept even though nobody was reading.

`--bootstrap-server` is just an entry point: the client asks that broker for the cluster layout, then talks to whichever broker leads each partition.

22.5. Part 2: producing from Python (Producer.ipynb)

```
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers=['kafka-broker:9092'])

producer.send('my_awesome_topic', b'message 1')    # returns a future:
    ↪ queued, NOT sent yet
producer.flush()                                # force the queue out now
```

`send` is **asynchronous**: messages are buffered and sent in batches, which is how Kafka reaches high throughput. If a script exits right after `send` without `flush()` (or `close()`), the messages may never leave. Values (and keys) are **bytes**: you choose the serialisation, e.g. `json.dumps(d).encode()`.

Close the producer and create a new one: the topic and its messages are unaffected, because producers are disposable.

Keys. `producer.send(topic, key=b'user42', value=...)`. The key is hashed to choose the partition, so **all messages with the same key go to the same partition, in order**. Without a key, the producer spreads messages over partitions.

Admin API. Creating topics from Python, with a chosen number of partitions:

```
from kafka.admin import KafkaAdminClient, NewTopic
admin = KafkaAdminClient(bootstrap_servers=['kafka-broker:9092'])
admin.create_topics([NewTopic(name='a_partitioned_topic', num_partitions=2,
    ↪ replication_factor=1)])
admin.list_topics()
```

`replication_factor=1` because there is a single broker; on a real cluster you would use 3. The notebook ends by sending 20 fake transactions (the JSON from the Spark streaming lab) to `a_partitioned_topic`, and creating a `results` topic for later.

22.6. Part 3: consuming (Consumer.ipynb)

```
from kafka import KafkaConsumer
consumer = KafkaConsumer(bootstrap_servers=['kafka-broker:9092'],
    consumer_timeout_ms=10000)    # stop iterating after
    ↪ 10 s without messages
consumer.subscribe(['my_awesome_topic'])
for msg in consumer:                # polls the broker
    print(msg.partition, msg.offset, msg.key, msg.value)
```

Without `consumer_timeout_ms` the `for` loop never ends (a consumer waits for new data forever), so interrupt the kernel to stop it.

22. Lab: Kafka, and Kafka + Spark

Each record carries its **partition**, **offset**, **timestamp**, **key** and **value**. `consumer.seek_to_beginning()` moves the offset back to 0 and the topic is read again, the Python version of `--from-beginning`.

On a `partitioned_topic` you see messages from partitions 0 and 1 interleaved: **order is guaranteed only within a partition**, never across the topic.

Reading one partition only. Instead of `subscribe` (let Kafka decide), `assign` picks partitions by hand:

```
from kafka import TopicPartition
c0 = KafkaConsumer(bootstrap_servers=['kafka-broker:9092'],
    ↪ consumer_timeout_ms=10000)
c0.assign([TopicPartition('a_partitioned_topic', 0)])
c0.seek_to_beginning()
```

22.7. Part 4: consumer groups (Consumer.ipynb + ConsumerGroup.ipynb)

Consumers that share a `group_id` **split the partitions** of a topic between them: each partition is read by exactly one member of the group. This is how Kafka scales reading.

```
consumer_one = KafkaConsumer(bootstrap_servers=['kafka-broker:9092'],
    client_id='consumer_one', group_id='my_group',
    auto_offset_reset='earliest',
    ↪ consumer_timeout_ms=10000)
consumer_one.subscribe(['a_partitioned_topic'])
```

Create `consumer_two` with the same `group_id` in `ConsumerGroup.ipynb`, then run both read loops **at the same time** (they are separate kernels, i.e. separate processes):

- `consumer_one.assignment()` and `consumer_two.assignment()` show **one partition each**: the group rebalanced.
- Each prints only its partition's messages.
- Stop one, and after a rebalance the other takes over both partitions.
- A consumer in a *different* group would get **all** messages again: groups are independent readers.

`auto_offset_reset='earliest'` applies only when the group has no stored offset yet (start from the beginning instead of only new messages). Afterwards the group's committed offsets win, so a second run continues where the group stopped.

22.8. Part 5: Kafka as Spark source and sink (KafkaSparkStructuredStreaming.ipynb)

The fraud detector from Spark 3, with the socket replaced by Kafka on both ends:

Producer.ipynb topic a_partitioned_topic Spark (parse, count flags per user) topic re

The Kafka connector is not part of core Spark; it is downloaded when the session starts:

```
.config("spark.jars.packages",
  ↪ "org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1")
.config("spark.sql.shuffle.partitions", 4)
```

Source. Same readStream, different format:

```
raw = (spark.readStream.format("kafka")
      .option("kafka.bootstrap.servers", "kafka-broker:9092")
      .option("subscribe", "a_partitioned_topic")
      .option("startingOffsets", "latest")           # only messages
      ↪ arriving from now on
      .load())
raw.printSchema()    # key, value (binary), topic, partition, offset,
      ↪ timestamp, ...
```

Kafka rows are binary key/value plus metadata, so the value is cast to string before parsing the JSON:

```
parsed = raw.select(from_json(col("value").cast("string"),
  ↪ schema).alias("data")).select("data.*")
```

Because of `startingOffsets = latest`, re-run the sending cell in `Producer.ipynb` after starting the query, or you will see nothing. The console output appears in the Jupyter container log, as in Spark 3.

Transform. Identical to Spark 3, except that the output must look like a Kafka record: a key and a value column, both strings or bytes:

```
sus = (parsed.where(col('flag') == 1)
      .withColumn('id', concat(col('name'), col('surname'))))
      .groupBy('id').count().where(col('count') > 1)
      .withColumn('fraud', lit(1))
      .select(col('id').alias('key'),
      ↪ col('fraud').cast(StringType()).alias('value'))
```

Sink.

```
query = (sus.writeStream.outputMode("update").format("kafka")
      .option("kafka.bootstrap.servers", "kafka-broker:9092")
      .option("topic", "results"))
```

```

        .option("checkpointLocation", "checkpoint")    # required for the
        ↪ Kafka sink
        .start()

```

The **checkpoint** directory is where Spark records which input offsets it has processed and its aggregation state, so a restarted query continues instead of starting over. It is created in the notebooks folder; delete it to start from scratch.

Finally, the last section of `Consumer.ipynb` subscribes to **results** and prints an “alert” for each flagged user: a third, independent application reacting to Spark’s output. For end-to-end **exactly-once** behaviour you would also need idempotent or transactional writes on the sink side (see the theory); this lab’s pipeline is at-least-once.

22.9. On a real cluster

- **At least 3 brokers**, topics with `replication_factor=3`, and producers with `acks="all"` (a write is confirmed only once in-sync replicas have it), together with `min.insync.replicas=2`. This configuration survives losing a broker without losing data.
- **Security**: the commented-out producer/consumer examples in the notebooks (`security_protocol="SSL", ssl_cafile=...`) are what a real cluster looks like: TLS plus client certificates or SASL credentials.
- **Retention**: logs are deleted after a time or size limit (`retention.ms`), or *compacted* to keep the last value per key. Kafka is a buffer and a history, not a permanent archive.
- **Consumer lag** (latest offset — committed offset per partition) is the key health metric: if it grows, consumers can’t keep up, so add consumers (up to the number of partitions) or partitions.
- Managed Kafka (AWS MSK, Confluent Cloud) handles brokers and upgrades; the client code is the same as in the notebooks, with different `bootstrap_servers` and credentials.

22.10. Wrap up

`query.stop()`, `spark.stop()`, then `docker compose down`.

💡 Takeaways

- A topic is a set of append-only, partitioned logs; reading does not consume, and consumers track offsets.
- `send` is asynchronous: `flush()` before exiting.
- Same key → same partition → ordered; no global order across partitions.
- Within a group, partitions are shared out (scaling); across groups, everyone gets everything (fan-out).
- Spark reads Kafka with `readStream.format("kafka")` and writes back with key/value columns and a checkpoint.